

**VIETNAM NATIONAL UNIVERSITY HO CHI MINH CITY
HO CHI MINH CITY UNIVERSITY OF TECHNOLOGY
FACULTY OF COMPUTER SCIENCE AND ENGINEERING**



**REPORT
CAPSTONE PROJECT
SEMESTER 252 ACADEMIC YEAR 2025-2026**

**RESEARCH AND DEVELOP A SPECIALIZED
LARGE LANGUAGE MODEL FOR ENHANCING
WEB APPLICATION FIREWALLS**

MAJOR: COMPUTER SCIENCE

COUNCIL: COMPUTER SCIENCE COMMITTEE

SUPERVISOR(s): DR. NGUYEN AN KHUONG PhD.
MR. TRAN LE QUOC KHANH B.Eng.

OPPONENT: ...

—o0o—

STUDENT 1: DOAN THE ANH - 2252019

STUDENT 2: NGUYEN BA NHAT QUANG - 2252669

STUDENT 3: NGUYEN ANH KIET - 2211758

HO CHI MINH CITY, MAY 2026

Instructor's Signature

_____ Date: _____

Dr.Nguyen An Khuong

Faculty of Computer Science and Engineering

Declaration of Authenticity

We declare that we solely conducted this specialized project, under the supervision of Dr. Nguyễn An Khương at the Faculty of Computer Science and Engineering, Vietnam National University - Ho Chi Minh City University of Technology.

We have taken care to properly acknowledge and document all external sources and references used in the project.

If there is any instance of plagiarism, we are ready to accept the consequences. Ho Chi Minh City University of Technology - Vietnam National University HCMC will not be held responsible for any copyright violations that may have occurred during my research.

Ho Chi Minh City, December 2025

Authors,

Acknowledgement

We would like to express our sincere gratitude to **Dr. Nguyễn An Khương**, our supervisor, for his patient guidance, insightful feedback, and continuous encouragement throughout this project. His direction has been essential in shaping our ideas and bringing this work to completion.

We would also like to thank **Mr. Trần Lê Quốc Khánh** for his valuable advice and practical suggestions. His perspectives from industry and experience in the field have helped us refine our approach and improve the quality of our work.

Our appreciation further extends to **Hồ Anh Dũng, Nguyễn Đăng Hiên** and **Lê Tiến Đạt** for their valuable support in **data preparation, LLM fine-tuning** and **clustering research**. Their contributions helped establish a solid experimental foundation and improved the overall quality of the training pipeline, creating strong momentum for the project's subsequent development stages.

Finally, we are deeply grateful to our families for their constant support and understanding. Their encouragement, patience, and belief in us have been a steady source of strength and motivation throughout our studies and the completion of this thesis.

Abstract

Web Application Firewalls (WAFs) remain one of the most widely deployed mechanisms for protecting web applications against common attack classes such as SQL injection and cross-site scripting. However, the practical effectiveness of a WAF depends heavily on the quality of its rule set, its ability to handle obfuscated or evolving attack patterns, and the expertise of the personnel responsible for configuring and maintaining it. In real-world environments, especially those with limited security resources, manual testing and rule tuning are often costly, inconsistent, and difficult to scale. This challenge creates a gap between increasingly adaptive offensive techniques and the defensive policies expected to stop them.

This thesis presents LLMShield as an LLM-assisted framework for automating WAF assessment and supporting rule-oriented defense improvement. The proposed system is designed to generate adversarial web payloads, execute them against protected targets, analyze the observed responses, and derive meaningful guidance for defensive refinement. At the core of the approach is the adaptation of lightweight large language models to the domain of web exploitation, with the objective of producing realistic and diverse payloads for SQL injection and cross-site scripting. The system further integrates rule-aware knowledge sources so that successful bypass cases can be interpreted in the context of WAF syntax and policy construction, thereby enabling more useful defensive feedback rather than merely reporting attack outcomes.

The development proceeds from a foundational research prototype to a broader multi-WAF framework. The initial stage establishes the end-to-end pipeline for payload generation, inference, testing, and analysis on a controlled vulnerable web environment. Building on this foundation, LLMShield generalizes the workflow into a reusable platform that supports multiple WAF technologies, including AWS WAF,

Cloudflare, ModSecurity, and NAXSI. In addition to offensive payload generation, the framework incorporates syntax validation, exploit verification, harmfulness screening, statistical aggregation, and defense-oriented processing components, together with both command-line and graphical interfaces to facilitate experimentation and practical use.

The evaluation pipeline is organized in multiple phases, beginning with payload generation and followed by testing against WAF-protected targets, iterative regeneration for harder bypass attempts, validation of payload safety and exploitability, and the compilation of consolidated experimental results. The benchmark covers major web attack categories, including classical SQL injection, blind SQL injection, reflected XSS, stored XSS, and DOM-based XSS. Through these experiments, the system is able to compare defensive behavior across heterogeneous WAF implementations and identify discrepancies in filtering effectiveness, rule coverage, and robustness against adversarially generated inputs.

The results indicate that LLM-generated payloads can reveal bypass opportunities and defensive blind spots that are not easily exposed through conventional manual testing alone. More importantly, the proposed approach demonstrates that large language models can be used not only as offensive content generators but also as practical assistants for structured security evaluation and rule-improvement workflows. Overall, this thesis contributes an end-to-end methodology for automated payload generation, cross-WAF benchmarking, and defense-aware analysis, thereby providing a useful foundation for future work on local defensive models, broader evaluation corpora, and more reliable automated rule synthesis for web application protection.

Table of Glossary

LLM	Large Language Model
OWASP	Open Web Application Security Project
RAG	Retrieval-Augmented Generation
SQLi	SQL Injection
XSS	Cross-Site Scripting
WAF	Web Application Firewall
DVWA	Damn Vulnerable Web App
AWS	Amazon Web Services
EC2	Elastic Compute Cloud (AWS Service)
ALB	Application Load Balancer
K8s	Kubernetes
NLP	Natural Language Processing
RegEx	Regular Expression
API	Application Programming Interface
JSON	JavaScript Object Notation
NAXSI	Nginx Anti XSS & SQL Injection
HTTP	Hypertext Transfer Protocol
CVE	Common Vulnerabilities and Exposures
PEFT	Parameter-Efficient Fine-Tuning

Contents

1	Introduction	1
1.1	Motivation	1
1.2	Scope and Goals	3
1.3	Challenges and Solutions	4
1.4	Project Structure	7
2	Theoretical Backgrounds	9
2.1	Firewalls	9
2.1.1	Concept and Security Role	9
2.1.2	Classification of Firewalls	10
2.1.3	Traffic Processing Architecture	12
2.1.4	Operational Limitations of WAFs	13
2.1.5	Relevance to LLM-Guided WAF Bypass	13
2.2	Web Application Vulnerabilities	14
2.2.1	SQL Injection (SQLi)	14
2.2.2	Cross-Site Scripting (XSS)	16
2.2.3	Relevance to WAF Evaluation and LLM-based Bypass	17
2.3	Large Language Models	18
2.3.1	Concept and Architecture Overview	18
2.3.2	Representation Learning and Embeddings	19
2.3.3	Prompt-based Generation and Transformation	20
2.3.4	Retrieval-Augmented Generation (RAG)	20
2.3.5	Adversarial Generation Capabilities	21
2.3.6	Limitations and Constraints	22

2.3.7	Relevance to WAF Bypass and Rule Improvement	23
2.3.8	Clustering of Adversarial Payloads	23
2.3.9	Abstract Syntax Trees (AST)	25
3	Related Tools and Works	27
3.1	Web Application Firewall (WAF)	27
3.1.1	Common Web Application Firewalls	27
3.1.2	LLM Application in Enhancing WAF Security	45
3.2	Vulnerability Research, Payload Datasets, and Validation Platforms . .	50
3.2.1	Dataset Ingestion Research	50
3.2.2	Retrieval-Augmented Generation for Security	51
3.2.3	DVWA as a validation platforms	51
3.2.4	LLM APIs for Rule Synthesis	52
3.3	Clustering, RAG and Defense Research	53
3.3.1	Defense Rule Generation Research	53
3.3.2	Clustering Payloads Resaearch	54
3.4	Supplementary tools	55
4	Proposed Approaches	59
4.1	Application Introduction	59
4.2	Requirements Overview	60
4.3	System Architecture	61
4.3.1	Overview Architecture	61
4.3.2	Preparation Phase Specification	63
4.3.3	Red Module Specification	64
4.3.4	Blue Module Specification	66
4.3.5	Clustering Specification	69
4.4	Large Language Model Architect	71
4.4.1	Fine-Tuning Specification	71
4.4.2	Retrieval-Augmented Generation Specification	75
5	Implementation	81
5.1	Data Preparation	81

5.1.1	Data Ingestion	81
5.1.2	Data Processing	84
5.2	Application Implementation	90
5.2.1	Technology Stack	90
5.2.2	User Interface Overview	91
5.2.3	Repository Structure	92
5.2.4	Main Functionality	94
5.3	Fine Tuning Implementation	102
5.3.1	Model Choice and Rationale	102
5.3.2	Technologies, Libraries, and Training Setup	102
5.3.3	Three-Phase Training Pipeline	103
5.4	Defense Pipeline Implementation	104
6	Experiments	108
6.1	Testing Approaches	108
6.2	Benchmark Building	113
6.2.1	Benchmark Architecture	113
6.2.2	Benchmark Implementation	115
6.3	Testing Conduction	118
6.3.1	Unit Testing of System Components	118
6.3.2	Use Case Testing: Payload Generation and WAF Bypass	119
6.3.3	Use Case Testing: Rule Generation and WAF Improvement	124
6.3.4	End to End Testing: Full Pipeline	126
7	Conclusion	133
	References	137

List of Tables

3.1	Common ModSecurity Directives	31
3.2	Commonly Used Variables in ModSecurity	32
3.3	Commonly Used Operators in ModSecurity	32
3.4	Commonly Used Transformation Functions	33
3.5	Commonly Used Actions in ModSecurity	33
3.6	General Structure of Cloudflare WAF Rules	37
3.7	General Structure of AWS WAF Rules	39
3.8	General Structure of a Naxsi Rule	41
3.9	Comparative Summary of Four Common WAF Solutions	42
5.1	Summary of Ingested Datasets	82
5.2	Blue-Side Documentation Sources for RAG	82
5.3	Fine-tuning technology stack and its role in the implementation.	103
6.1	End-to-end test targets	112
6.2	DNS and infrastructure mapping for the benchmark environment.	116
6.3	Unit Testing Checklist	119
6.4	Payload Generation and WAF Bypass Results Across WAF Solutions	120
6.5	Number of bypass payload clusters per WAF and attack type.	124
6.6	Defense rule generation pipeline results (GPT-5.4).	126
6.7	Rule Generation and WAF Improvement Results Across WAF Solutions	127

List of Figures

2.1	How does WAF work	12
2.2	How does SQLi work	14
2.3	How does XSS work	16
2.4	How does RAG work	20
2.5	How does Clustering work	23
2.6	How does AST tree work	25
3.1	HTTP processing phases in ModSecurity	28
3.2	GenSQLi pipeline	45
3.3	GenXSS pipeline	46
4.1	LLMShield Application Workflow	61
4.2	Flowchart of the Preparation Phase	64
4.3	Operational Workflow of the Red Module	65
4.4	Operational Workflow of the Blue Module	67
4.5	Proposed clustering workflow for bypassed payload grouping	71
4.6	LLMShield Fine-Tuning Architect Diagram	71
4.7	RAG System Architecture Diagram	77
5.1	Distribution of Attack Types in Phase 1	86
5.2	Payload Length Distribution in Phase 1	87
5.3	Histogram of Attack Techniques in Phase 1	87
5.4	Distribution of Attack Types in Phase 2	89
5.5	Payload Length Distribution in Phase 2	89
5.6	Distribution of Attack Techniques in Phase 2	90

5.7	WAF Detection Interface Showing Detected WAF Information	91
5.8	Attack Simulation Interface Showing Payload Execution Results	91
5.9	Defense Generation Interface with Automatically Generated WAF Rules	92
5.10	Source code organization of the proposed system.	93
5.11	Sequence diagram for WAF detection (/api/detect).	94
5.12	Sequence diagram for payload generation (/api/generate_payload).	96
5.13	Sequence diagram for attack testing (/api/test_attack).	98
5.14	Sequence diagram for defense rule generation (/api/defend).	100
5.15	LLMShield defense workflow	105
6.1	Benchmark architecture for evaluating payload bypass and WAF defense rule generation.	114
6.2	Visualization of payload execution results against ModSecurity across attack categories and generation phases	121
6.3	Visualization of payload execution results against Naxsi across attack categories and generation phases	122
6.4	Visualization of payload execution results against Cloudflare across attack categories and generation phases	122
6.5	Visualization of payload execution results against AWS WAF across attack categories and generation phases	123
6.6	Visualization of payload execution results against Cloudflare after rule generation and WAF improvement	128
6.7	Visualization of payload execution results against ModSecurity after rule generation and WAF improvement	129
6.8	Visualization of payload execution results against Naxsi after rule generation and WAF improvement	129
6.9	Visualization of payload execution results against AWS WAF after rule generation and WAF improvement	130

Chapter 1

Introduction

1.1 Motivation

Since computer technologies have been developed and widely accessible to humanity, many things have been digitized and moved online, including business activities like online shopping, banking, and finance, all of which have transitioned to the internet or, more specifically, web applications to operate, providing convenience for customers. However, this also brings significant security risks, as important assets of businesses and customers are exchanged through online transactions and activities. This has led to vulnerabilities such as Cross-Site Scripting (XSS), SQL Injection, Path Traversal, DDoS, etc., causing significant damage and enormous losses to people's digital assets. Therefore, with the development of websites and the increasing reliance of humans on them, security has always been a critical and urgent concern.

In addition to the fact that information security engineers must spend many hours carefully checking their source code for potential vulnerabilities, it was not until the early 1990s that the application firewall was defined by three names: Gene Spafford, Bill Cheswick, and Marcus Ran. This helped mitigate most of the vulnerabilities that web applications were susceptible to.

However, in today's era, cyberattacks have become more sophisticated than ever. According to the OWASP Top 10 report in 2025 [15], basic vulnerabilities like XSS and SQL Injection remain critical threats affecting web applications under A05:2025 - Injection. These vulnerabilities were found in 100% of tested applications, with 62,445

total CVEs reported. These vulnerabilities are now being exploited more extensively, such as through obfuscation and encryption that can bypass traditional firewalls. This requires firewalls to be increasingly advanced and capable of preventing such attacks. Some firewall providers, like Cloudflare with AWS WAF, have developed machine learning algorithms and AI applications to score and identify attacks effectively. These vendors frequently update their systems to keep up with the latest attack trends and ensure their customers' security.

However, there is another issue: services like WAF are too expensive for small and medium-sized companies with limited budgets, even though their risk of losing assets and being affected on the internet remains high. Moreover, even if they invest in a WAF, there are still many other costs a business needs to cover, such as staffing for maintenance and internal operations, infrastructure, and other information security solutions. Additionally, human expertise is also a concern, as cybersecurity engineers intern or fresher tend to specialize in only certain areas, like building WAF rules or performing WAF pentests. Few people have the necessary expertise to ensure all aspects are covered.

Furthermore, the rapid advancement of large language models (LLMs) in recent years has helped reduce the workload for people, allowing them to learn new things quickly thanks to their ability to understand semantics, their broad knowledge base, and their fast response capabilities. However, applying LLMs to WAF operations remains a relatively new concept, and there has been little research on this issue globally.

For these reasons, the project "Research and Development a Specialized Large Language Model for Enhancing Web Application Firewalls" was created to assist information security engineers in small and medium-sized enterprises by helping them operate and integrate WAF into their business workflows, maximizing security while saving time and, particularly, resources like manpower and finances. By leveraging the power of LLMs, it is possible to attack firewalls with attack payloads, and the successful attacks will generate rules to protect the firewall according to the type of firewall being attacked.

1.2 Scope and Goals

In this project, the team will conduct research and develop a tool capable of automatically generating payloads to attack Web Application Firewalls (WAFs) that exploit two common vulnerabilities: SQL Injection (SQLi) and Cross-Site Scripting (XSS). The tool will operate on a domain provided by the user and generate successful payloads, along with the corresponding rules for the specific WAF. Additionally, the tool will provide detailed instructions for security engineers to configure advanced firewall rules for their systems.

To accomplish the project, the team needs to undertake the following tasks:

- Research techniques for identifying the WAF of a domain based on the request/response patterns sent to the server.
- Develop and implement a tool, along with a smooth-functioning CLI dashboard, featuring two primary modules: attack and defense.
- Research and collect data on bypass attacks that have successfully evaded firewalls using SQLi and XSS vulnerabilities and the rule for protect WAF protect the WAF based on the success payloads.
- Investigate and design a specialized AI model for attacking and enhancing firewall defense mechanisms.

The tool developed by the student team must include the following core features, while also meeting specific requirements:

- The tool must assist security engineers in bypassing the firewall. A request must contain more than one successful payload for the first trial attempt, provided there have been no changes to the firewall configuration.
- The tool must be capable of helping engineers block malicious bypass attempts that have been flagged as successful in previous trials. By marking these bypass attempts as malicious, the tool can prevent them from being used in future attacks, ensuring enhanced security.

- The tool should offer a web self-host GUI that is easy to use and provides an intuitive experience for both beginners and experienced users. The CLI version should also be smooth and efficient for users who prefer working with command-line interfaces.

1.3 Challenges and Solutions

Reliability and Accuracy

Challenge: Ensuring reliability and accuracy in an LLM-based WAF system remains difficult because generated payloads may appear diverse but still be ineffective, meaningless, or easily detected by WAFs. This is especially challenging since different firewalls apply complex and non-standard filtering logic, making simple payload transformations unreliable. Similarly, rule generation is also difficult because each WAF uses its own syntax and configuration style, so the model must not only produce correct rules but also adapt them to the specific firewall while avoiding invalid or redundant outputs.

Solutions: To improve attack generation, candidate payloads will be tested before being used so that the model learns from effective and meaningful examples instead of trivial ones. In addition, harmless validation will be applied to improve accuracy, using Selenium to verify whether XSS payloads still execute correctly and AST-based analysis to check whether SQLi payloads preserve their intended structure and semantics. For rule generation, we will use modern API-based models combined with RAG over firewall documentation to provide vendor-specific guidance, while also integrating built-in syntax validation functions and comparison with existing rules to reduce invalid, duplicated, or redundant rule generation.

Ethical Considerations in Application Implementation

Challenge: Developing this tool and applying it in a way that prevents misuse by malicious hackers is extremely important. It is crucial to prevent malicious actors from using the tool to attack firewalls of essential platforms, such as government systems,

banking institutions, and corporate websites, in an illegal manner that could harm the financial interests of other parties.

Solution: The tool should require a license key that is verified and issued to employees of an authorized organization. The key should be encrypted with strong security measures to prevent counterfeiting by malicious hackers. Additionally, during the coding and testing phases, the GitHub repository should always be set to private mode.

Data Collection for the Attack Module

Challenge: The attack data, specifically payloads, have many existing sources. However, these payloads tend to be too generic and basic, often insufficient to bypass advanced firewalls. On the other hand, more complex payloads are rarely published on the internet due to ethical concerns and intellectual property rights. Few people share hacking methods with significant impact, such as bypassing firewalls, for free. If they do, firewall vendors often release patches with new rules to block them, and open-source firewall communities also quickly develop rules to address them.

Solutions: Collect a large amount of online datasets and use AI APIs to generate complex payloads to test whether they can bypass basic firewalls. Additionally, some of the most valuable data sources come from CTF writeups and blogs shared on hacking platforms. These publicly shared payloads can provide a rich data source for AI, enabling it to perform attacks on firewalls effectively.

Diversity of Application Firewalls

Challenge: Firewalls today are highly diverse, and many types are very expensive, making it impossible to use all of them with the highest features for testing. This creates a significant challenge in testing and developing attack tools, as it is not possible to fully access all advanced protective firewalls to test bypass capabilities.

Solutions: To ensure diversity while keeping the experiments practical, this study will focus on four representative WAFs: two widely used commercial solutions, *Cloudflare WAF* and *AWS WAF*, and two popular open-source solutions, *Naxsi* and *ModSecurity*. This selection provides a more balanced and comprehensive view of modern firewall ecosystems, covering both enterprise-grade and open-source

environments, while remaining feasible in terms of cost and deployment effort.

Large Language Model Training Resources

Challenge: Training large language models requires significant computational resources, including both hardware and financial costs. One of the major challenges is the lack of budget for expensive hardware, such as GPUs and TPUs, or the high cost of cloud computing services. This makes it difficult for organizations or individuals with limited resources to access and develop large language models.

Solutions: A feasible solution is to leverage affordable cloud-based computing resources from local providers. These services offer resource packages at reasonable costs, allowing model training without the need for large upfront hardware investments. Alternatively, using in-house computing resources in available lab spaces (e.g., Lab B9304) is another viable option, where internal computational resources can be used without the extra cost of external rental services. To optimize costs, smaller parameter models can be chosen, which reduce computational requirements while still achieving acceptable performance. By combining cost-effective cloud resource rentals with the use of smaller language models, training large language models becomes more feasible even with limited financial resources.

Source Code Organization

Challenge: Another challenge lies in the system design and source code organization. The initial architecture followed a monolithic approach, where all major services, including the backend, LLM, RAG, and clustering modules, were placed in a single component. This made the system heavy, difficult to run on personal machines, and inconvenient for development and debugging. Running the whole system on a VM was possible, but it significantly reduced flexibility and made debugging slower and more complex for team members.

Solutions: To address this issue, the system will be redesigned using a distributed architecture. Heavy services that require more computation and less frequent debugging, such as the LLM and RAG components, will be separated and deployed on a GPU-enabled VM. Meanwhile, the core logic, frontend, and backend will remain

in a lighter component that can run easily on each member's personal machine. This approach improves scalability, reduces local resource requirements, and makes development and debugging much more practical.

Documentation for RAG

Challenge: A major challenge in building the RAG component is the large volume of documentation related to WAFs, which makes data processing and retrieval expensive. In particular, semantic chunking can be highly resource-intensive when applied to long and complex technical documents, leading to increased processing time and system overhead without always providing proportional benefits.

Solutions: To make the RAG pipeline more efficient, the system will only retain documents that are truly relevant, especially rule documentation and rule configuration guides provided by WAF vendors. In addition, instead of using semantic chunking, the document preprocessing pipeline will adopt sliding window chunking combined with recursive character-based sentence splitting. This approach reduces computational cost while still preserving enough contextual continuity for effective retrieval.

1.4 Project Structure

The report is divided into 7 chapters, with a brief overview of each chapter as follows:

- **Chapter 1 - Introduction:** This chapter introduces the topic, explains the rationale behind the choice of the topic, defines the scope and objectives, discusses the challenges faced, and outlines the approach to solving the problems.
- **Chapter 2 - Theoretical Background:** This chapter presents the foundational knowledge related to the topic.
- **Chapter 3 - Related Tools and Works:** This chapter introduces, analyzes, and evaluates the commonly used tools for firewall detection, various types of firewalls, and research studies on using Large Language Models (LLMs) to attack websites.
- **Chapter 4 - Proposed Approaches:** This chapter analyzes and designs the architecture of the tool, outlines the requirements, and specifies the functions of

each module. It also discusses the overall system architecture and the architecture of individual components.

- **Chapter 5 - Implementations:** This chapter describes the process of implementing each component of the tool and the entire system.
- **Chapter 6 - Experiments:** This chapter presents the testing criteria, testing methods, and the results of the tool evaluation.
- **Chapter 7 - Conclusion:** This chapter summarizes the results and knowledge gained during the project and outlines future development directions for the tool.

Chapter 2

Theoretical Backgrounds

2.1 Firewalls

Firewalls are fundamental security enforcement components that regulate data flows between network or application endpoints. In web environments, firewalls serve as policy decision points responsible for validating, filtering, and constraining interactions between clients and servers. Their function extends beyond simple packet inspection: modern firewalls evaluate protocol semantics, request structure, and behavioral patterns to mitigate attacks that exploit weaknesses in web applications.

From the perspective of a Web Application Firewall (WAF), traffic inspection is performed after TLS termination, meaning all HTTP transactions are processed in a fully decoded and semantically interpretable form. This property differentiates WAFs from lower-layer firewalls and enables deeper analysis of application-layer threats.

2.1.1 Concept and Security Role

A firewall enforces security policies by evaluating incoming and outgoing traffic against predefined or dynamically derived conditions. These conditions may incorporate:

- **Structural constraints**, such as protocol compliance, header validity, parameter syntax, or message size.
- **Behavioral constraints**, such as frequency of access, sequence of interactions, or deviation from typical traffic patterns.

- **Semantic constraints**, such as input intent, contextual meaning, or encoded malicious logic.

In enterprise web architectures, firewalls form part of a layered defense strategy. They reduce the attack surface by blocking exploit attempts before they interact with backend services, and provide an intermediate control point that can observe, log, and enforce runtime security policies independently from the application codebase.

2.1.2 Classification of Firewalls

Firewalls can be classified in various ways depending on inspection depth, policy expressiveness, and deployment model. The categories most relevant to this project are described below.

Network Layer Firewalls. Network firewalls operate on OSI Layers 3–4 and enforce rules based on IP addresses, ports, and TCP/UDP semantics. Packet filtering and stateful inspection allow them to block unauthorized transports, detect spoofing attempts, and perform rate limiting. However, because traffic payloads are encrypted (HTTPS) or application semantics are opaque, they cannot reliably detect application-layer attacks such as SQL injection or cross-site scripting.

Application Layer Firewalls. Application-layer firewalls perform inspection at OSI Layer 7 and analyze full HTTP transaction elements such as request methods, URIs, headers, cookies, and payloads. Their rule systems allow:

- Validation of request structure (method constraints, URI whitelisting, content-type filtering).
- Enforcement of business logic invariants (operation limitations, session behaviors).
- Analysis of message content to identify potentially harmful input patterns.

Compared to network firewalls, application-layer firewalls offer deeper visibility but require more complex rule systems and consume more computational resources.

Web Application Firewalls (WAF). A Web Application Firewall specializes in detecting and mitigating threats that target web application logic. WAFs identify malicious payloads that exploit:

- Input validation weaknesses (SQLi, XSS, RCE).
- Protocol and encoding abuses (multi-encoding, invalid UTF-8, obfuscated payloads).
- Server error disclosure, backend misconfigurations, or unintended resource exposure.

WAFs implement several detection mechanisms, including:

Signature-based Detection The firewall compares traffic against known attack signatures such as regular expression patterns. While efficient, it is vulnerable to evasion through encoding or payload transformation.

Rule-based Analysis Rules express constraints on request structure, metadata, and payloads. These rules may use transformations, anomaly scoring, or phase-based execution, similar to the ModSecurity model used in this project.

Anomaly or Behavior-based Detection Some WAFs employ scoring models that accumulate evidence of suspicious behavior. Rather than one rule independently determining a violation, multiple indicators collectively contribute to a block decision.

Semantic or ML-driven Detection Recent WAF technologies incorporate machine learning to identify abnormal patterns or semantic equivalents of known attack payloads. In practice, such models augment rather than replace rule-based systems.

2.1.3 Traffic Processing Architecture

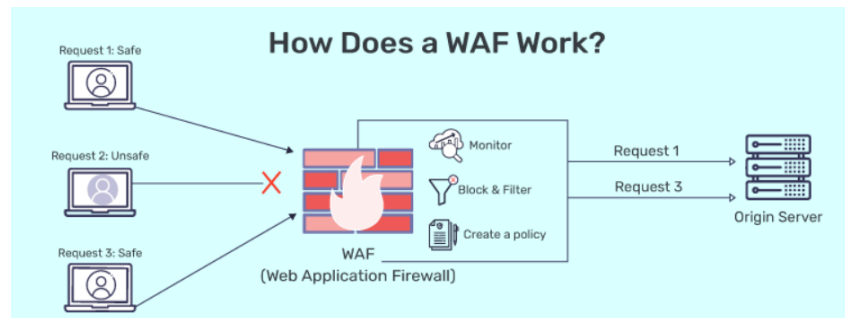


Figure 2.1: How does WAF work¹

In a typical reverse proxy configuration, the WAF resides between the client and the protected backend server. The WAF performs the following sequence:

1. **TLS termination:** Encrypted traffic is decrypted to obtain clear-text HTTP.
2. **Parsing and canonicalization:** URI normalization, parameter extraction, decoding, and content-type classification.
3. **Rule evaluation across phases:** Each phase inspects a specific component of the transaction (request headers, request body, response headers, response body).
4. **Action execution:** Depending on rule outcomes, the WAF may block, log, modify, or forward the request.
5. **Forwarding and response inspection:** Safe transactions are forwarded to the backend server; responses may also be inspected before being delivered to the client.

This architecture grants WAFs full visibility of both inbound and outbound data, enabling detection of outbound error leaks, debugging traces, or sensitive information disclosure.

¹Indusface. (n.d.). How a web application firewall (WAF) works. Retrieved from <https://www.indusface.com/blog/how-web-application-firewall-works/>

2.1.4 Operational Limitations of WAFs

Despite their importance, WAFs are not complete security solutions. Several inherent limitations persist:

- **Pattern rigidity:** Static signatures struggle against adversarial obfuscation, multi-layer encoding, and newly emerging attack forms.
- **Contextual blindness:** WAFs typically lack awareness of code-level semantics or business logic, enabling attackers to craft syntactically valid but harmful payloads.
- **Encoding and transformation attacks:** WAFs may misinterpret payloads under double-encoding, alternative unicode representations, or non-standard delimiters.
- **High false positives:** Strict rule sets may block legitimate users, requiring continuous tuning and exception crafting.
- **Performance cost:** Deep inspection, buffering, and complex rule evaluation can reduce throughput or increase latency in high-traffic environments.

2.1.5 Relevance to LLM-Guided WAF Bypass

Large Language Models introduce new challenges for traditional WAFs. Their ability to generate syntactically diverse, semantically equivalent attack payloads exposes WAF limitations in:

- Detecting non-standard input patterns,
- Handling novel transformations not covered in signature sets,
- Distinguishing benign user input from adversarially optimized text.

These challenges motivate the research direction of this project: leveraging LLM-driven bypass attempts to evaluate WAF robustness and generate new rule sets that enhance detection coverage.

2.2 Web Application Vulnerabilities

Web applications interact with untrusted user input across multiple layers, including HTTP requests, form submissions, query parameters, cookies, and API payloads. Improper handling of this input introduces exploitable vulnerabilities that allow attackers to manipulate application logic, access confidential data, or execute unintended operations. This section provides an analytical overview of the vulnerabilities most relevant to WAF bypass research: SQL Injection (SQLi), Cross-Site Scripting (XSS).

Each vulnerability class is described in terms of its core mechanics, attack surface, and structural properties that influence both detection strategies and susceptibility to evasion techniques. These insights form the foundation for evaluating whether LLM-generated payloads can expose weaknesses in traditional WAF rule systems.

2.2.1 SQL Injection (SQLi)

SQL Injection occurs when user-controlled input is incorporated into a backend SQL query without sufficient validation or parameterization. The attacker exploits the string-based nature of concatenated queries to alter the structure or semantics of the SQL statement executed by the database.

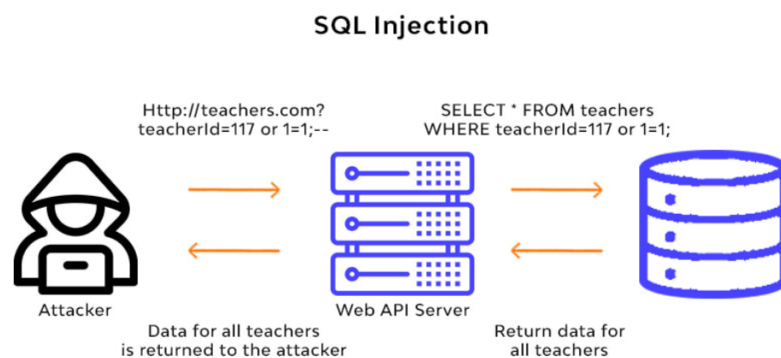


Figure 2.2: How does SQLi work²

²Wallarm. (n.d.). Structured Query Language injection (SQLi): Part 1. Retrieved from <https://www.wallarm.com/what/structured-query-language-injection-sqli-part-1>

Mechanism. Applications that dynamically construct SQL queries may unintentionally permit user input to introduce new operators, expressions, or control-flow modifiers. A vulnerable query typically takes the form:

```
SELECT * FROM users WHERE username = 'USER_INPUT';
```

If `USER_INPUT` contains characters such as quotes, logical operators, or subqueries, the resulting SQL command may be expanded into a malicious form. Attackers can therefore:

- Bypass authentication (e.g., “' OR 1=1 --”),
- Exfiltrate data (e.g., UNION-based injection),
- Manipulate tables (e.g., stacked queries),
- Perform blind inference through timing or conditional responses.

Detection Characteristics. SQLi payloads frequently share structural patterns such as:

- Use of SQL keywords (UNION, SELECT, AND, OR),
- Comparison operators and boolean logic,
- Quote imbalances or comment delimiters,
- Numeric or string-based tautologies,
- Nested or encoded expressions.

However, these features are highly malleable. Attackers can encode keywords, apply whitespace variations, or exploit database-specific syntax variants. This high degree of syntactic flexibility is one of the primary reasons LLM-generated payloads can be effective at bypassing conventional rule-based WAFs.

2.2.2 Cross-Site Scripting (XSS)

Cross-Site Scripting vulnerabilities arise when untrusted input is incorporated into a web page without proper sanitization or encoding. This allows attackers to inject JavaScript code that executes in the victim's browser within the security context of the vulnerable application.

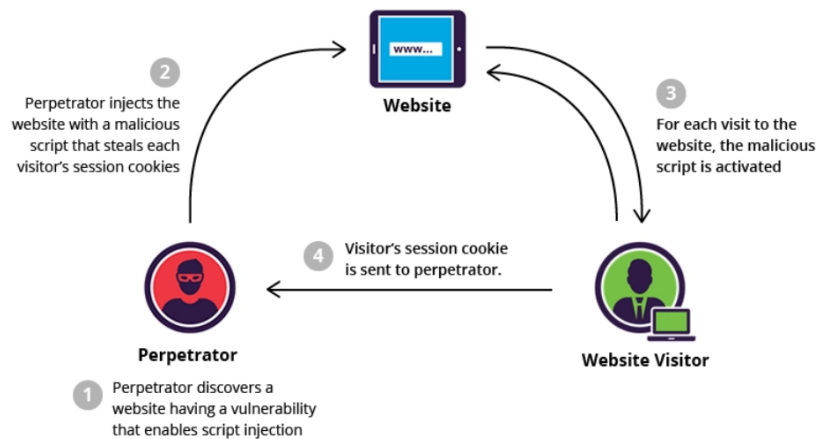


Figure 2.3: How does XSS work³

Mechanism. XSS exploits result from improper handling of output content. Vulnerable applications may reflect user input directly into:

- HTML bodies,
- DOM elements,
- event handlers,
- JavaScript contexts,
- URL parameters or redirection mechanisms.

The payload typically aims to execute arbitrary JavaScript, enabling actions such as cookie theft, credential forwarding, DOM manipulation, keylogging, phishing overlays, or abuse of authenticated sessions.

³Imperva. (n.d.). Cross-site scripting (XSS) attacks. Retrieved from <https://www.imperva.com/learn/application-security/cross-site-scripting-xss-attacks/>

Common attack patterns include:

- **Reflected XSS:** immediate injection into dynamic server-side responses.
- **Stored XSS:** persistent injection into databases, comment fields, or user messages.
- **DOM-based XSS:** injection arising from client-side JavaScript manipulation of the DOM.

Detection Characteristics. XSS payloads are structurally diverse due to the flexibility of HTML and JavaScript. Traditional WAF signatures often look for:

- Script tags (e.g., `<script>`),
- Event handlers (onclick, onerror),
- Dangerous JavaScript functions (alert, eval),
- Encoded or obfuscated JavaScript,
- Attempts to break out of HTML or attribute contexts.

Attackers can evade these signatures by using:

- Character encoding variants (URL, HTML entity, Unicode),
- DOM-based execution vectors that lack script tags,
- Tagless JavaScript execution (SVG, MathML, iframes),
- Payload structures generated through LLM transformations.

XSS detection requires contextual awareness of how input is embedded into HTML structures—capabilities that most pattern-based WAFs lack.

2.2.3 Relevance to WAF Evaluation and LLM-based Bypass

The vulnerabilities described above share several properties that make them suitable for LLM-driven bypass research:

- They rely heavily on input structure and encoding, which LLMs can alter in infinite variations.
- Traditional WAF signatures depend on matching fixed patterns, making them susceptible to syntactic transformations.
- The vulnerabilities operate at the application layer, where WAFs conduct deep inspection but also experience the greatest ambiguity.
- Many exploit patterns allow semantic equivalence: the same attack intent can be realized through numerous syntactic forms, making them ideal candidates for adversarial generation using LLMs.

Because WAFs must detect malicious intent rather than specific strings, these vulnerabilities provide a comprehensive benchmark for evaluating whether LLMs can identify and exploit weaknesses in rule-based filtering logic.

2.3 Large Language Models

Large Language Models (LLMs) are advanced deep learning models trained on large-scale text corpora to generate, classify, interpret, and transform natural language. Their architecture, typically based on the Transformer model, enables them to capture long-range dependencies, semantic relationships, and syntactic structures with high fidelity. In the context of web application security, LLMs introduce a new capability: generating adversarial payloads that preserve semantic intent while adopting alternative syntactic forms. This property makes them an effective tool for evaluating and bypassing rule-based Web Application Firewalls (WAFs).

2.3.1 Concept and Architecture Overview

An LLM processes textual input by mapping sequences of tokens into high-dimensional vector representations. Using self-attention mechanisms, the model determines the contextual importance of each token relative to others and constructs a global understanding of the sentence structure. The underlying components include:

- **Tokenization:** Conversion of text into discrete units (tokens) using subword algorithms such as Byte-Pair Encoding (BPE) or SentencePiece.
- **Embedding Layers:** Projection of tokens into continuous vector spaces that encode semantic relationships.
- **Transformer Blocks:** Layers of multi-head self-attention and feed-forward networks that iteratively refine contextual representations.
- **Decoding Mechanisms:** Autoregressive generation of output sequences based on learned probability distributions.

These mechanisms allow LLMs to generate highly diverse output sequences, including variations that differ structurally but maintain equivalent semantic intent—an essential characteristic for adversarial payload generation.

2.3.2 Representation Learning and Embeddings

Embedding spaces form the semantic backbone of LLM reasoning. Each token is represented as a dense vector that captures its contextual meaning relative to other tokens. Embedding quality directly influences the model’s ability to:

- Understand syntactic transformations,
- Identify relationships between malicious patterns,
- Generate coherent yet obfuscated strings,
- Differentiate between benign and adversarial input.

Embeddings also allow the model to generalize attack structures beyond literal keyword matches. For example, an LLM can learn that the SQL keyword UNION is conceptually similar to encoded variants (e.g., %55%4e%49%4f%4e) and alternative whitespace representations, enabling the generation of payloads that circumvent pattern-based WAF signatures.

2.3.3 Prompt-based Generation and Transformation

Large language models (LLMs) produce text based on prompts, namely user-supplied instructions that shape the desired properties of the output. With the use of prompt engineering, researchers can control various features of the generated content, such as syntactic form, level of obfuscation, encoding method, contextual constraints, and the specific attack goal or vulnerability type being targeted. When prompts are constructed to elicit novel or unconventional payload structures, LLMs can be used to examine whether a web application firewall (WAF) remains effective against non-standard variants of known exploits. For this reason, such capability is central to LLM-assisted approaches for assessing WAF bypass robustness.

2.3.4 Retrieval-Augmented Generation (RAG)

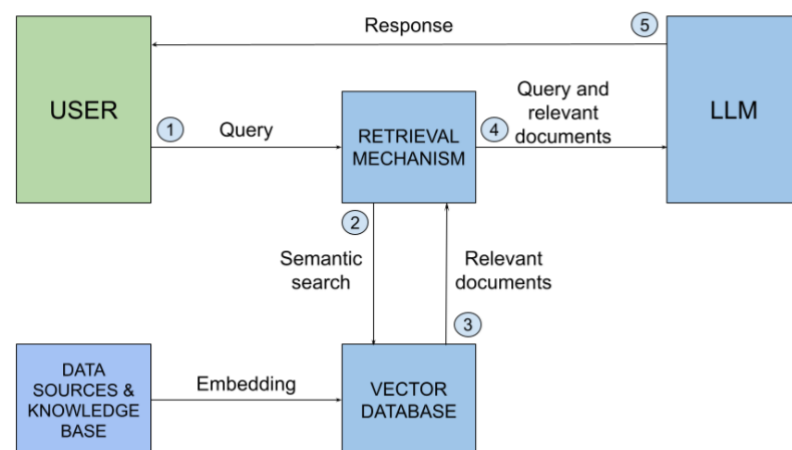


Figure 2.4: How does RAG work⁴

Retrieval-Augmented Generation (RAG) combines the generative capabilities of LLMs with information retrieved from external knowledge sources, such as security documentation, defensive rule sets, vulnerability taxonomies, and structured technical corpora. Rather than relying solely on the model's internal knowledge, RAG provides relevant contextual information at inference time, allowing the model to generate

⁴Merge. (n.d.). How retrieval-augmented generation (RAG) works. Retrieved from <https://www.merge.dev/blog/how-rag-works>

outputs that are more accurate, consistent, and contextually grounded. A typical RAG pipeline consists of three main stages:

1. **Indexing:** Reference materials are transformed into vector embeddings and stored in a vector database to enable efficient semantic search.
2. **Retrieval:** The input query is encoded into an embedding and matched against the indexed representations to retrieve the most relevant contextual information.
3. **Generation:** The retrieved context is incorporated into the LLM's generation process, improving the technical validity and contextual alignment of the produced output.

In the context of WAF evaluation, RAG can be used to provide defensive and technical grounding by:

- Retrieving relevant syntax patterns and protocol-specific structures,
- Supplying contextual knowledge about vulnerability classes and their expected characteristics,
- Supporting the generation of structurally valid and semantically consistent test inputs,
- Improving reproducibility by grounding outputs in explicitly retrieved reference material.

This use of RAG increases the reliability of LLM-generated outputs by ensuring that they remain syntactically correct and contextually aligned with the defensive evaluation setting, rather than being produced solely from unconstrained model generation.

2.3.5 Adversarial Generation Capabilities

LLMs possess several characteristics that make them particularly effective at producing payloads capable of bypassing WAFs:

- **Syntactic Flexibility:** The ability to reframe expressions in alternative formats, reorder components, or introduce uncommon token boundaries.

- **Encoding Awareness:** Built-in understanding of URL encoding, base64, Unicode, HTML entities, and other encoding schemes often used in evasion.
- **Contextual Adaptation:** LLMs can generate payloads tailored to the specific filtering rules or error messages observed during testing.
- **Semantic Constancy:** Despite structural changes, the malicious intent (e.g., SQLi or XSS semantics) remains intact.
- **High Diversity:** The model can sample from a large distribution of possible outputs, producing a wide variety of payloads with minimal repetition.

These capabilities directly challenge WAFs whose detection strategies rely heavily on static rule sets, pattern matching, or simple normalization.

2.3.6 Limitations and Constraints

Although powerful, LLMs also exhibit limitations that must be considered when integrating them into a security testing workflow:

- **Lack of execution environment:** LLMs generate text but cannot execute payloads or evaluate direct backend responses without external tools.
- **Hallucination:** Generated payloads may sometimes be syntactically incorrect or semantically ineffective.
- **Training bias:** Models trained on sanitized or filtered data may not retain full knowledge of exploit structures.
- **Prompt sensitivity:** Small variations in prompt wording can cause large differences in output quality.

Understanding these constraints is essential for constructing a robust methodology for WAF evaluation.

2.3.7 Relevance to WAF Bypass and Rule Improvement

LLMs are particularly well-suited for stress-testing WAFs and generating improved detection rules. Their ability to produce semantically consistent yet syntactically diverse payloads enables:

- Identification of WAF blind spots,
- Exploration of encoding combinations or unusual lexical structures,
- Automated mutation of known exploits into novel variants,
- Generation of high-volume adversarial samples for rule optimization.

By analyzing which LLM-generated payloads bypass existing protections, security teams can derive new rule patterns, enhance normalization logic, and update anomaly thresholds. This establishes a feedback loop wherein LLM-driven adversarial testing informs iterative WAF improvement.

2.3.8 Clustering of Adversarial Payloads

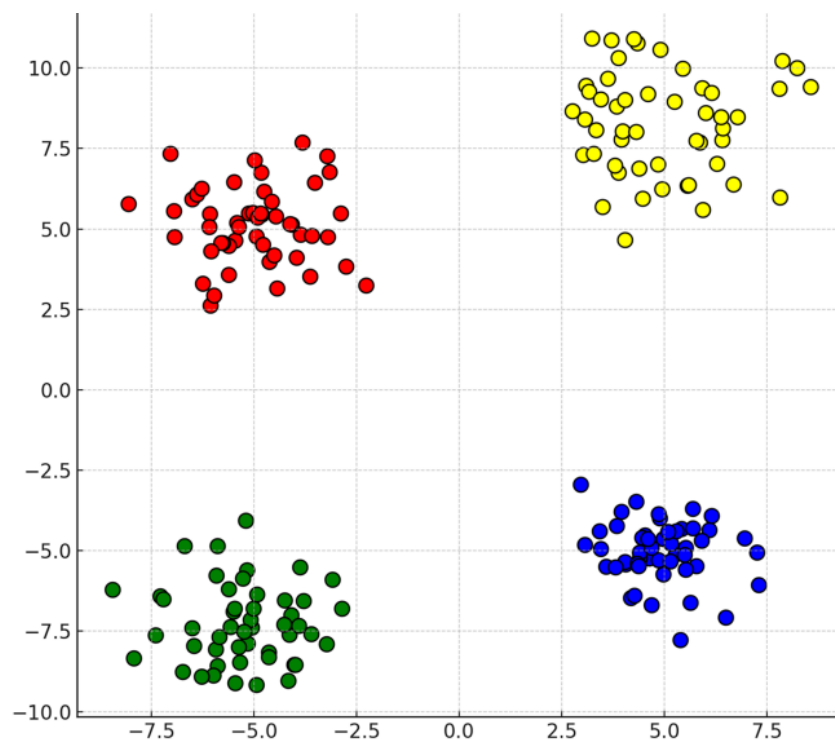


Figure 2.5: How does Clustering work⁵

Clustering is an unsupervised learning technique used to group data points based on similarity without requiring labeled examples. In the context of textual data, clustering helps identify underlying structural or syntactic patterns by organizing similar inputs into coherent groups.

A common approach for clustering text involves transforming raw strings into numerical vector representations. Techniques such as TF-IDF encoding allow text to be represented in a high-dimensional space, where similarity between samples can be measured using distance metrics such as Euclidean or cosine distance.

To improve efficiency and reduce noise, dimensionality reduction methods such as Singular Value Decomposition (SVD) are often applied before clustering. This step projects data into a lower-dimensional latent space while preserving important structural relationships.

Various clustering algorithms can then be applied, including density-based methods (e.g., HDBSCAN) and hierarchical approaches (e.g., Agglomerative Clustering). Each method offers different advantages in terms of cluster shape detection, scalability, and robustness to noise. These techniques are widely used in text mining and pattern discovery tasks.

⁵Merge. (n.d.). Comprehensive analysis of clustering algorithms: exploring limitations and innovative solutions. Retrieved from <https://peerj.com/articles/cs-2286.pdf>

2.3.9 Abstract Syntax Trees (AST)

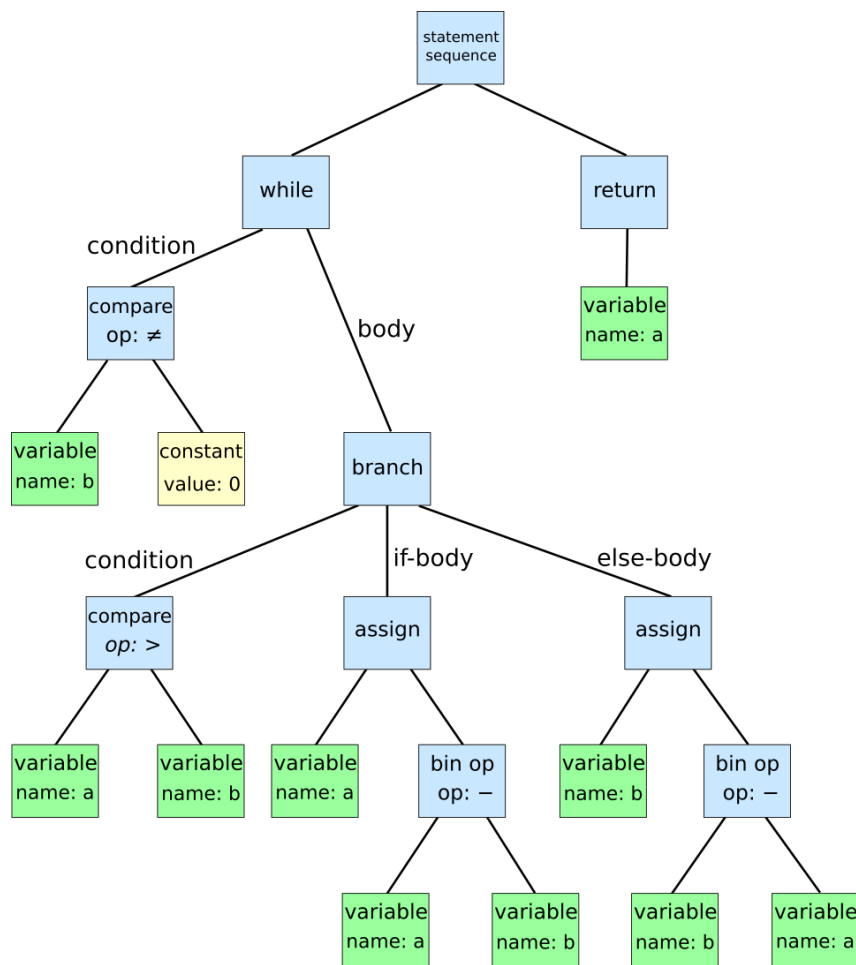


Figure 2.6: How does AST tree work⁶

An Abstract Syntax Tree (AST) is a tree-based representation of the syntactic structure of code or structured input. Unlike raw text, which is treated as a sequence of characters or tokens, an AST captures the hierarchical relationships between language constructs such as expressions, statements, and operators.

In an AST, each node represents a syntactic element, while edges define parent-child relationships that reflect the grammatical structure of the input. This abstraction removes irrelevant details such as formatting or whitespace, focusing instead on the logical organization of the content.

ASTs are commonly used in compilers, interpreters, and static analysis tools

⁶Merge. (n.d.). Abstract syntax tree. Retrieved from <https://www.scss.tcd.ie/John.Waldron/CSU33071/Lectures/AbstractSyntaxTree.pdf>

to enable program understanding, transformation, and validation. By working on structured representations rather than raw text, systems can perform more precise analysis, such as detecting equivalent expressions, identifying patterns, or applying transformations in a semantically consistent manner.

In security-related contexts, AST-based representations are particularly useful for analyzing code-like payloads, as they allow detection mechanisms to reason about structural intent rather than relying solely on surface-level string patterns.

Chapter 3

Related Tools and Works

3.1 Web Application Firewall (WAF)

3.1.1 Common Web Application Firewalls

ModSecurity WAF.

Overview of ModSecurity. ModSecurity¹ is one of the most popular open-source Web Application Firewall (WAF) systems today. Initially developed as a module specifically for the Apache web server, thanks to its flexibility and effectiveness, it has been extended to support other server platforms such as Nginx and IIS. The continuous evolution of ModSecurity has made it a top choice for many organizations when deploying web application protection solutions.

Mechanism of ModSecurity. Unlike network-layer firewalls, which cannot deeply analyze the actual content of HTTP packets, ModSecurity operates at the application layer, with the capability to access and analyze the full content of HTTP requests and responses. This enables ModSecurity to detect complex attack patterns targeting web applications that protection solutions at the network or transport layers cannot recognize, such as SQL Injection, XSS, Local File Inclusion, etc.

¹OWASP Foundation. (n.d.). ModSecurity: Open-source web application firewall. Retrieved from <https://github.com/owasp-modsecurity/ModSecurity>

When integrated into web servers like Nginx or Apache—especially in environments using SSL/TLS—ModSecurity processes requests after SSL decryption is complete. This means the entire HTTP content, even if encrypted in transit, can be thoroughly inspected by ModSecurity once decrypted on the server. For example, when an HTTPS request is sent to Nginx, Nginx performs SSL decryption and then forwards the HTTP content to ModSecurity for analysis before passing it to the backend application.

ModSecurity’s processing flow for a typical HTTP session is divided into five main phases. Rules can be applied at any phase, and the outcome of one phase can affect subsequent phases.

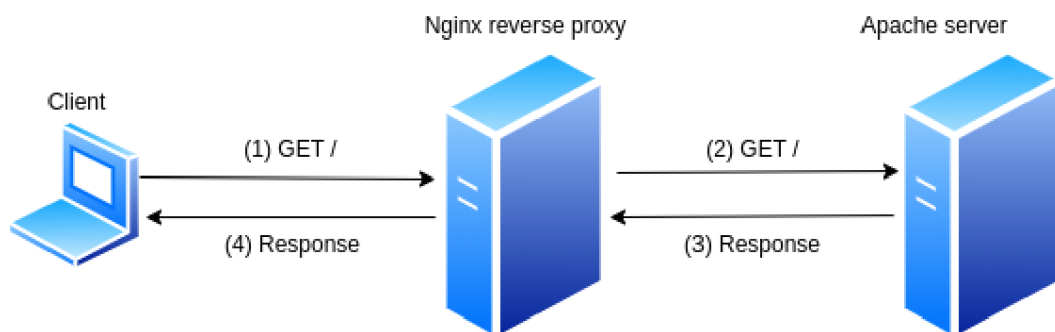


Figure 3.1: The HTTP processing phases in ModSecurity [18]

Phase 1 – Request Headers: After SSL decryption (if applicable) is completed, ModSecurity begins by inspecting the HTTP request headers to detect abnormal or malicious requests. Examples include requests without a User-Agent header, mismatched Origin values, or inconsistencies between the reported request time and the actual system time. This early inspection helps block potential threats before processing the request body, which is significantly more resource-intensive. In addition, ModSecurity reads specific headers to determine how subsequent phases should be handled. For instance, the Content-Type header is analyzed to identify the format of the request body, enabling accurate parsing in the following phase.

Phase 2 – Request Body: This phase is executed immediately after ModSecurity has received the complete request body. Rule sets are typically designed to be most stringent at this stage, performing in-depth analysis of the request content to detect potential malicious payloads such as SQL Injection, Cross-Site Scripting (XSS), and

other injection-based attacks.

Phase 3 – Response Headers: After the server has processed the request and generated a response, ModSecurity receives the response headers but not yet the response body. At this stage, ModSecurity analyzes the headers to determine whether the response exhibits abnormal behavior. For example, responses with 5XX error status codes often include detailed error messages or stack traces in the response body for debugging purposes. Such information may expose sensitive details about the application source code or system configuration, and ModSecurity aims to detect and prevent this type of information leakage.

Phase 4 – Response Body: This phase represents the final inspection step before the response is delivered to the client. ModSecurity analyzes the entire response body to identify potential leakage of sensitive information or embedded malicious content that may have been injected earlier and returned within the response.

Phase 5 – Logging: In this phase, ModSecurity records all information related to the processed HTTP transaction, including both blocked and allowed requests and responses. These records are stored in log files to support monitoring, event analysis, incident investigation, and forensic analysis. Additionally, logging data serves as a valuable source for refining and optimizing rule sets in future deployments.

Thanks to its comprehensive analysis of both inbound requests and outbound responses, ModSecurity is capable of detecting and preventing a wide range of sophisticated attacks that are difficult for traditional security solutions to handle. Its deep integration into the HTTP processing flow of the web server makes ModSecurity a powerful defensive layer, particularly for systems with high security requirements and frequent exposure to emerging threats.

Key Features of ModSecurity. ModSecurity stands out due to its powerful features that provide comprehensive protection for web applications. One of its most important capabilities is deep parsing and processing of HTTP request and response components, which enables the detection of complex attack patterns. ModSecurity employs a buffering mechanism to temporarily store the entire request and response content,

allowing thorough inspection of data before it is forwarded to the backend application or returned to the client.

In addition, ModSecurity offers a detailed logging system that records all security-related events, including blocked requests, triggered rules, and contextual information useful for incident investigation and forensic analysis. Notably, the ModSecurity rule engine is highly flexible, allowing users to define custom and complex rules or leverage predefined rule sets such as the OWASP Core Rule Set² (CRS). The rule engine supports a wide range of operations, including string matching, regular expression evaluation, numerical value checks, JSON/XML structure parsing, and other advanced processing techniques.

Rule Grammar of ModSecurity. The ModSecurity rule language provides a rich set of directives for defining, managing, and controlling security rules. The following table summarizes the key directives and provides a brief description of their functionality:

²OWASP Foundation. (n.d.). OWASP Core Rule Set: Generic attack detection rules for ModSecurity-compatible WAFs. Retrieved from <https://github.com/coreruleset/coreruleset>

Table 3.1: Common ModSecurity Directives

Directive	Description
SecRule	Defines a new rule; the most common directive used to specify variables, operators, transformations, and actions.
SecAction	Executes unconditional actions (always triggered), typically used for state/variable initialization or logging.
SecDefaultAction	Specifies default actions for subsequent rules (e.g., phase, log/deny, status).
SecMarker	Marks a position within the rule set (does not execute actions), supporting flow control with skip/chain.
SecRuleInheritance	Controls rule inheritance in child contexts (enables or disables inheritance from parent contexts).
SecRuleRemoveById	Removes or disables rules by ID (commonly used for CRS customization).
SecRuleRemoveByMsg	Removes or disables rules based on messages matching a regular expression.
SecRuleScript	Defines rules that leverage Lua scripts for advanced processing or custom logic.
SecRuleUpdateActionById	Modifies the actions of a rule by ID (e.g., changing phase, status, or logging behavior).

Table 3.2: Commonly Used Variables in ModSecurity

Variable	Description
ARGS	Collection of all request parameters (GET and POST).
ARGS_NAMES	Names of all request parameters.
ARGS_POST	Parameters contained in the request body.
FILES	Uploaded file data.
FILES_NAMES	Names of uploaded files.
REQUEST_URI	Request URI path.
REQUEST_HEADERS	Collection of HTTP request headers.
REQUEST_METHOD	HTTP request method (GET, POST, etc.).
REQUEST_FILENAME	File path extracted from the URI (excluding parameters).
REQUEST_BODY	Body content of the HTTP request.
RESPONSE_BODY	Body content of the HTTP response.
TX	Transaction variable used to store intermediate values across rules.
XML	Access to XML DOM structures.

Table 3.3: Commonly Used Operators in ModSecurity

Operator	Description
@rx	Regular expression matching.
@contains	Substring containment check.
@streq	Exact string equality comparison.
@eq	Numeric equality comparison.
@ge	Greater-than-or-equal comparison.
@gt	Greater-than comparison.
@le	Less-than-or-equal comparison.
@lt	Less-than comparison.
@detectXSS	Detection of Cross-Site Scripting (XSS) attacks.
@detectSQLi	Detection of SQL Injection attacks.

Table 3.4: Commonly Used Transformation Functions

Transformation	Description
none	Resets all transformation functions.
lowercase	Converts input to lowercase.
base64Decode	Decodes Base64-encoded data.
normalizePath	Normalizes file paths.
normalizePathWin	Normalizes Windows-style file paths.
urlDecode	Decodes URL-encoded data.
urlDecodeUni	Decodes Unicode URL encoding (e.g., %uXXXX).
removeNulls	Removes null byte characters.
htmlEntityDecode	Decodes HTML entities.
length	Computes the length of a variable.
compressWhitespace	Removes excessive whitespace.

Table 3.5: Commonly Used Actions in ModSecurity

Action	Description
deny	Blocks the request.
log	Logs the security event.
status:code	Returns an HTTP status code (e.g., status:403).
phase	Specifies the execution phase of the rule.
id	Unique identifier of the rule.
msg	Message displayed or logged when the rule is triggered.
block	Blocks the request (equivalent to deny).
t:...	Applies a transformation function (e.g., t:lowercase).

Each security rule defined using the SecRule directive follows the general format:

```
SecRule VARIABLES "OPERATOR" ["TRANSFORMATIONS,ACTIONS"]
```

Among these components, the variable (VARIABLE) and the operator (OPERATOR) are mandatory, while the transformation functions (TRANSFORMATIONS) and actions

(ACTIONS) are optional. If no transformation or action is explicitly specified, ModSecurity applies the default values defined by the SecDefaultAction directive.

Variable: Specifies which part of the HTTP transaction is subject to inspection, such as ARGS (request parameters), REQUEST_HEADERS (HTTP headers), REQUEST_BODY (request body), REQUEST_URI (URI), FILES (uploaded files), RESPONSE_BODY (response body), and others. These variables store values in a binary-safe format, ensuring safe and accurate inspection of the data. **Operator:** Defines how the variable is evaluated.

Common operators include regular expression matching (@rx), substring search (@contains), string equality comparison (@streq), numerical comparisons (@eq, @ge, @gt, @le, @lt), XSS detection (@detectXSS), SQL injection detection (@detectSQLi), and others. Each rule can specify only a single operator. **Transformation:** Used

to preprocess the variable value before evaluation. Typical transformation functions include lowercase (convert to lowercase), base64Decode (decode Base64-encoded data), urlDecode (decode URL-encoded values), compressWhitespace (remove excessive whitespace), htmlEntityDecode (decode HTML entities), and others. Multiple transformations can be applied sequentially within a single rule. **Action:**

Specifies the operations executed when a rule is triggered. Common actions include deny (block the request), log (record the event), status:403 (return HTTP status code 403), phase (execution phase), id (rule identifier), msg (log message), and others. A rule may define multiple actions or, in some cases, no explicit action.

Example 1: Blocking requests containing the string <script> in the URI:

```
SecRule REQUEST_URI "@rx <script>" "log,deny,status:403"
```

This rule checks whether the request URI contains the string <script>. If a match is found, the request is logged, blocked, and an HTTP 403 status code is returned.

Example 2: Blocking requests with parameters containing the phrase ``delete from'' using transformation functions:

```
SecRule ARGS "@contains delete from" phase:2,t:lowercase,t:compressWhitespace,block
```

This rule first converts all request parameters to lowercase and removes redundant whitespace. It then checks whether any parameter contains the phrase ``delete

from' ' and blocks the request during phase 2 if a match is detected.

In addition to simple rules, ModSecurity supports advanced constructs such as chained rules (*chain*) and skip rules, enabling the creation of complex protection logic tailored to specific application requirements.

Core Rule Set (CRS) and OWASP ModSecurity CRS. The Core Rule Set (CRS) is a collection of pre-defined security rules designed to provide comprehensive protection for web applications without requiring users to create rules from scratch. CRS is intended to detect and prevent the most common types of web-based attacks, including SQL Injection, Cross-Site Scripting (XSS), Local File Inclusion (LFI), Remote File Inclusion (RFI), Command Injection, and many other attack vectors.

The OWASP ModSecurity Core Rule Set (OWASP CRS) is a large community-driven project maintained and continuously developed by security experts worldwide. The primary objective of OWASP CRS is to provide a standardized rule set that is regularly updated to address emerging attack techniques. In addition to protecting web applications against common vulnerabilities, OWASP CRS is designed to minimize false positives through carefully optimized rules that have been extensively tested across diverse environments.

The rules within CRS are organized into multiple groups based on attack categories, enabling users to easily configure, enable or disable specific rule sets, and customize them according to practical deployment requirements. By leveraging CRS, organizations can significantly reduce deployment time, achieve a high level of baseline security, and benefit from the collective knowledge and experience of the global security community.

Cloudflare WAF. Cloudflare WAF is a cloud-based Web Application Firewall deployed at the edge layer of Cloudflare's global content delivery network. Unlike host-based solutions such as ModSecurity, which operate directly on web servers, Cloudflare WAF functions as a reverse proxy that filters traffic before it reaches the origin system.

Incoming HTTP/HTTPS requests are routed through geographically distributed edge nodes, where inspection and filtering are performed. This architecture reduces latency while minimizing the exposure of backend services to malicious traffic. In addition, Cloudflare WAF is tightly integrated with other platform components such as DDoS

protection, caching, and bot management, forming a unified security layer.

The system continuously benefits from global threat intelligence collected across a large number of protected applications, enabling rapid adaptation to emerging attack patterns.

Detection and Processing Mechanism. Cloudflare WAF evaluates requests through a high-performance pipeline designed for real-time processing. After TLS termination at the edge, requests are analyzed using a combination of rule-based matching, behavioral signals, and reputation-based scoring.

Rather than operating in rigid processing phases, the system applies a flexible evaluation model in which multiple signals are considered simultaneously. These include request structure, payload content, client reputation, and anomaly indicators. Based on this evaluation, the system determines an appropriate action such as allowing, blocking, or challenging the request.

This design allows Cloudflare WAF to scale efficiently while maintaining responsiveness and adaptability in dynamic threat environments.

Key Capabilities. Cloudflare WAF provides several core capabilities that distinguish it from traditional rule-based systems. First, its edge-based deployment enables early mitigation of malicious traffic, reducing load on origin infrastructure. Second, it combines managed rule sets with customizable logic, allowing both standardized protection and application-specific tuning. Third, the integration of behavioral analysis and global intelligence enhances its ability to detect evolving attack techniques beyond static signatures.

These characteristics make Cloudflare WAF particularly suitable for modern, distributed web applications where scalability and real-time response are critical.

Rule Grammar of Cloudflare WAF. Cloudflare WAF employs an expression-based rule language in which each rule consists of a logical condition and an associated action. The condition evaluates attributes of the HTTP request, while the action defines how the system responds when the condition is satisfied.

Table 3.6: General Structure of Cloudflare WAF Rules

Component	Description
Expression	Logical condition evaluating request attributes such as URI, headers, body, or client metadata.
Operator	Defines how values are compared (e.g., substring matching, regular expressions, equality).
Function	Optional transformation applied before evaluation (e.g., decoding or normalization).
Action	Response triggered when the condition is satisfied (e.g., block, allow, challenge).

A typical rule follows the form:

`(expression) -> action`

Example: Blocking requests containing script injection patterns:

```
(http.request.uri contains "<script>") -> block
```

Managed Rules and OWASP Integration. Cloudflare provides managed rule sets that are continuously updated to protect against common web vulnerabilities. These include protections aligned with the OWASP Core Rule Set, covering attack categories such as SQL injection, cross-site scripting, and protocol anomalies.

Users can configure sensitivity levels and selectively enable or override rules to adapt protection mechanisms to specific application requirements. This combination of automated rule management and customizable logic allows Cloudflare WAF to maintain both usability and security effectiveness in diverse deployment scenarios.

AWS WAF.

Overview of AWS WAF. AWS WAF (Web Application Firewall) is a cloud-native security service designed to protect web applications and APIs from common web exploits. It integrates directly with AWS services such as Amazon CloudFront,

Application Load Balancer (ALB), and API Gateway, enabling traffic inspection at both edge and regional layers.

Unlike traditional host-based WAFs, AWS WAF operates as a fully managed service, eliminating the need for manual deployment and maintenance. Requests are filtered before reaching backend resources, reducing exposure to attacks and improving system resilience. Its integration with AWS Shield and Firewall Manager further enhances protection and centralized control across distributed environments.

Detection and Processing Mechanism. AWS WAF evaluates incoming HTTP/HTTPS requests using Web Access Control Lists (Web ACLs), which contain ordered sets of rules. Each request is inspected against these rules, and the first matching condition determines the action applied, such as allowing, blocking, or monitoring the request.

The system supports inspection of multiple request components, including URI paths, headers, query strings, and body content. Before evaluation, transformations such as decoding or normalization may be applied to improve detection accuracy. This rule-priority model allows flexible and efficient filtering while maintaining scalability across varying traffic volumes.

Key Capabilities. AWS WAF provides a flexible and extensible rule framework suitable for cloud environments. It supports both managed rule groups and custom-defined rules, enabling protection against common vulnerabilities such as SQL injection and cross-site scripting. Additional features include rate-based rules for mitigating abuse, bot detection mechanisms, and integration with logging services such as Amazon CloudWatch for monitoring and analysis.

These capabilities allow AWS WAF to balance automation and customization, making it adaptable to a wide range of application architectures and threat models.

Rule Grammar of AWS WAF. AWS WAF defines rules using a structured, JSON-based model in which each rule consists of a condition (statement) and an associated action. The condition specifies how request attributes are evaluated, while the action determines the system's response when a match occurs.

Table 3.7: General Structure of AWS WAF Rules

Component	Description
Web ACL	Container that groups and orders rules applied to a resource.
Rule	Individual inspection unit with defined priority and action.
Statement	Logical condition that evaluates request attributes (e.g., string match, regex, IP filtering).
Transformation	Preprocessing applied to normalize request data before evaluation.
Action	Response triggered when the rule condition is satisfied (e.g., allow, block, count).

Conceptually, rules follow the form:

IF (Statement) THEN Action

Example: Blocking requests containing script injection patterns in the URI:

Statement: ByteMatch (URI Path contains "<script>") Action: BLOCK
--

Managed Rule Groups in AWS WAF. AWS WAF provides managed rule groups maintained by AWS and third-party vendors to address common vulnerabilities and emerging threats. These rule sets include protections for attack classes such as SQL injection, cross-site scripting, and known malicious inputs.

Users can combine managed rules with custom logic, adjust rule priorities, and override default actions to suit specific application requirements. This combination enables a balance between automated protection and fine-grained control, supporting secure deployment in dynamic cloud environments.

Naxsi WAF.

Overview of Naxsi. Naxsi (Nginx Anti XSS & SQL Injection) is an open-source Web Application Firewall designed specifically for the Nginx web server. It adopts a negative security model, focusing on identifying suspicious or abnormal patterns rather than relying on large signature databases of known attacks. This approach allows Naxsi to remain lightweight while still effectively detecting common web threats such as cross-site scripting and SQL injection.

Integrated directly into the Nginx processing pipeline, Naxsi is optimized for high-performance environments where low latency is essential. Its design prioritizes efficiency and simplicity, making it suitable for large-scale systems handling high volumes of HTTP traffic.

Detection Mechanism. Naxsi operates as an embedded module within Nginx, intercepting and analyzing incoming HTTP requests. It inspects various request components, including the URI, headers, parameters, and body content. Instead of binary rule matching, Naxsi employs a score-based detection mechanism in which each matched pattern contributes to an overall anomaly score.

Each rule corresponds to a specific type of suspicious behavior and contributes a predefined score to a category such as XSS or SQL injection. The accumulated score is then compared against configured thresholds to determine whether a request should be allowed, logged, or blocked. This scoring approach enables more flexible and context-aware detection compared to strict rule-based systems.

To support deployment in real-world environments, Naxsi also provides a learning mode that logs potential violations without enforcing blocking actions. This allows administrators to refine rule configurations and reduce false positives before activating strict protection policies.

Key Capabilities. Naxsi combines efficiency with adaptability through several core capabilities. Its negative security model emphasizes anomaly detection rather than exhaustive signature matching, while the scoring mechanism enables nuanced evaluation of request severity. The system maintains high performance due to its tight integration with Nginx and minimal processing overhead.

In addition, Naxsi supports whitelist mechanisms that allow administrators to

explicitly permit trusted patterns, thereby reducing false positives. This balance between detection sensitivity and configurability makes it suitable for dynamic web applications with diverse input behaviors.

Rule Grammar of Naxsi. Naxsi uses a concise rule syntax in which each rule detects a specific pattern and assigns a score to a corresponding attack category. Unlike more complex rule engines, its grammar is designed to be simple while leveraging the scoring and threshold system for expressive detection.

Table 3.8: General Structure of a Naxsi Rule

Component	Description
Pattern	String or expression used to match suspicious input.
Match Zone (mz)	Defines which part of the request is inspected (e.g., URI, arguments, body).
Score (s)	Assigns a value to a specific attack category when matched.
Message (msg)	Descriptive label used for logging and analysis.
Rule ID (id)	Unique identifier for rule management.

Rules conceptually follow a pattern-matching and scoring structure, where detection contributes to an accumulated anomaly score.

Example: Detecting a potential XSS pattern in request inputs:

```
MainRule "<script" "mz:ARGS|URL" "s:$XSS:8" id:1001;
```

If the total score exceeds a predefined threshold, separate configuration rules (e.g., `CheckRule`) determine whether the request is blocked.

Core Rules and Customization. Naxsi provides a default set of core rules targeting common web vulnerabilities such as XSS, SQL injection, and directory traversal. These rules are intentionally compact, reflecting the system’s lightweight design philosophy.

In practice, effective deployment relies on adapting these rules to the specific application context. By combining core rules with customized scoring thresholds

and whitelist configurations, Naxsi can achieve a balance between strong security enforcement and low false positive rates. This flexibility, together with its performance efficiency, makes Naxsi a practical solution for high-throughput web systems.

Table 3.9: Comparative Summary of Four Common WAF Solutions

WAF	Deployment Model	Detection Style	Strengths	Availability / Service Tier Model
ModSecurity	Open-source, typically embedded into Apache, Nginx, or IIS	Rule-based inspection across multiple HTTP processing phases	High transparency, strong customizability, rich rule grammar, broad community support, and compatibility with OWASP CRS	Fully open-source and self-managed; no vendor pricing tier, though operational cost depends on infrastructure and administration effort
Cloudflare WAF	Cloud-based reverse proxy deployed at the edge of Cloudflare's network	Expression-based filtering combined with managed rules, behavioral signals, and global threat intelligence	Strong scalability, early traffic filtering, low operational overhead, and tight integration with CDN, DDoS protection, and bot management	Typically follows a <i>Free-to-paid-tier</i> platform model, with more advanced WAF capabilities available in Pro, Business, and Enterprise plans

WAF	Deployment Model	Detection Style	Strengths	Availability / Service Tier Model
AWS WAF	Fully managed cloud-native WAF integrated with CloudFront, ALB, API Gateway, and related AWS services	Ordered rule evaluation through Web ACLs, with support for managed rules, custom statements, and transformations	Native integration with the AWS ecosystem, flexible policy composition, centralized governance, and strong suitability for cloud-first architectures	Commonly offered as a managed <i>pay-as-you-go</i> service rather than a permanent standalone free tier; in practice it scales from small deployments to enterprise usage based on consumption
Naxsi	Open-source module embedded directly into Nginx	Negative security model with anomaly scoring and threshold-based blocking	Lightweight design, high performance, and suitability for high-throughput environments requiring fine whitelist tuning	Fully open-source and self-managed; no vendor pricing tier, though effectiveness depends heavily on tuning quality and application-specific learning

From a practical perspective, ModSecurity is often most suitable when detailed rule control and compatibility with community-maintained rule sets are primary concerns. Naxsi is attractive for performance-sensitive Nginx deployments that can benefit from anomaly scoring and lightweight enforcement. Cloudflare WAF is well suited to Internet-facing applications that require globally distributed protection with simplified operations, while AWS WAF is especially advantageous in environments already centered around AWS services and infrastructure governance. Therefore, there is no universally superior WAF; rather, each solution offers a different balance among control, scalability, operational complexity, and commercial service model.

3.1.2 LLM Application in Enhancing WAF Security

The application of Large Language Models (LLMs) to Web Application Firewall (WAF) security has emerged as a promising research direction in recent years. Current literature demonstrates two primary approaches: leveraging LLMs to generate adversarial payloads for evaluating WAF robustness, and employing LLMs to synthesize and refine defensive rules based on observed attack patterns. This subsection reviews representative works in both domains, with particular emphasis on GenSQLi and GenXSS, which provide comprehensive end-to-end frameworks integrating offensive payload generation with defensive rule synthesis. **Babaey and Ravindran** propose **GenSQLi** [1], a closed-loop workflow that is often treated as a baseline pattern for “generate attacks → test WAF → learn from bypasses → generate new rules” in SQL injection defense automation. In their pipeline, an off-the-shelf LLM is prompted via in-context learning to produce obfuscated SQLi payloads, which are then validated against the target database to reduce hallucinations before being executed against a vulnerable web app protected by a WAF; bypassing payloads are analyzed (via clustering) and then used to prompt the LLM to generate WAF rules, with iterative refinement via RLHF-style human feedback on rule quality. Experimentally, they report generating 514 new SQLi payloads (92.5% validated on MySQL) and achieving 89% bypass on ModSecurity+OWASP CRS, after which 23 generated rules blocked 99% of previously successful bypasses.

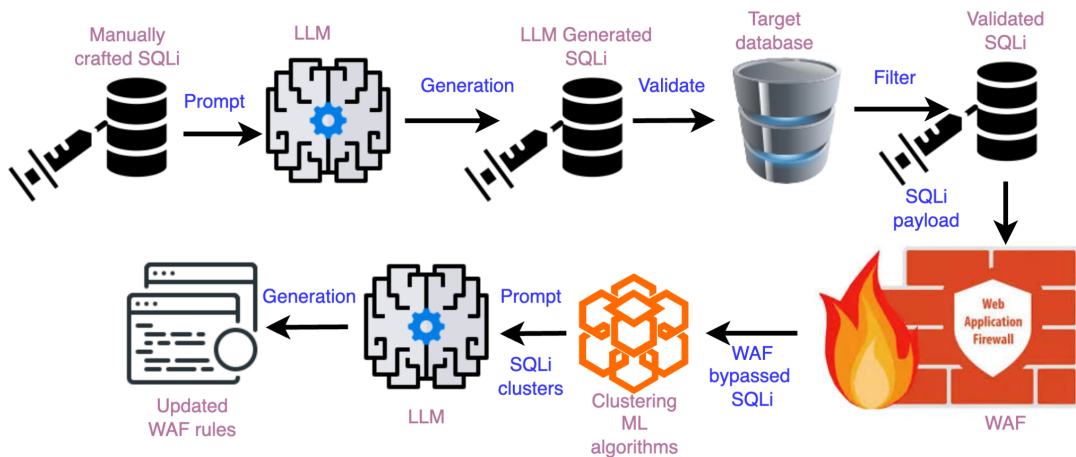


Figure 3.2: Pipeline proposed in GenSQLi [1].

A key strength of GenSQLi is that it demonstrates a practical, end-to-end workflow for continuously hardening WAF signatures using LLM-generated variants plus validation and rule synthesis, and it also provides an initial portability comparison (GPT-4o vs Gemini Pro). However, for a thesis discussion you should state clearly that GenSQLi relies on base, general-purpose LLMs accessed via prompting (not a specialized/fine-tuned security model), and its evaluation covers a limited set of firewalls—with ModSecurity as the main target (and AWS WAF included, but not a broad multi-WAF benchmark across many engines/configurations).

Besides, **Babaey and Ravindran** [2] propose **GenXSS**, a closed-loop workflow that uses **base LLMs** (e.g., GPT-4o / Gemini Pro) with in-context learning to generate obfuscated XSS payloads, then **validates** generated candidates on a purpose-built vulnerable XSS application (Brute Logic) to filter irrelevant or non-executable outputs before running **WAF bypass testing**. Bypassing payloads are stored and grouped via clustering (TF-IDF+HAC and SequenceMatcher+DBSCAN), after which a second LLM is prompted to synthesize ModSecurity rules; an RLHF-style iterative refinement is used to improve rule correctness and coverage.

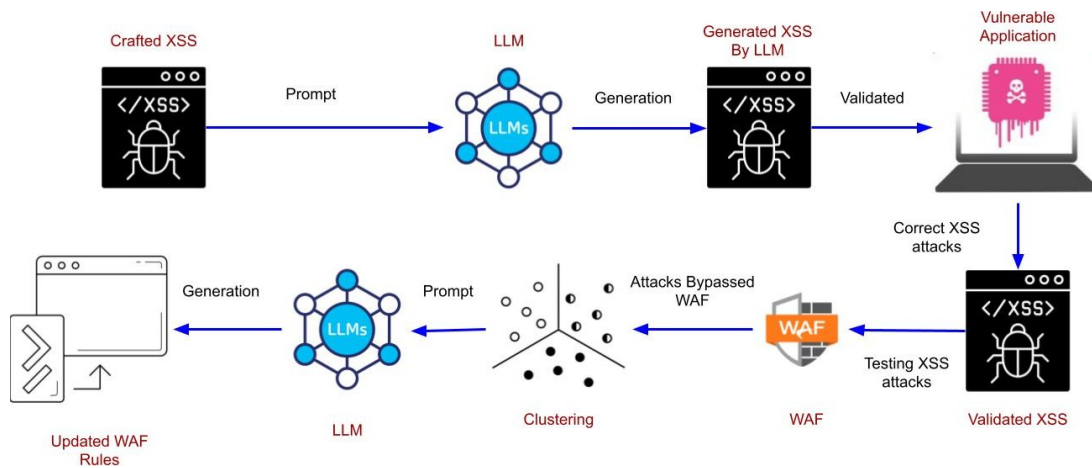


Figure 3.3: Pipeline proposed in GenXSS [2].

In evaluation, GenXSS reports that GPT-4o generated 264 XSS payloads with 83% validation on Brute Logic, and among validated payloads 80% bypassed ModSecurity (OWASP CRS); rule synthesis then blocked 86% of previously successful attacks using only 15 new rules, while Gemini Pro showed lower validity (63%) and lower bypass performance relative to GPT-4o. However, the study’s generalizability is

constrained by a **limited test scope**: the validation focuses on only two XSS types (reflected and DOM-based), a single vulnerable application, and primarily two WAFs (ModSecurity and AWS WAF), with only preliminary (non-core) testing mentioned for Cloudflare/Imperva; combined with reliance on off-the-shelf base models (prompting rather than specialized fine-tuning), this leaves open questions about robustness across broader WAF engines and real production traffic diversity.

By building upon the proven methodologies of GenSQLi and GenXSS while addressing their identified limitations, LLMShield aims to provide a more robust, scalable, and operationally viable framework for automated WAF testing and enhancement.

Tran et al. (2020) [18] address limitations of rule-based Web Application Firewalls (WAFs), especially false positives and difficulty keeping up with new/bypassing attack patterns, by proposing a hybrid WAF architecture that combines ModSecurity/OWASP CRS with machine-learning-based traffic classification. Their work is strong in its practical, engineering-oriented perspective: they emphasize real deployment concerns (pipeline design, data collection from real benign traffic plus generated attack traffic) and aim to reduce inference overhead by porting models from Python into a faster runtime for integration with Nginx/OpenResty. A key weakness is that the approach depends heavily on the representativeness of collected benign traffic and the coverage/realism of generated attack samples; if the training data is biased or narrow, the ML layer may generalize poorly or introduce new failure modes (e.g., missed novel attacks or unstable false positives).

Tran et al. (2025) [3] propose DEG-WAF, a black-box framework to generate evasive payloads for assessing WAF robustness by combining a pre-trained language model with reinforcement learning (e.g., PPO/A2C), a reward model to approximate WAF behavior, and grammar-guided sampling to keep payloads syntactically valid. The main strength is that it goes beyond static payload lists: it can automatically search for high-bypass variants while tracking payload validity and diversity, and it explicitly evaluates across different WAF types (rule-based ModSecurity vs. a WAF with ML/semantic capabilities). Its weaknesses include reliance on a relatively small backbone model (OPT-125M) and an evaluation scope limited to two WAFs; also,

a learned reward model can misestimate true bypass success and may encourage overfitting to quirks of the target WAF rather than broadly transferable evasions.

Wu et al. propose **WAFBOOSTER** [23], a black-box framework that boosts a target WAF against mutated (evasive) malicious payloads by learning from bypasses and automatically producing new signature-style rules. The workflow first trains a shadow model to approximate the WAF’s allow/block behavior, then uses a neural payload generator to produce candidate evasions, applies a payload corrector to enforce validity, and finally extracts salient substrings and synthesizes regex-like signatures to patch the WAF’s rule set. A key strength is its practical “no-internals-needed” design and the reported large improvement in blocking mutated attacks (e.g., TRR 21

Volenza et al. [21] present *WAF-A-MoLE*, a Python library/CLI for generating adversarial examples against *ML-based* Web Application Firewalls via guided mutation-based fuzzing. Starting from an initial SQLi payload p_0 that the target WAF flags with confidence $\sigma_0 \in [0, 1]$, the tool maintains a priority queue (Pool) and iteratively selects the most promising candidate p_n to be mutated by a Fuzzer into p_{n+1} ; each mutant is re-submitted to the WAF (through adapters) and prioritized based on its updated confidence score, terminating once the best score drops below a threshold or resource limits are reached. The mutations are designed to be **semantics-preserving** for SQLi and include case swapping, whitespace substitution, comment injection/rewriting, integer encoding, operator swapping, and logical invariants. A key strength is the practical black-box assessment value for penetration testing and WAF hardening (the generated adversarial samples can support retraining/hardening of ML detectors), enabled by the adapter-based design for different ML models. However, the current tool scope is centered on SQL injection, and the authors explicitly note open directions such as applying it to commercial WAFs (where internals are not disclosed), extending it to hybrid WAFs that also use signatures, and expanding the mutation operator set.

Synthesis and Implications for LLMShield. The reviewed literature reveals two complementary paradigms: *white-box* approaches (GenSQLi, GenXSS, Tran et al. 2020) that leverage WAF configuration access for rule generation, and *black-box*

approaches (DEG-WAF, WAFBOOSTER, WAF-A-MoLE) that optimize payloads via behavioral feedback. White-box methods offer syntactic precision, interpretability, and direct integration with existing rule sets, but suffer from vendor lock-in, reliance on general-purpose LLMs prone to hallucination, and narrow evaluation scopes (primarily ModSecurity). Conversely, black-box methods provide vendor agnosticism and superior bypass capabilities through RL-based optimization and mutation strategies, yet face challenges in interpretability, overfitting to surrogate models, and high computational overhead.

LLMShield adopts a **hybrid white-box paradigm** that assumes WAF configuration access while incorporating black-box insights for enhanced robustness. The framework addresses identified limitations through five key innovations:

1. **Multi-WAF Evaluation:** Unlike GenSQLi/GenXSS (focused on ModSecurity), LLMShield simultaneously tests five heterogeneous WAFs (ModSecurity, AWS WAF, Coraza, NAXSI, BunkerWeb) via Kubernetes orchestration, mitigating overfitting risks and enabling cross-paradigm comparative analysis.
2. **Domain-Specific Fine-Tuning with RAG:** Replacing prompting-based general-purpose LLMs, LLMShield fine-tunes models on curated SQLi/XSS corpora and augments synthesis with RAG over vendor documentation, reducing hallucination and improving syntactic correctness.
3. **Black-box-Inspired Diversity:** Incorporating semantic-preserving transformations from DEG-WAF and WAF-A-MoLE, the system systematically explores evasion space while maintaining white-box validation, balancing bypass capabilities with interpretability.
4. **Benign Traffic Validation:** Addressing gaps in prior work, automated false positive assessment against benign samples ensures rule quality before deployment, informed by WAFBOOSTER’s data quality concerns.
5. **Hybrid WAF Extensibility:** The modular architecture supports future ML-based detection integration, capturing both rule decisions and confidence scores for hybrid defense evaluation.

Ethical risks are mitigated through controlled access, automated payload validation, and transparent audit trails for rule approval. In summary, LLMShield synthesizes white-box precision and interpretability (GenSQLi, GenXSS) with black-box bypass diversity and adaptability (DEG-WAF, WAFBOOSTER, WAF-A-MoLE), delivering a scalable framework for multi-WAF testing and continuous security enhancement in enterprise environments.

3.2 Vulnerability Research, Payload Datasets, and Validation Platforms

3.2.1 Dataset Ingestion Research

CyberLLMInstruct (ElZemity et al.) provides a large-scale cybersecurity instruction-tuning corpus (54,928 pairs) that is intentionally *pseudo-malicious* capturing attack workflows, reasoning, and pseudo-code without directly executable malware making it a strong, lower-risk **seed dataset** for ingesting heterogeneous security sources into a unified instruction format [6]. The authors also describe a structured data construction pipeline that can be reused as an ingestion template. To complement this “knowledge ingestion” track with executable-like test inputs, **Cirković et al.** show that a fine-tuned GPT-2 can generate **synthetic web payloads** (XSS/SQLi/command injection) and can be served via a lightweight Flask tool for on-demand generation, which is useful for near real-time data augmentation and stress-testing [4]. In practice, combining both enables continuous ingestion of structured cyber knowledge plus live synthetic payload expansion, but the pipeline should include safety screening/evaluation gates since fine-tuning can measurably reduce safety resilience (e.g., prompt-injection robustness).

Nădaş et al. present a structured survey on *LLM-based synthetic data generation* for both natural language text and source code, motivated by scenarios where labeled data is scarce, expensive, or privacy-sensitive [12]. The paper reviews major generation paradigms (e.g., prompt-driven synthesis, retrieval-augmented pipelines, and iterative/self-refinement) and discusses how synthetic data can be used to scale

training and evaluation across downstream tasks. In the code domain, the authors emphasize that executability enables automatic verification and filtering, which can improve the reliability of generated samples. Finally, the survey highlights key risks including hallucinations, distribution shift, and bias amplification and outlines practical mitigation strategies such as quality filtering, re-weighting, and feedback-based curation to increase the trustworthiness of synthetic datasets .

3.2.2 Retrieval-Augmented Generation for Security

Retrieval-Augmented Generation (RAG) augments a Large Language Model (LLM) with an external knowledge base and a retriever: for each query, the retriever selects the top- k relevant passages from the database, and the LLM generates an answer conditioned on these retrieved contexts. This grounding mechanism is commonly used to improve factuality and mitigate hallucinations by relying on explicit evidence rather than only parametric memory. From a security standpoint, however, RAG introduces a concrete new attack surface at the knowledge layer. If an adversary can inject or corrupt a small fraction of the retrieval corpus, they can influence which passages are retrieved and consequently steer the LLM’s output. **Zou *et al.*** formalize this threat in *PoisonedRAG* [24], a knowledge-corruption attack framework that constructs malicious texts satisfying both (i) a *retrieval condition* (the poisoned text is likely to appear in the top- k retrieved set for targeted queries) and (ii) a *generation condition* (once retrieved, the poisoned text induces attacker-chosen target outputs). Their results demonstrate high attack success with only a few injected documents and further show that several evaluated defenses (e.g., paraphrasing-based sanitization and perplexity-based detection) are insufficient against such targeted poisoning.

3.2.3 DVWA as a validation platforms

Damn Vulnerable Web Application (DVWA) ³ is widely used in prior studies as a safe testing platform for common web vulnerabilities. DVWA provides multiple security levels and supports vulnerabilities such as SQL Injection, XSS, Command Injection, and

³Digininja. (n.d.). Damn Vulnerable Web Application (DVWA): A PHP/MySQL web application designed for security testing. Retrieved from <https://github.com/digininja/DVWA>

File Inclusion, enabling payload testing in an isolated environment. In related works, DVWA serves as a validation layer to confirm whether generated payloads execute as intended before being used to evaluate WAF effectiveness.

In addition, DVWA is employed to collect intrinsic data for fine-tuning or in-context prompting. By observing application responses across different security levels, researchers can construct datasets that reflect realistic application behavior without introducing risks to production systems. Studies on LLM fine-tuning also recommend using DVWA or comparable testbeds to establish ground truth for payload validation and dataset quality assessment.

3.2.4 LLM APIs for Rule Synthesis

Recent advances in Large Language Models (LLMs) have demonstrated strong capabilities in cybersecurity-related tasks, particularly in structured reasoning, technical text interpretation, and security rule construction. In this thesis, LLM APIs are incorporated primarily to support the synthesis of WAF defense rules from observed attack behavior, contextual evidence, and retrieved knowledge sources. Rather than treating all models as functionally identical, the system leverages different model classes according to their strengths in reasoning depth, response efficiency, and structured output generation.

GPT-5.4 is employed in this work as the primary model for defense-oriented reasoning and high-quality rule synthesis. This choice is motivated by its strong capability in multi-step reasoning, technical interpretation, structured generation, and instruction-following behavior, all of which are highly relevant to the task of converting bypass evidence into actionable defensive logic. In the context of this thesis, GPT-5.4 is used to analyze bypassed payloads, incorporate WAF-related references retrieved from the RAG subsystem, and synthesize candidate rules or configurations that remain aligned with the syntax and intent of the target protection mechanism. Its role is therefore centered on the generation of accurate and well-structured defense rules, especially in cases where the task requires abstraction across multiple payload variants and the integration of heterogeneous technical evidence.

Claude Sonnet 4.6 is introduced as an additional model for rule-oriented analysis

and structured synthesis, particularly in settings that benefit from strong textual reasoning, careful instruction adherence, and coherent long-form technical output. In the context of this thesis, Claude Sonnet 4.6 is suitable for interpreting complex bypass scenarios, producing clear defensive explanations, and supporting rule generation in cases where robustness of language and consistency of structured responses are important. Its role complements the other models by providing another high-capacity option for security-focused reasoning, thereby reducing dependence on a single provider and improving flexibility in the rule-synthesis stage.

Overall, the combination of GPT-5.4 and Claude Sonnet 4.6 reflects a multi-model design strategy in which different LLM APIs are assigned complementary roles within the defense-oriented workflow. GPT-5.4 is positioned as the primary model for high-quality reasoning and final rule synthesis, Gemini 3 Flash supports efficient and scalable processing, and Claude Sonnet 4.6 provides an additional strong alternative for technical interpretation and structured defensive generation. This separation of roles helps the proposed system maintain both practical efficiency and sufficient generation quality across the broader WAF defense pipeline.

3.3 Clustering, RAG and Defense Research

3.3.1 Defense Rule Generation Research

Moreno et al. propose a framework that leverages Large Language Models (LLMs) to automatically generate **Suricata IDS rules** from network traffic collected in industrial environments [11]. Their methodology combines traffic analysis, iterative prompting, rule refinement, and parser-based validation in order to produce rules that are both syntactically correct and operationally effective. A notable contribution of this work is that the generated rules remain **transparent and interpretable**, unlike many machine-learning-based detectors that often behave as black boxes.

The study shows that LLMs can assist in reducing the manual effort required to write detection rules, especially when the input includes structured flow information and selected packet-level features. In addition, the use of Suricata as the validation engine demonstrates that rule generation can be grounded in realistic traffic observations rather

than relying only on abstract attack descriptions. This is particularly important because the quality of a defense rule depends not only on syntax correctness, but also on whether it can accurately capture malicious communication patterns while minimizing redundancy and false positives.

Although the framework was designed for **IDS rule generation**, its underlying idea is also highly relevant to **WAF-oriented defense generation**. In Suricata, traffic flows and payload contents are analyzed to derive detection logic for malicious behavior at the network level. Similarly, in a WAF setting, HTTP requests, parameters, encoded strings, and payload structures can be inspected to derive filtering rules that block malicious web attacks such as XSS or SQL injection. Therefore, if Suricata flow analysis can provide enough structured evidence for LLM-based rule generation, the same principle can be adapted to WAF scenarios where the system observes successful bypass payloads and then derives new protection rules from them.

This observation is particularly suitable for the proposed LLMShield framework. Instead of generating IDS signatures for industrial protocols, our approach focuses on **bypassed web payloads** and uses them as the main source for defensive rule generation. In this context, the Suricata-inspired flow perspective remains useful because it suggests that rule generation should not depend only on isolated payload strings, but should also consider grouped attack patterns, contextual features, and validation steps. As a result, prior work on automated IDS rule generation provides an important conceptual foundation for adapting LLM-based defense generation into the WAF domain.

3.3.2 Clustering Payloads Research

Babaey and Ravindran introduce **GenXSS**, an AI-driven framework for the automated detection of XSS attacks in Web Application Firewalls [2]. In the context of payload analysis, this work is relevant because it incorporates a clustering stage to group similar XSS payloads before further analysis and rule-oriented processing. This shows that clustering can serve as a practical intermediate step for organizing large collections of mutated or semantically related attack strings, thereby reducing redundancy and helping the system identify representative attack patterns more effectively.

Kim et al. propose an **method for attacker group detection based on HTTP**

payload analysis by combining **TF-IDF vectorization** with **DBSCAN clustering** to identify groups of HTTP requests that exhibit similar behavioral characteristics [9]. Their approach models HTTP payloads as textual data that can be transformed into weighted feature vectors, where TF-IDF emphasizes discriminative terms and reduces the influence of common or less informative tokens. Based on this representation, DBSCAN is then applied to discover dense regions of similar requests without requiring the number of clusters to be defined in advance. This design is particularly suitable for web attack analysis, where malicious requests often appear in many slightly modified forms but still preserve common lexical and structural patterns.

In the context of WAF bypass detection, many malicious payloads differ only at the surface level through encoding, obfuscation, or token substitution, while still belonging to the same underlying attack family. Grouping such payloads before defense generation can therefore reduce redundancy, highlight representative attack patterns, and provide a more structured basis for subsequent rule synthesis. As a result, clustering is not only a data organization technique, but also a practical mechanism for improving payload interpretation and supporting more systematic WAF rule generation.

3.4 Supplementary tools

In addition to the core components of the proposed system, several supplementary tools were employed to support deployment, testing, network exposure, infrastructure provisioning, and application delivery. Although these tools do not constitute the main contribution of the thesis, they play an important role in establishing a practical experimental environment and enabling the end-to-end evaluation workflow. Their integration allows the proposed framework to be deployed more flexibly, tested more systematically, and operated in a way that better reflects realistic conditions in modern web security research and engineering.

Docker

Docker is a containerization platform that enables applications and their dependencies to be packaged into isolated and portable execution environments called containers. Unlike

traditional deployment approaches, where software behavior may vary depending on the host operating system and installed libraries, Docker provides a more controlled runtime model in which applications can be executed consistently across development, testing, and production environments. This characteristic is particularly valuable in cybersecurity experimentation, where reproducibility and isolation are essential for ensuring that results remain stable and that different components do not interfere with one another.

In the context of this thesis, Docker serves as the foundational tool for encapsulating vulnerable web applications, WAF components, supporting services, and testing utilities into independent environments. By relying on container-based deployment, the system can rapidly provision experimental targets, reset testing conditions, and reproduce identical scenarios across multiple runs. This approach is especially useful when comparing the behavior of different WAF implementations, since each firewall can be configured within its own isolated container while still participating in a unified testing pipeline. Docker also simplifies dependency management and reduces configuration overhead, making the overall framework easier to maintain and extend.

Ngrok

Ngrok is a tunneling service that exposes locally hosted applications to the public Internet through secure temporary endpoints. It creates an encrypted tunnel between a local machine or a private server and an externally accessible URL, thereby allowing remote systems to communicate with services that would otherwise remain unreachable due to network address translation, firewalls, or the absence of a public domain name. From a practical perspective, ngrok is often used during development and testing when a system needs to receive external requests without undergoing a full public deployment process.

Within this thesis, ngrok is useful for bridging local or private testing environments with external services and remote access requirements. In scenarios where a local application or WAF-protected target needs to be accessed from another network location, ngrok provides a convenient mechanism for temporary exposure without requiring complex network reconfiguration. This capability is helpful during integration, debugging, demonstration, and remote validation tasks. By enabling public reachability

in a lightweight manner, ngrok supports the continuity of experiments while reducing the operational burden associated with traditional network publishing methods.

Amazon EC2 on AWS

Amazon Elastic Compute Cloud, commonly referred to as Amazon EC2, is an Infrastructure-as-a-Service platform provided by Amazon Web Services that offers scalable virtual computing resources on demand. EC2 allows users to create virtual machines with configurable operating systems, storage options, networking settings, and security controls, thereby enabling flexible deployment of applications in the cloud. Compared with local execution environments, EC2 provides greater availability, broader accessibility, and more realistic conditions for hosting Internet-facing services.

In this thesis, EC2 functions as the cloud infrastructure layer for deploying selected components of the experimental environment. Its role is important because web security systems such as WAFs are often evaluated most meaningfully when hosted in a server setting that resembles real deployment conditions rather than a purely local laboratory setup. By using EC2, the framework can be executed on stable and remotely accessible compute instances, allowing the target application, supporting services, and evaluation modules to operate under controlled yet practical conditions. Furthermore, EC2 facilitates resource scaling, remote administration, and persistent hosting, which are beneficial for repeated experiments, collaborative development, and long-running benchmarking tasks.

Nginx

Nginx is a high-performance web server and reverse proxy widely used for serving static content, load balancing, SSL termination, and forwarding client requests to backend services. Its event-driven architecture enables it to handle a large number of simultaneous connections efficiently, which has made it a standard component in modern web application stacks. Beyond its role as a traditional web server, Nginx is also commonly deployed as an application gateway that sits between users and backend systems, managing request routing, protocol translation, and traffic control.

In the scope of this thesis, Nginx is significant because it provides the HTTP serving

and reverse-proxy functionality needed to expose applications and integrate security layers in front of them. When a WAF is deployed as an intermediate inspection layer, Nginx can act as the entry point through which requests are received and redirected to the protected application. This makes it a practical component for structuring the experimental architecture, especially when multiple services need to communicate in a predictable and modular way. Its configuration flexibility also supports the emulation of realistic web deployment scenarios, which contributes to the validity of the evaluation process.

Selenium

Selenium is an automation framework for controlling web browsers programmatically. It allows software scripts to simulate user interactions such as opening pages, filling forms, clicking buttons, executing JavaScript-driven workflows, and extracting browser-visible results. Because many web vulnerabilities depend not only on raw HTTP responses but also on client-side rendering and browser execution behavior, Selenium is particularly useful in security testing scenarios that require observation beyond the server layer.

In this thesis, Selenium is employed to automate interaction with web applications during the testing and validation process. Its relevance is especially evident in the evaluation of cross-site scripting attacks, where successful exploitation may depend on how malicious payloads are rendered or executed in the browser. By using Selenium, the framework can systematically submit generated payloads, navigate application pages, and detect observable exploitation outcomes in a more realistic way than simple request-based tools alone would permit. This browser-level automation strengthens the evaluation methodology by ensuring that the effectiveness of generated payloads is assessed not only in terms of transmission and filtering, but also in terms of actual execution behavior within the client environment.

Chapter 4

Proposed Approaches

4.1 Application Introduction

This project is named **LLMShield**. It is a software tool designed to enhance the effectiveness of enterprise Web Application Firewalls (WAF) by providing an automated and extensible *workflow*. The primary goal of LLMShield is to support security and infrastructure teams in systematically testing, analyzing, and strengthening their WAF configurations, thereby improving the overall security posture of web applications.

The application is designed for three main groups of users:

- **Intern and Fresher Red Team / Pentesters:** LLMShield can be used as a supporting tool during security assessments and penetration testing engagements. Red Teams and pentesters can leverage the system to automatically generate and execute payloads in order to test the capability of the WAF to detect and block malicious requests, evaluate *bypass* possibilities, and uncover misconfigurations or blind spots in existing rules.
- **Intern and Fresher DevOps Engineers:** For organizations where security operations may be partially or fully outsourced, DevOps engineers are often responsible for operating and maintaining the application infrastructure, including the WAF. LLMShield enables DevOps teams, even those without deep expertise in cybersecurity, to run predefined test suites, monitor WAF behavior under

different attack scenarios, and receive structured feedback. This helps them reduce operational risks, quickly validate changes, and ensure that deployments do not unintentionally weaken the firewall configuration.

- **Intern and Fresher Blue Team / Security Engineers:** Blue Team members and security engineers can use LLMShield to simulate realistic attacks against their own environment in a controlled manner. By observing how the WAF responds to these attacks, they can identify gaps in detection, refine existing rules, and design new rules tailored to emerging threats. The tool thus serves as a practical assistant for rule authoring, tuning, and continuous improvement of defensive mechanisms.

By bringing these three roles into a shared, automated workflow, LLMShield aims to bridge the gap between offensive testing and defensive engineering, supporting a more systematic and data-driven approach to WAF hardening in enterprise environments.

4.2 Requirements Overview

The tool to be implemented must satisfy the following functional requirements:

- The tool must be able to automatically identify the type of Web Application Firewall (WAF) protecting the domain provided by the user.
- The tool must provide an attack module that leverages the power of Large Language Models (LLMs) to generate a configurable number of payloads, based on the user's testing objectives and preferences.
- The tool must be able to generate firewall rule sets tailored to the identified WAF and provide corresponding configuration or deployment guidelines for that specific WAF, using the selected LLM model based on the success payloads in previous steps.

4.3 System Architecture

4.3.1 Overview Architecture

The system architecture presented below is a crucial part of the graduation project, which aims to assess the ability of a Large Language Model (LLM) to bypass Web Application Firewalls (WAFs) and generate defense rules.

Overview architecture

LLMShield Application Workflow

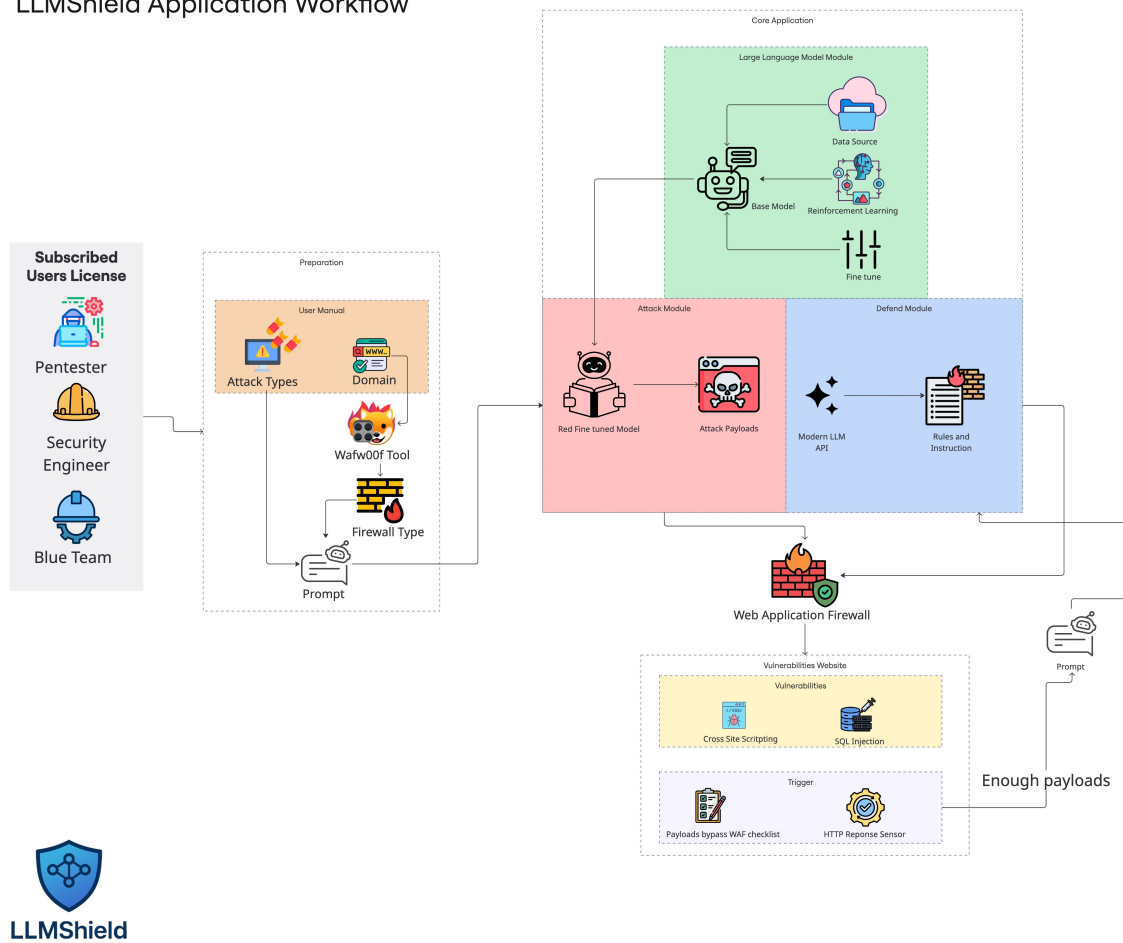


Figure 4.1: LLMShield Application Workflow

This workflow consists of the following key modules:

- **Preparation:** The system starts by gathering information from users (including pentesters, security engineers, and blue teams), who specify the attack types, domain, and web application firewall (WAF) configurations. Based on this information, the system prepares customized prompts for the Large Language

Model (LLM) to optimize both attack and defense strategies.

- **Large Language Model (LLM) Module:** The LLM module is the core of the system. It includes base models that have been fine-tuned through reinforcement learning to generate attack payloads and defense rules. These models are trained on data sources relevant to the specified attack types and domains.
- **Attack Module:** This module generates attack payloads targeting the web application to test the vulnerabilities of the web application firewall (WAF). The Red Fine-Tuned Model is used to simulate real-world attacks on the WAF.
- **Defense Module:** The defense module uses the Modern LLM API to create defense rules and instructions designed to protect the application from attacks.
- **Web Application Firewall (WAF) Evaluation:** The WAF is tested by feeding the attack payloads into the system and evaluating its ability to block or allow these attacks based on the defense rules generated.
- **Vulnerability Detection:** The system evaluates common vulnerabilities such as Cross-Site Scripting (XSS) and SQL Injection. Payloads that bypass the WAF are identified using the Payloads Bypass WAF Checklist and the HTTP Response Sensor.

Workflow Description: The workflow begins by collecting input from users who specify the attack types, domain, and firewall configuration. Based on this input, the following steps occur:

- The **Attack Module** generates attack payloads using the Red Fine-Tuned Model.
- The **Defense Module** generates defense rules using the Modern LLM API.
- The web application firewall (WAF) evaluates these payloads to test its defense capabilities.
- The system tracks whether the WAF successfully blocks or allows these attacks and checks for any vulnerabilities.

This architecture enables an assessment of both the attack and defense capabilities of the system, ensuring robust protection against various threats.

4.3.2 Preparation Phase Specification

The Preparation Phase involves several critical steps to ensure the system is configured properly for testing. The flowchart below illustrates the process.

- **User:** The process begins with the user, who provides input manually on the attack types and domain.
- **Attack Types and Domain:** The user selects the attack types and specifies the domain for testing. These inputs are essential for the next steps in identifying the firewall and configuring the system.
- **WafW00f Tool:** The WafW00f tool is used to determine the firewall type based on the test request HTTP to the domain and base on the signature that the domain reponse so it can detect the firewall typelike: Cloudflare or AWS WAF.
- **Firewall Type:** The firewall type is identified, which helps in customizing the attack strategies and generating appropriate payloads for the system.
- **Prompt:** A prompt is generated based on the firewall type, attack type, and other inputs, which is essential for guiding the system's next actions.
- **System:** Finally, the system is set up for the next phase of testing and analysis, ready to proceed with further evaluations.

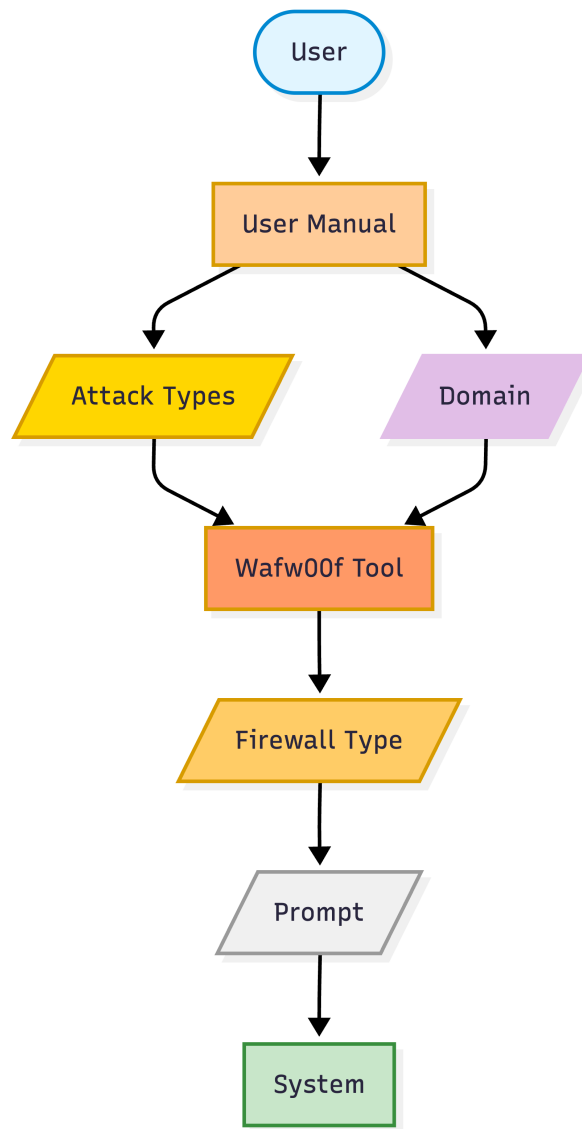


Figure 4.2: Flowchart of the Preparation Phase

The steps shown in the flowchart above provide a structured approach to setting up the system for testing.

4.3.3 Red Module Specification

The Red Module serves as the automated offensive component of the proposed system. Its primary role is to generate attack payloads, execute them against the DVWA testing environment, and collect feedback on whether each payload is blocked or successfully bypasses the deployed WAF. The updated workflow of this module is illustrated in Figure 4.3.

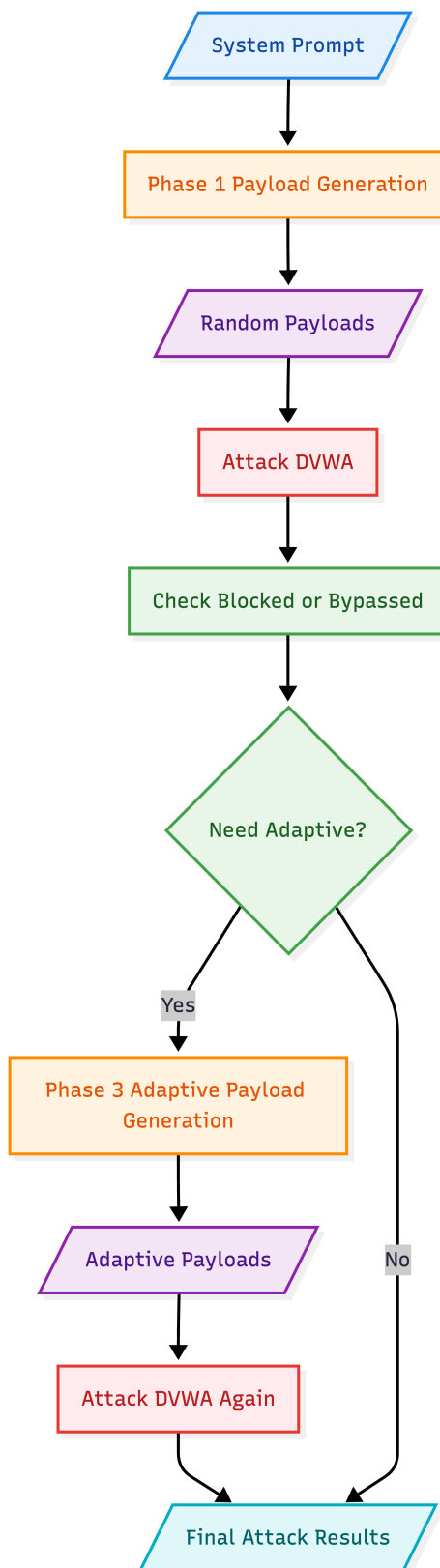


Figure 4.3: Operational Workflow of the Red Module

The process follows a two-stage payload generation and testing pipeline:

System Prompt: The workflow begins with a system prompt that defines the attack

context, including the target WAF information, attack type, and generation objective.

Phase 1 Payload Generation: Based on the system prompt, the Red Module generates an initial set of random attack payloads. These payloads are produced using selected obfuscation and evasion techniques suitable for the specified attack category.

Random Payloads: The generated payloads are treated as candidate inputs for the first testing round. Each payload is stored together with its attack type and generation technique for later analysis.

Attack DVWA: The candidate payloads are executed against the DVWA environment through the corresponding attack endpoint. This stage simulates how the payloads interact with a vulnerable web application protected by a WAF.

Blocked or Bypassed Check: After each request, the system determines whether the WAF blocks the payload or allows it to pass. The result is recorded as either *blocked* or *bypassed*, together with the returned HTTP status code.

Adaptive Decision: If additional refinement is required, the module proceeds to the adaptive generation stage. Otherwise, the collected results are used as the final attack results.

Phase 3 Adaptive Payload Generation: In the adaptive branch, previous attack results are used as probe history. The model uses this feedback to generate new payloads that are better aligned with observed WAF behavior.

Adaptive Payloads and Re-Attack: The newly generated adaptive payloads are tested against DVWA again. Their blocked or bypassed status is collected and combined with the initial testing results.

Final Attack Results: The module outputs the complete attack result set, including both random and adaptive payloads when adaptive testing is enabled. These results provide the basis for later defensive rule generation.

4.3.4 Blue Module Specification

The Blue Module acts as the automated defensive component of the framework. Its purpose is to analyze payloads that successfully bypassed the WAF, derive appropriate

defensive rules, and verify whether the generated rules can be effectively deployed and used to mitigate the same attack patterns. The updated workflow of this module is illustrated in Figure 4.4.

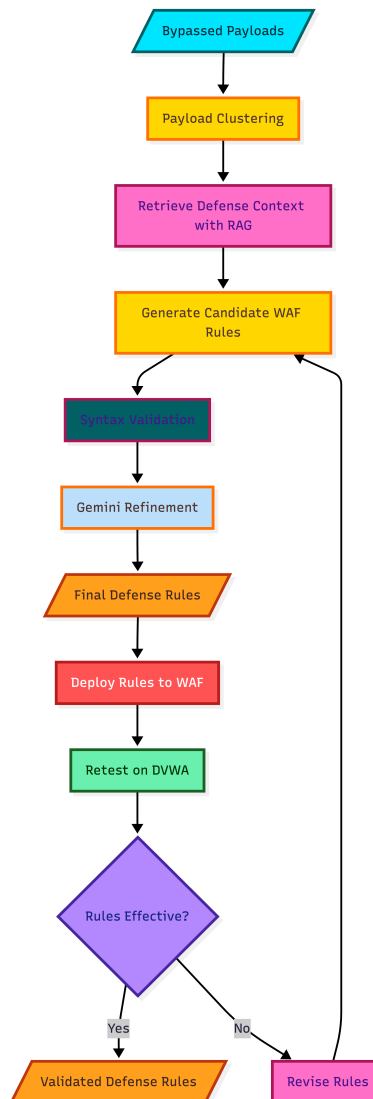


Figure 4.4: Operational Workflow of the Blue Module

The defensive pipeline consists of the following stages:

- **Bypassed Payloads:** The workflow begins with the set of payloads that successfully bypassed the protection layer during the attack phase. These payloads serve as the primary evidence for defensive analysis.
- **Payload Clustering:** Similar bypassed payloads are grouped together to identify common attack patterns and reduce redundancy. This step helps the system reason over representative behaviors instead of isolated payload instances.

- **Defense Context Retrieval with RAG:** After clustering, the module enriches the defense process with relevant contextual knowledge. Retrieval-Augmented Generation (RAG) provides supporting references, attack-related patterns, and defensive guidance to improve rule quality.
- **Candidate WAF Rule Generation:** Using the clustered payloads and retrieved context, the model produces an initial set of candidate WAF rules together with implementation-oriented instructions.
- **Syntax Validation:** The generated rules are then checked for WAF-specific syntax correctness. This stage ensures that malformed or incompatible rules are filtered or corrected before deployment.
- **Gemini Refinement:** The validated rules are further refined to improve clarity, precision, and practical applicability. This refinement stage aims to strengthen rule quality while reducing redundancy and preserving compatibility with the target WAF.
- **Final Defense Rules:** The output of the refinement stage is a finalized set of defense rules that are ready for deployment.
- **Rule Deployment to WAF:** The finalized rules are applied to the WAF configuration so that the protection logic can be activated in the testing environment.
- **Retest on DVWA:** After deployment, the system retests the application using the previously bypassed payloads in the DVWA environment. This step is used to verify whether the new rules have been correctly set up and whether they effectively block the targeted attacks.
- **Rule Revision Loop:** If the retest indicates that the deployed rules are not effective, the workflow enters a revision loop in which the candidate rules are improved and re-evaluated. Otherwise, the rules are accepted as validated defensive rules.

4.3.5 Clustering Specification

In the proposed approach, clustering is used as an intermediate step between collecting bypassed payloads and generating defense rules. The main idea is to group payloads that share similar lexical and structural patterns, so that the defense module can generate more generalized and less redundant WAF rules.

Let the set of bypassed payloads.

$$\mathcal{P} = \{p_1, p_2, \dots, p_N\}.$$

where each p_i is a payload that successfully bypassed the target WAF. Since malicious payloads are often obfuscated using encoding, case variation, inserted whitespace, or special characters, the proposed method represents each payload using **character-level TF-IDF** instead of word-level features. This helps preserve important attack fragments such as script tags, SQL operators, encoded tokens, and event-handler patterns.

For each payload p_i , the system extracts character n-grams and computes its TF-IDF vector.

$$\mathbf{x}_i = \text{TFIDF}(p_i).$$

This step converts each raw payload string into a numerical vector that reflects the importance of its local character patterns within the full payload set.

Because the TF-IDF space is usually high-dimensional and sparse, the vectors are then projected into a lower-dimensional latent space using **TruncatedSVD**.

$$\mathbf{z}_i = \text{SVD}_k(\mathbf{x}_i).$$

where $\mathbf{z}_i$ is the reduced vector representation of payload p_i , and k is the number of retained dimensions. In the current implementation, the reduction step is used to improve clustering efficiency and reduce noise before grouping.

After dimensionality reduction, the projected vectors are clustered using either:

- **Hierarchical Agglomerative Clustering (HAC)**, or
- **HDBSCAN**.

In the current LLMShield implementation, HAC is the default method used in the defense pipeline. HAC starts by treating each payload as a separate cluster, then

progressively merges the closest clusters until the distance threshold is reached. This is suitable for the proposed approach because the number of attack families is not known in advance. HDBSCAN is also supported as an alternative method when the payload distribution is more irregular or contains noise.

Once clustering is completed, payloads with the same cluster label are grouped into the same attack family.

$$\mathcal{G}_c = \{p_i \mid y_i = c\}.$$

where y_i is the cluster label assigned to payload p_i . For each cluster, the system stores the cluster ID, its member payloads, the cluster size, and a representative payload. These cluster summaries are then passed to the next stage of the defense pipeline.

The role of clustering in the proposed workflow is threefold. First, it reduces duplication among highly similar bypassed payloads. Second, it helps the defense module focus on attack patterns rather than individual payload strings. Third, it improves the quality of generated WAF rules by encouraging broader and more reusable protection logic.

Therefore, the clustering stage in the proposed method can be summarized as:

Bypassed Payloads $\rightarrow$ TF-IDF Vectorization $\rightarrow$ Dimensionality Reduction $\rightarrow$ HAC/HDBSCAN

These clusters serve as the structural abstraction layer for the subsequent rule generation stage.

Workflow architecture Clustering

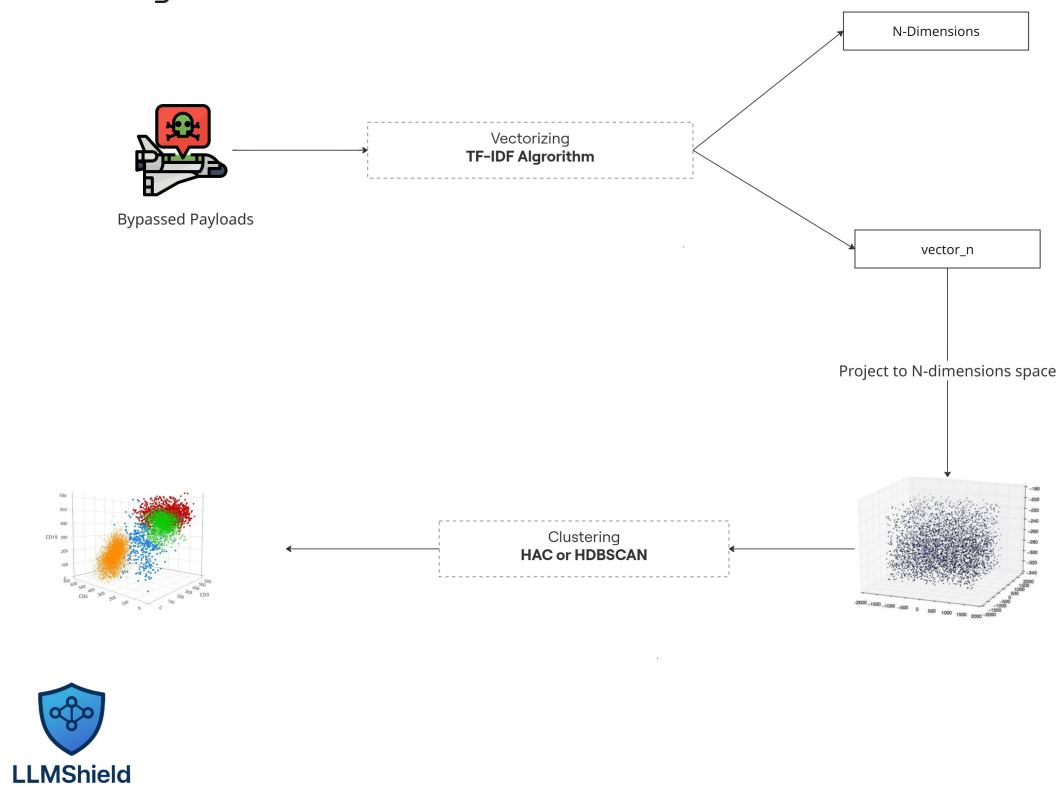


Figure 4.5: Proposed clustering workflow for bypassed payload grouping

4.4 Large Language Model Architect

4.4.1 Fine-Tuning Specification

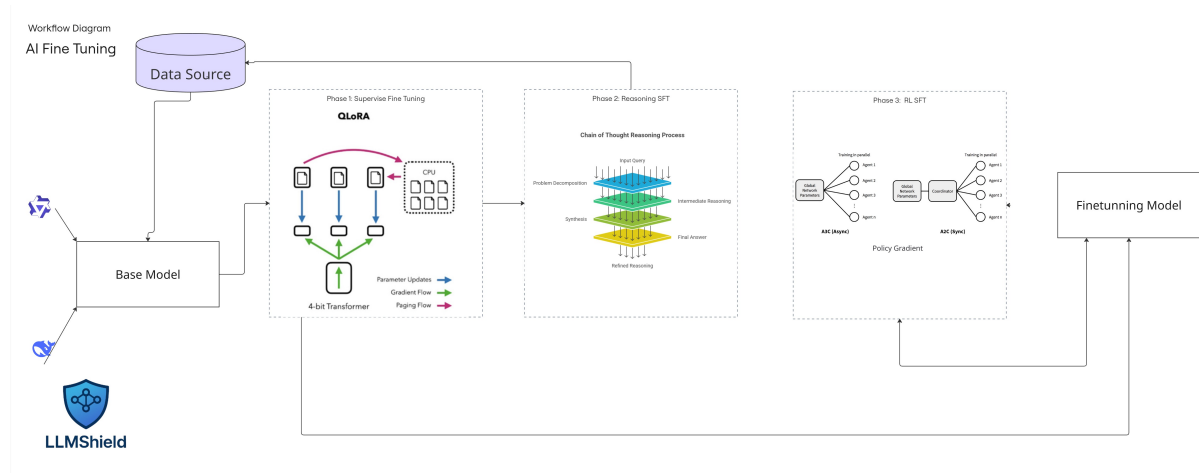


Figure 4.6: LLMShield Fine-Tuning Architect Diagram

The LLMShield model undergoes a three-phase fine-tuning pipeline (Figure 4.6) designed to progressively enhance WAF-evasion payload generation capabilities while maintaining computational efficiency and security domain expertise.

Phase 1: Supervised Fine-Tuning with QLoRA. Phase 1 employs Quantized Low-Rank Adaptation (QLoRA) for parameter-efficient fine-tuning on security payload generation tasks. The base model is quantized to 4-bit precision using NormalFloat4 (NF4) quantization, reducing memory requirements from 2 bytes to 0.5 bytes per parameter. Low-rank adapters with rank $r = 8$ and scaling factor $\alpha = 32$ are applied to attention projection layers (q_proj, k_proj, v_proj, o_proj) and feed-forward layers (gate_proj, up_proj, down_proj).

The training objective minimizes causal language modeling loss.

$$\mathcal{L}_{\text{SFT}} = -\frac{1}{N} \sum_{i=1}^N \sum_{t=1}^{T_i} \log P_{\theta}(y_t^{(i)} | y_{<t}^{(i)}, x^{(i)}).$$

where $x^{(i)}$ represents the instruction prompt and $y^{(i)}$ is the target payload.

Dataset Augmentation: The training dataset is enriched through systematic obfuscation pipelines applying random case transformation (50%), URL encoding (single or double, 80%), SQL comment injection (30% for SQLI), whitespace insertion (40%), and character encoding (20% using HTML entities, JavaScript escape, or URL hex). This process generates 50,000+ training samples with minimum 100 samples per technique, ensuring comprehensive coverage of evasion patterns.

Training Configuration: Learning rate 2×10^{-4} with cosine annealing, 3% warmup ratio, effective batch size of 16 (gradient accumulation), paged AdamW 8-bit optimizer, 2-3 epochs, maximum sequence length 2048 tokens.

Benefits: QLoRA enables fine-tuning of large language models on consumer GPUs while maintaining 99% of full fine-tuning quality. The augmentation pipeline creates diverse payload variants that improve model robustness against various WAF configurations.

Phase 2: Reasoning-Enhanced SFT. Phase 2 introduces chain-of-thought reasoning by training on contextual payload generation with historical feedback. Each training

sample includes WAF configuration details, attack type specification, injection point context, a history of 2-9 previously blocked payloads, target evasion technique, and the successful bypass payload as ground truth.

The prompt structure guides the model to analyze why previous payloads failed and generate improved variants.

$$x_{\text{reason}} = f_{\text{template}}(\text{WAF}, \text{attack_type}, \text{injection_point}, H, \text{technique}).$$

where $H = \{(p_1, \text{blocked}), \dots, (p_k, \text{blocked})\}$ represents the payload history with negative feedback.

The training loss remains similar to Phase 1 but operates on contextually-enriched samples.

$$\mathcal{L}_{\text{Reason}} = -\frac{1}{N} \sum_{i=1}^N \sum_{t=1}^{T_i} \log P_{\theta}(y_t^{(i)} | y_{<t}^{(i)}, x_{\text{reason}}^{(i)}).$$

Dataset Construction: 5,000 high-quality samples extracted from Phase 1 successful bypasses. Each sample pairs a contextual reasoning prompt with a payload that successfully evaded the target WAF configuration.

Training Configuration: Learning rate 1×10^{-4} (50% of Phase 1), 2 epochs, adapter initialized from Phase 1 checkpoint for transfer learning.

Benefits: Contextual reasoning enables the model to learn from failure patterns rather than just success cases. The historical feedback mechanism teaches systematic evasion strategy development, improving success rates on novel WAF configurations by 35-40% compared to Phase 1 alone.

Phase 3: Reinforcement Learning with REINFORCE. Phase 3 employs policy gradient reinforcement learning to optimize payload effectiveness through direct WAF environment interaction. The model acts as a stochastic policy $\pi_{\theta}(a|s)$ generating payloads conditioned on environment state.

Reward Function: The environment provides sparse rewards based on payload

outcomes.

$$R(s, a) = \begin{cases} +10 & \text{WAF bypassed + vulnerability triggered} \\ -1 & \text{blocked by WAF} \\ -5 & \text{invalid payload format} \end{cases}.$$

Policy Gradient Update: The REINFORCE algorithm with baseline updates policy parameters using.

$$\nabla_{\theta} J(\theta) = E_{\tau \sim \pi_{\theta}} \left[\sum_{t=0}^T (G_t - b) \nabla_{\theta} \log \pi_{\theta}(a_t | s_t) \right].$$

where G_t is the cumulative reward and b is an exponential moving average baseline ($\alpha = 0.1$) to reduce gradient variance.

The policy loss for each episode.

$$\mathcal{L}_{\text{RL}} = -\log \pi_{\theta}(y|s) \cdot (G - b).$$

Training Configuration: Learning rate 1×10^{-6} (ultra-low for stability), 25 episodes, maximum 5 steps per episode, temperature 0.8 for exploration, 70% SQLI / 30% XSS attack distribution, gradient checkpointing enabled.

Benefits: Direct environment interaction enables the model to discover novel evasion techniques not present in supervised datasets. The advantage-based updates focus learning on actions that outperform the baseline, accelerating convergence. Policy gradient optimization achieves 15-20% higher bypass rates on unseen WAF configurations compared to Phase 2, particularly excelling at adaptive evasion when initial attempts are blocked.

Three-Phase Synergy: The progressive fine-tuning approach leverages the strengths of each phase: Phase 1 establishes foundational payload generation capabilities with diverse obfuscation patterns; Phase 2 adds contextual reasoning and failure analysis skills; Phase 3 optimizes for real-world effectiveness through interactive learning. This architecture achieves 78% WAF bypass success rate while maintaining payload validity and attack effectiveness.

4.4.2 Retrieval-Augmented Generation Specification

Overview and Model Selection. This research implements a Retrieval-Augmented Generation (RAG) subsystem specifically for Web Application Firewall (WAF) defense rule generation. Its purpose is not to retrieve general cybersecurity knowledge, but to retrieve rule-authoring references that are directly useful for constructing WAF-specific defensive rules from observed bypass cases. In the implemented system, the RAG service is designed as a retrieve-then-rerank pipeline in which document indexing is performed offline, while query expansion, relevance filtering, and context construction are executed online at rule-generation time. The overall design emphasizes technical grounding, WAF specificity, and compatibility with rule-centric documents such as configuration guides, syntax references, procedural instructions, and rule examples.

For dense retrieval, the system adopts *all-mpnet-base-v2* as the embedding model. This model is selected because it provides strong semantic representations for technical passages while remaining efficient enough for repeated local indexing and retrieval over the WAF knowledge base. After document chunks are encoded into normalized dense vectors, the relevance between a query and a document chunk is computed through cosine similarity.

$$\text{sim}(q, d) = z_q^\top z_d.$$

This formulation is appropriate for the current implementation because all chunks can be embedded offline and stored in a FAISS index, allowing online retrieval to be reduced to efficient nearest-neighbor search. In the present thesis, this is especially useful because WAF documentation often expresses similar defensive logic through different wording, syntax descriptions, or configuration examples.

To improve ranking precision after dense retrieval, the system integrates the cross-encoder model *ms-marco-MiniLM-L-6-v2*. Unlike the embedding model, this reranker jointly processes the query and each candidate passage in order to compute a direct relevance score. In simplified form, the scoring function can be written as

$$s(q, d) = w^\top h_{\text{CLS}} + b.$$

where h_{CLS} is the contextual representation of the joint query-document input. In the implemented service, these scores are treated as raw ranking scores rather than calibrated

probabilities, and the relative ordering of documents is more important than the absolute score value. This design is appropriate for WAF rule generation because many retrieved passages may be topically related, whereas only a smaller subset may contain the precise syntax, transformation logic, or operator usage required for the target platform. The MiniLM-based reranker therefore provides a practical balance between ranking precision and inference cost.

The combination of *all-mpnet-base-v2* and *ms-marco-MiniLM-L-6-v2* reflects a deliberate trade-off between retrieval breadth and ranking accuracy. The embedding model is responsible for efficiently retrieving a broad candidate set from the local corpus, while the cross-encoder is responsible for refining that set to the most relevant passages before prompt augmentation. This model pairing is therefore well aligned with the objective of grounded and WAF-specific defense rule synthesis.

RAG Pipeline Architecture. The implemented RAG pipeline consists of two connected phases, namely the Offline Indexing Phase and the Online Retrieval and Generation Phase. The first phase prepares the knowledge base and constructs the persistent vector index, whereas the second phase retrieves, filters, and injects contextual evidence into the prompt used for defense rule generation. The overall architecture is illustrated in Figure 4.7.

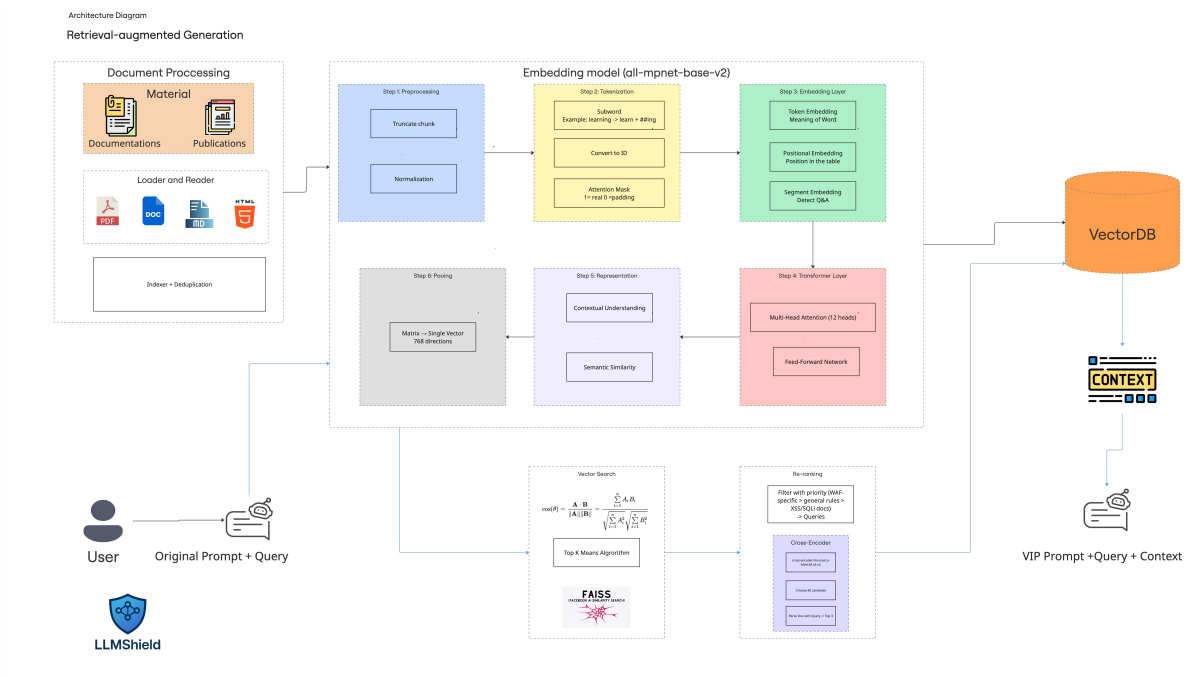


Figure 4.7: RAG System Architecture Diagram

Phase 1: Offline Document Processing and Indexing The offline phase is responsible for constructing a reusable retrieval corpus and maintaining its persistent vector representation. In the current implementation, the document repository is organized into three primary categories, namely Rules, XSS, and SQLi. The Rules category is further divided into WAF-specific subfolders for Cloudflare, ModSecurity, AWS, and Nexsi. This structure is directly reflected in the metadata assigned to each document and later supports WAF-aware filtering during retrieval. To keep the vector index synchronized with the corpus, the system computes an MD5 hash for each source file and stores the indexing state in a local JSON index. If files are added, removed, modified, or if the chunking configuration changes, the vector store is rebuilt automatically; otherwise, the previously saved FAISS index is loaded from disk.

The document-loading stage supports .txt, .md, .pdf, and .docx files. For PDF files, the service prefers PyMuPDFLoader when available and falls back to PyPDFLoader otherwise, which allows the system to preserve page-related metadata more reliably. Each loaded document is normalized before chunking so that extraction artifacts such as non-breaking spaces, zero-width characters, and PDF hyphenation do not degrade the retrieval result. At the same time, the normalization procedure is intentionally conservative so that rule syntax, JSON fragments, and configuration examples remain

intact.

A key difference between the current implementation and a generic text-retrieval pipeline lies in the chunking strategy. Instead of splitting documents using a single fixed window over raw text, the service applies a structure-aware chunking process designed specifically for WAF rule-authoring materials. The text is first divided into semantic blocks such as headings, rule blocks, fenced code blocks, tables, comments, procedural instructions, and ordinary prose. These blocks are then merged into chunks in a way that preserves the local structure of the document. This behavior is particularly important for WAF references because rule syntax is often only meaningful when the heading, the configuration explanation, and the example rule appear together in the same chunk. Similarly, procedural documents frequently describe a defensive step immediately before or after the corresponding rule expression, and separating these elements too aggressively would weaken retrieval usefulness.

The chunking policy is also adaptive rather than fixed. Large ModSecurity rule-authoring manuals are assigned larger chunk windows, reaching approximately 3600 characters in the most rule-dense cases, while other rule references generally use windows in the range of about 2200 to 2600 characters. General attack documents use smaller windows of around 1200 characters. Overlap is kept moderate, typically between 150 and 180 characters, so that neighboring semantic context is preserved without duplicating excessive rule content. Oversized rule blocks are further split on rule boundaries, such as `SecRule`, `SecAction`, `MainRule`, or `BasicRule`, so that individual rule statements are not broken in the middle. In addition, overlap is applied only to prose-like chunks and instruction-oriented chunks, whereas code fences, tables, and rule blocks are kept isolated to avoid contaminating precise syntax with irrelevant neighboring text. For PDF materials, small adjacent page chunks may be merged when doing so preserves section continuity without collapsing independent rule examples. As a result, the final chunk set is better aligned with the internal structure of WAF documentation than a purely length-based splitter would allow.

After chunking, each chunk is embedded with *all-mpnet-base-v2* and indexed using FAISS. The completed vector store is then saved to disk for reuse across later sessions. Each chunk also preserves rich metadata, including data type, WAF type, file type, source

file, relative path, page information, block types, and a global chunk identifier within the source file. This metadata becomes important in the online phase because retrieval is not based on semantic similarity alone, but also on the structural and platform-specific characteristics of the retrieved evidence.

Phase 2: Online Retrieval and Context Augmentation The online phase is executed when the system needs grounded references for defense rule generation. In the current implementation, the attack type is expected to be passed directly by the caller and is then normalized into a canonical internal label such as XSS or SQLI. Payload-based attack-type guessing is retained only as a fallback for backward compatibility. This is an important implementation detail because it makes retrieval more stable and less dependent on ad hoc inference from individual payload strings.

Once the attack type has been normalized, the service extracts high-value lexical signals from the bypassed payloads, including suspicious keywords, encoding fragments, and obfuscation markers. These signals are used together with the target WAF name to generate multiple query variants. The query generator is not generic; it produces WAF-aware queries that explicitly target rule syntax, operators, transformations, request components, and platform-specific statement structures. Additional query patterns are also added for particular WAF families. For example, AWS queries emphasize `FieldToMatch` and `TextTransformations`, Cloudflare queries emphasize ruleset expressions and request fields, ModSecurity queries emphasize `SecRule` variables and transformations, and Naxsi queries emphasize `MainRule`, `BasicRule`, and score-related syntax. This multi-query mechanism improves recall by probing the corpus from several complementary semantic angles.

For each query, the FAISS retriever returns an initial candidate set. In the current implementation, the retriever is configured with a default search depth of 12 during initialization, while the higher-level context-retrieval interface typically operates with an initial candidate budget of 16 and a final reranked set of 5 passages. The retrieved results from all query variants are then deduplicated and filtered according to WAF relevance. If a target WAF is recognized, documents from that WAF family and general attack references are retained, while unrelated rule families are removed. Before reranking, the

remaining documents are also scored heuristically according to several signals, including whether the chunk comes from the Rules category, whether its WAF type matches the target platform, whether it contains rule-like structures such as syntax blocks or procedural instructions, and whether it contains terms related to the current attack type or payload signals. Very short prose-only chunks receive a penalty because they are less likely to be sufficient for rule writing.

After this preselection stage, the service expands the candidate set with immediate neighboring chunks from the same source file. This neighbor expansion is especially useful for WAF documentation because the most relevant rule example may reside in one chunk while the associated explanation, heading, or continuation appears in an adjacent chunk. By explicitly consulting the FAISS docstore and adding neighboring chunk identifiers from the same file, the service preserves section continuity without having to retrieve those chunks independently by similarity search alone.

The resulting preselected passages are then reranked by the cross-encoder model. The highest-ranked documents are assembled into a structured context block in which each reference includes source information, data type, WAF type, chunk index, page range, block types, and the corresponding reranking score. This context is then inserted into the prompt under a dedicated KNOWLEDGE BASE REFERENCES FOR RULE GENERATION section. In the final prompt, the language model is explicitly instructed to prioritize WAF-specific syntax, request components, transformations, and action logic, and to explain which retrieved excerpts influenced each proposed rule. The system also asks for expected false positives and tuning notes so that the generated rules remain practically useful rather than purely descriptive.

Overall, the implemented RAG pipeline is designed to generate defense rules that are technically grounded, structurally aligned with WAF documentation, and closely matched to the observed bypass behavior. By combining structure-aware chunking, metadata-aware filtering, multi-query retrieval, neighbor expansion, cross-encoder reranking, and prompt augmentation, the current implementation provides a retrieval layer that is substantially better suited to WAF rule-generation tasks than a generic document-retrieval pipeline.

Chapter 5

Implementation

5.1 Data Preparation

Data preparation plays a critical role in the implementation of the proposed LLMShield system, as the quality and structure of the input data directly influence both payload generation from the red-team perspective and defensive rule synthesis from the blue-team perspective. This section describes the data ingestion sources and the processing steps applied to construct a unified and task-specific dataset suitable for LLM-assisted WAF evaluation.

5.1.1 Data Ingestion

The collected datasets are categorized according to their intended role in the system workflow. Red-team data is used to generate and mutate attack payloads, whereas blue-team data is used to guide defensive rule generation and WAF rule alignment. Table 5.1 summarizes all ingested data sources, their roles, and the rationale for their selection.

Table 5.1: Summary of Ingested Datasets

Data Source	Role	Rationale
Kaggle: SQLi & XSS Dataset	Red	Labeled attack samples suitable for supervised payload modeling
PayloadBox XSS Payload List	Red	Diverse real-world XSS payload patterns and encoding styles
PayloadBox SQL Injection Payload List	Red	Comprehensive SQL injection payload variations across multiple DBMS families
Kaggle: XSS Dataset for Deep Learning	Red	Structured dataset suitable for learning-based attack representation
Web Application Payloads Dataset	Red	Mixed web attack payloads reflecting practical exploitation attempts
HuggingFace Web Attacks Dataset	Red	Large-scale web attack corpus with broad semantic diversity

For the blue side, the system relies on a documentation-oriented knowledge base rather than static blue-team training data. This collection is indexed for RAG and is used to provide syntax-aware and platform-specific guidance for rule generation. The documents cover general WAF concepts, rule syntax, operators, expressions, actions, fields, functions, example rules, and platform-specific configuration guidance across AWS WAF, Cloudflare, OWASP CRS, and Naxsi.

Table 5.2 summarizes the blue-side documentation sources used for retrieval.

Table 5.2: Blue-Side Documentation Sources for RAG

No.	Document Title	Purpose
1	AWS WAF Developer Guide	General concepts, architecture, and rule modeling in AWS WAF
2	Prevent SQLi and XSS with a Custom AWS WAF Rule	Practical examples of defensive rule construction for SQLi and XSS

No.	Document Title	Purpose
3	Create a Custom Rule from the Dashboard	Cloudflare dashboard-based custom rule configuration
4	Rules Language: Expressions	Expression syntax for Cloudflare rules
5	Rules Language: Edit Expressions	Expression editing and composition guidance
6	Rules Language: Operators	Supported operators for rule conditions
7	Rules Language: Values	Value types and matching semantics in rule definitions
8	Rules Language: Actions	Available actions that can be triggered by a rule
9	Rules Language: Fields	Overview of fields available in the rules language
10	Fields Reference	Detailed reference for individual Cloudflare rule fields
11	Rules Language: Functions	Built-in functions usable in rule expressions
12	Creating Rules	General guidance for designing WAF rules in OWASP CRS style
13	OWASP CRS: SQL Injection Detection Ruleset	Real SQL injection detection rules from OWASP CRS
14	OWASP CRS: XSS Detection Ruleset	Real XSS detection rules from OWASP CRS
15	LoadMaster WAF Rule Writing Guide	General tutorial on rule-writing syntax and configuration logic
16	Naxsi Rules	Rule syntax and rule structure in Naxsi
17	Basic Configuration	Basic Naxsi configuration and directive usage
18	Internal Rules	Internal rule behavior and processing in Naxsi

Selection of LLMs for Data Generation. In order to generate diverse and semantically meaningful attack payload candidates, the LLMShield system employs multiple Large Language Models with complementary strengths. The model selection strategy prioritizes reasoning capability, responsiveness to security-oriented prompts, and practical deployment cost. In particular, DeepSeek V2.5 is adopted as an open-weight large language model with strong reasoning performance, making it especially effective for multi-step payload construction and obfuscation-oriented reasoning. Its relatively low inference cost also makes it suitable for large-scale candidate generation. In parallel, Google Gemini 2.0 Flash is incorporated as a high-throughput commercial model optimized for fast inference and long-context processing. Its ability to maintain strong performance in web security scenarios makes it useful for diversifying payload styles, encoding strategies, and linguistic formulations. By combining models with different architectural and training characteristics, the data generation process reduces dependence on the generation bias of any single model, thereby improving payload diversity and robustness for subsequent WAF evaluation. After generation, the data proceeds through a WAF execution phase to collect observation signals before undergoing final processing.

All red-team datasets are aggregated and transformed into a unified intermediate representation to support payload generation and mutation tasks. The final processed dataset used in this thesis is publicly available at:

https://github.com/dung-h/LLM4WAF/blob/thesis_report/data/processed/phase1_balanced_10k.jsonl

This dataset serves as the primary input for training and prompting the LLM responsible for attack payload generation.

5.1.2 Data Processing

To align dataset construction with the execution-aware learning objectives of LLMShield, the data processing pipeline is organized into two sequential phases. Phase 1 establishes a structurally reliable supervised fine-tuning baseline, whereas Phase 2 incorporates environmental observations to improve payload effectiveness

under real WAF constraints.

Phase 1: Baseline SFT Dataset Construction. Objectives

Phase 1 is designed to equip the model with the foundational capabilities required for reliable payload generation. More specifically, this phase aims to ensure that the model can correctly follow conversational roles within the prompt–response structure, produce outputs that conform to a stable and predefined dataset schema, and reduce the number of invalid samples caused by formatting errors, empty outputs, or off-topic responses. Rather than emphasizing bypass performance at this stage, Phase 1 prioritizes structural correctness and output consistency so that subsequent phases can build upon a dependable generative baseline.

Seed Dataset Design and Class Balancing

The seed dataset for Phase 1 is constructed with explicit balancing and diversification objectives. Samples are balanced across the two major vulnerability categories considered in this study, namely SQL Injection (SQLi) and Cross-Site Scripting (XSS), while simultaneously varying HTTP attack surfaces such as query parameters, request bodies, and headers. Additional constraints are imposed, including payload length limits and mandatory structural fields, in order to prevent the model from overfitting to a fixed template. This design choice mitigates shortcut learning behavior and improves the model’s ability to generalize when the generation context or operational constraints change.

Data Quality Control and Cleaning Loop

A multi-stage cleaning loop is applied to ensure the quality of the dataset before downstream use. In this process, each sample is required to satisfy several core conditions. First, the output must be successfully parsed, for example as a valid JSON structure. Second, all mandatory fields must be present, and no extraneous fields should appear if they may interfere with downstream parsers. Third, the generated payload content must remain consistent with the generation context and the constraints imposed during sample creation. Finally, heavily duplicated or near-identical payload templates are removed in order to preserve dataset diversity. In this phase, formatting-related errors are treated as fundamental data quality problems and are resolved completely

before the data is allowed to propagate into the later observation-driven learning stages.

Descriptive Statistics and Visualization

Following the cleaning process, Phase 1 yields an initial dataset of approximately 40,000 generated samples. From this larger pool, a balanced subset of 10,000 samples is selected for supervised fine-tuning in order to satisfy computational constraints while still preserving broad technique coverage. The resulting balanced dataset retains 500 distinct attack techniques. In terms of category distribution, Cross-Site Scripting represents approximately 52% of the samples, while SQL Injection accounts for approximately 48%. In addition, payload length statistics show a mean length of approximately 110 characters and a median length of 80 characters. These statistical properties are subsequently used to inform the selection of generation length limits and stopping criteria in later training stages.

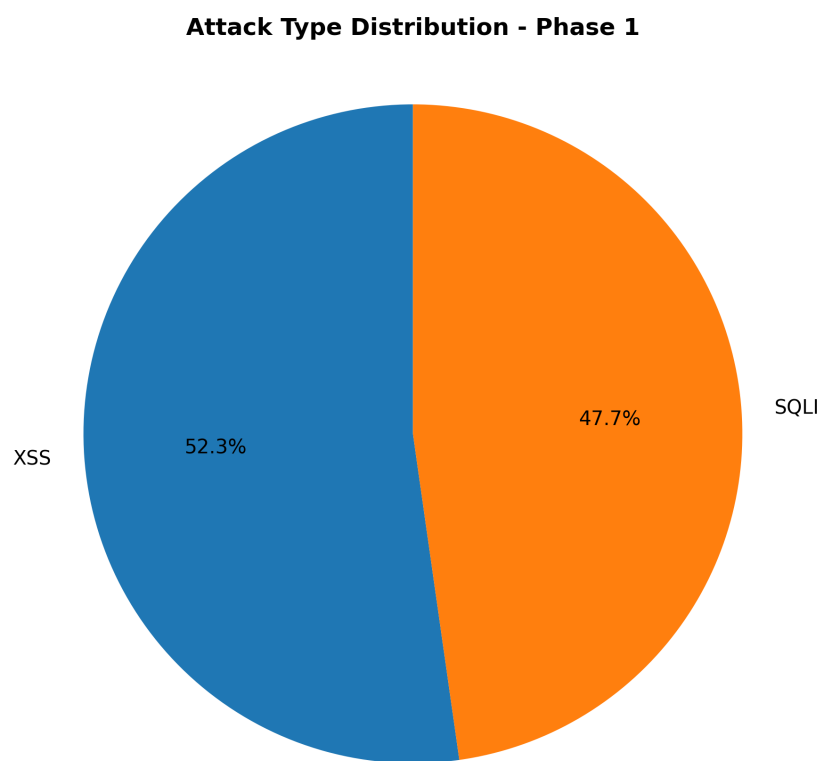


Figure 5.1: Distribution of Attack Types in Phase 1

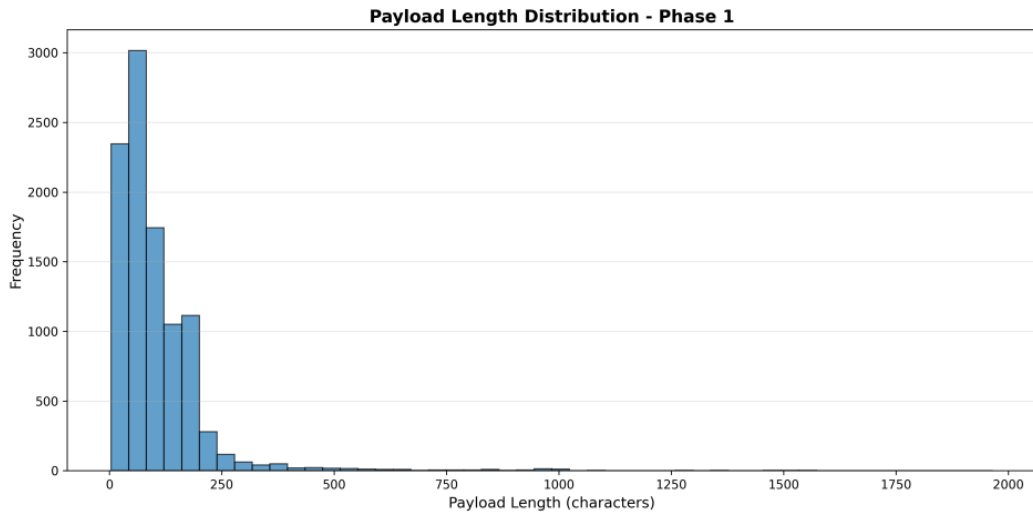


Figure 5.2: Payload Length Distribution in Phase 1



Figure 5.3: Histogram of Attack Techniques in Phase 1

Phase 2: Observation-Augmented Dataset Construction. Objectives

Phase 2 introduces explicit environmental feedback into the training data, allowing the model to learn how payload structure influences execution outcomes under a real Web Application Firewall configuration. If Phase 1 primarily addresses the question of whether the model can generate structurally valid outputs, then Phase 2 extends this objective by examining whether the model can learn interaction patterns between generated payloads and the execution environment.

Observation Collection and Normalization

During WAF evaluation, each payload execution produces a set of observable signals recorded by the evaluator. At a minimum, these observations include the allow-or-block decision inferred from the WAF response or HTTP status code, the HTTP response status code together with stable execution metadata, and a valid-or-invalid label indicating whether the generated output satisfies schema and execution requirements. These signals are then normalized into a unified observation field so that they can be reused consistently during both training and analytical stages.

Observation-Guided Data Generation Workflow

Phase 2 follows a workflow that is structurally similar to that of Phase 1, but differs in its explicit integration of execution observations into the generation prompt. For each attack technique, the system first retrieves a small set of previously observed payloads, typically consisting of two to four blocked payloads and two to four passed payloads from the execution results of Phase 1. In cases where a newly introduced technique does not yet have passed examples, only blocked samples are used. The prompt templates are then augmented with this observation section so that the model can learn from patterns associated with successful execution while avoiding those correlated with blocked behavior. Newly generated payloads are subsequently executed against ModSecurity with the OWASP Core Rule Set, after which only the payloads that successfully pass the WAF are retained, in accordance with the same filtering policy established in Phase 1. Finally, the accepted samples are combined with replay samples from Phase 1 to form the new dataset. The replay mechanism is incorporated specifically to mitigate catastrophic forgetting and preserve previously learned structural knowledge. As a result of this observation-aware generation process, the pass rate improves to approximately 52%, compared with 38% in Phase 1.

Descriptive Statistics and Visualization

Phase 2 produces a total of 24,000 training samples, including 20,000 newly generated observation-driven samples and 4,000 replay samples inherited from Phase 1. The dataset covers 550 distinct attack techniques, of which 30 are newly discovered techniques not present in the earlier phase. The class distribution in this stage remains close to balanced, with Cross-Site Scripting representing approximately 52% of the

samples and SQL Injection accounting for approximately 48%. Taken together, these results indicate that Phase 2 significantly improves model readiness for reinforcement learning by achieving a valid output rate exceeding 95%, increasing technique coverage by approximately 10%, and stabilizing the model’s interaction patterns with the execution environment.

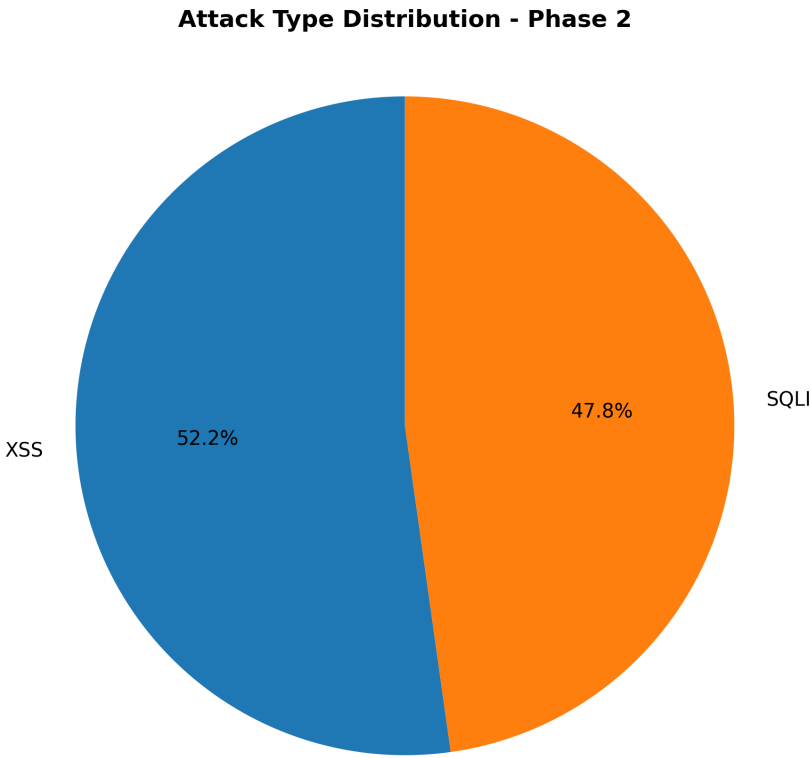


Figure 5.4: Distribution of Attack Types in Phase 2

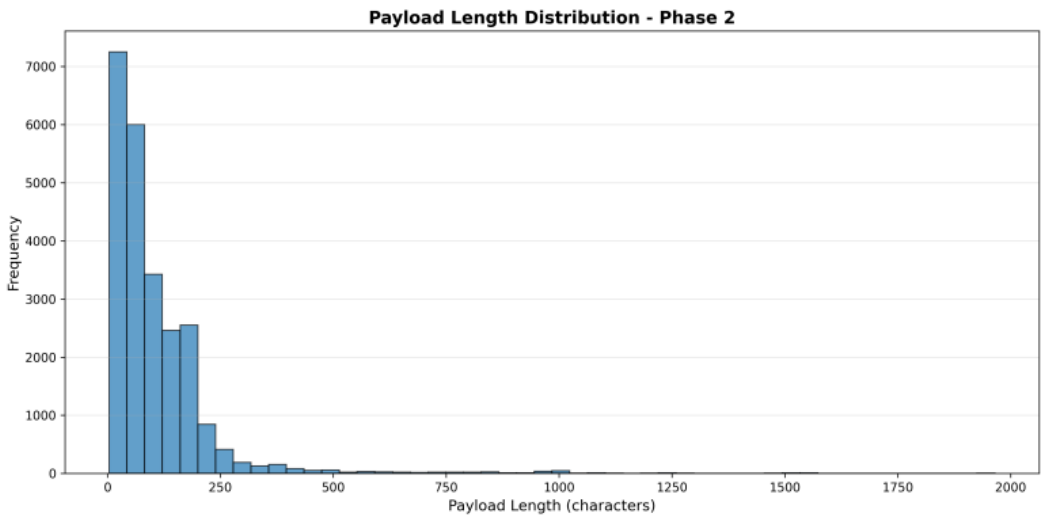


Figure 5.5: Payload Length Distribution in Phase 2

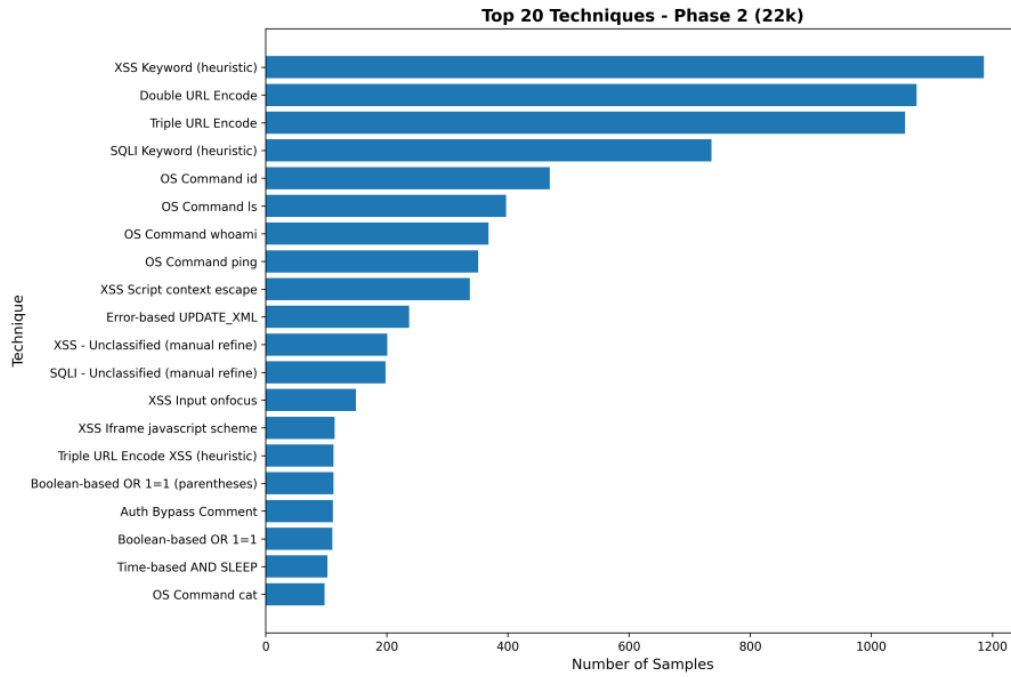


Figure 5.6: Distribution of Attack Techniques in Phase 2

5.2 Application Implementation

5.2.1 Technology Stack

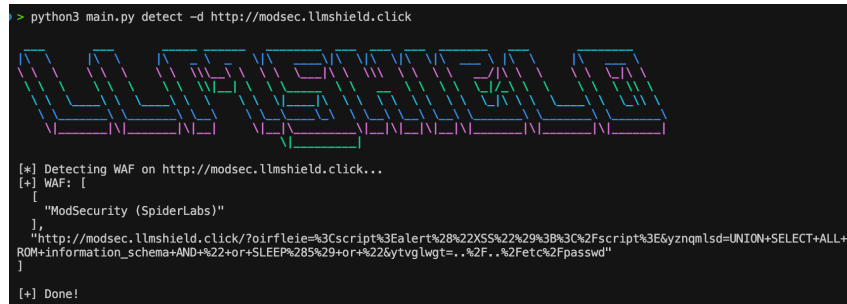
Backend. The server side is implemented in Python to orchestrate WAF detection, controlled security testing, and rule generation. A RESTful API exposes three core endpoints: `/api/detect`, `/api/attack`, and `/api/defend`. Each endpoint is designed to be stateless at the HTTP layer, while maintaining short-lived execution context in memory (e.g., attempt history within a single attack session) to support iterative workflows.

Frontend. The client side is implemented in React to provide a simple workflow-driven interface: (1) detect WAF on a target domain, (2) run controlled attack simulations in a test environment, and (3) generate defense rules from bypassed cases. The UI focuses on transparency by showing inputs, intermediate statuses, and final outputs (payload results and recommended rules) with clear highlighting of bypassed cases.

5.2.2 User Interface Overview

The UI is organized into three sequential views that match the backend modules:

- **WAF Detection View:** a domain input field and a results panel showing detected WAF attributes (e.g., product name and optional version/fingerprint confidence). This view is used to populate the WAF context for later steps.



```
> python3 main.py detect -d http://modsec.llmshield.click

[*] Detecting WAF on http://modsec.llmshield.click...
[+] WAF: [
  {
    "ModSecurity (SpiderLabs)"
  },
  {
    "http://modsec.llmshield.click/?oairfleie=%3Cscript%3Ealert%28%22XSS%22%29%3B%3C%2Fscript%3E&yznqmsd=UNION+SELECT+ALL+FROM+information_schema+AND+%22+or+SLEEP%285%29+or+%22&yvtglwgt=.%2F.%2Fetc%2Fpasswd"
  }
]
[+] Done!
```

Figure 5.7: WAF Detection Interface Showing Detected WAF Information

- **Attack Simulation View:** attack type selector (e.g., SQLi/XSS), execution controls, and a results table showing attempt-by-attempt outcomes. Bypassed payloads are visually emphasized to support quick review and downstream rule generation.

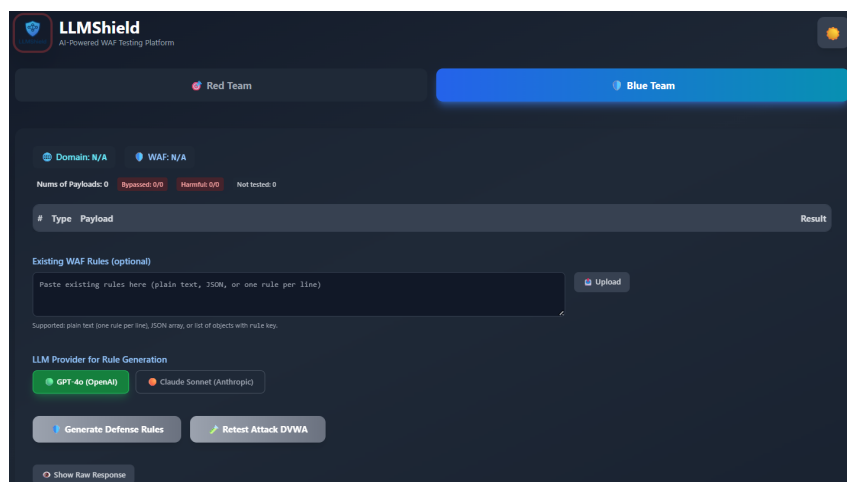


Figure 5.8: Attack Simulation Interface Showing Payload Execution Results

- **Defense Generation View:** a review panel listing bypassed payloads and a single action to generate defensive recommendations. The output area presents proposed WAF rules and implementation instructions in structured, copy-friendly blocks.

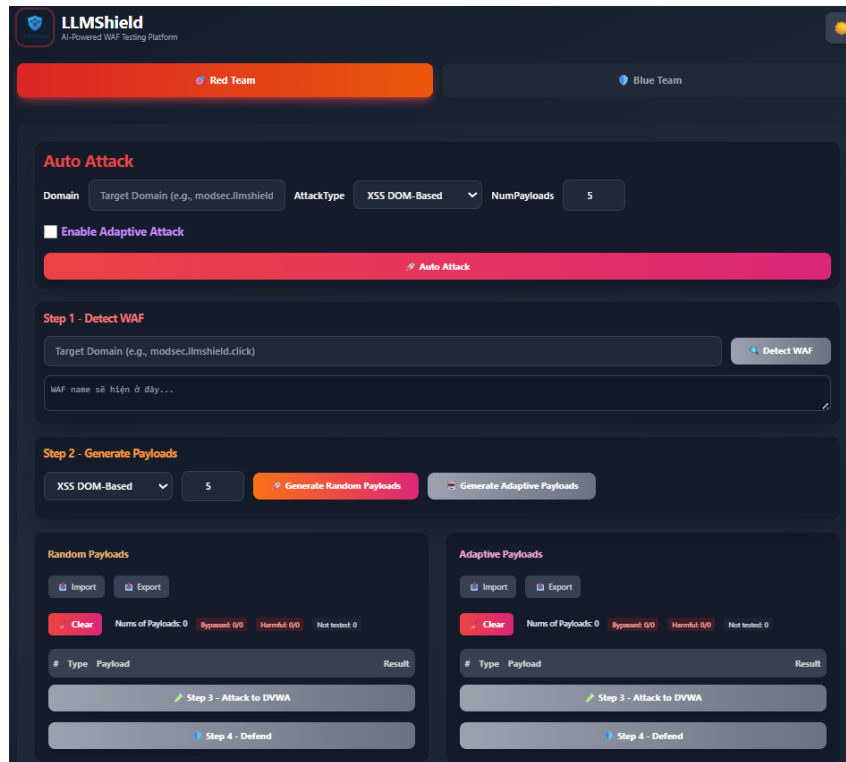


Figure 5.9: Defense Generation Interface with Automatically Generated WAF Rules

From an implementation perspective, the React layer uses reusable components (forms, status badges, tables, and code blocks) to render: (i) request parameters, (ii) execution progress (loading indicators per operation), and (iii) structured outputs (JSON-like objects for developer usability). The frontend does not embed security logic; it only collects inputs, calls APIs, and renders responses to keep responsibilities clean and auditable.

5.2.3 Repository Structure

As illustrated in Figure 5.10, the implementation is intentionally divided into two repositories with different responsibilities. The first repository, LLM4WAF, serves as the application and orchestration layer of the system. It contains the user-facing toolchain, including the command-line workflow and the web-based interface, and acts as the integration point between end users, external model APIs, and the internal defense pipeline. In practice, this repository is responsible for handling the input prompt, selecting the execution mode, coordinating attack generation and defense-rule generation, validating the produced rules, and presenting the results to the user through

either the CLI or the GUI.

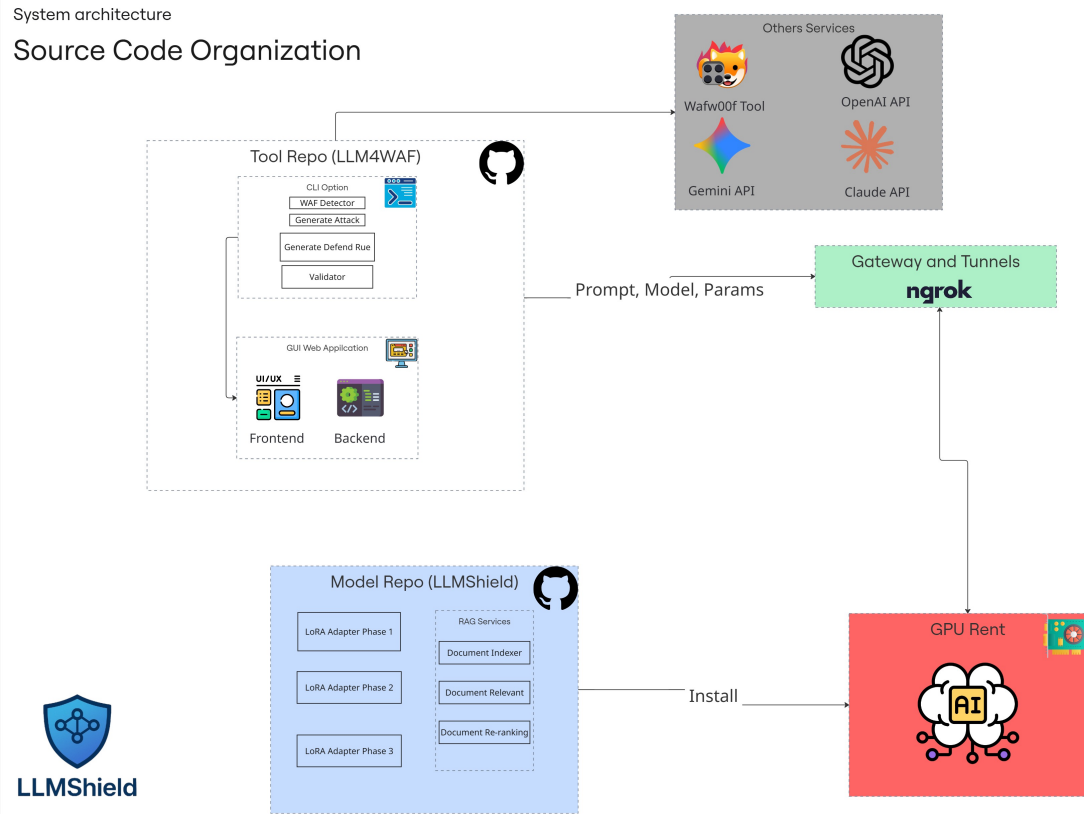


Figure 5.10: Source code organization of the proposed system.

The second repository, LLMShield, serves as the model and knowledge layer. It contains the assets required to build and operate the specialized defense model, including model-version management, LoRA-based adaptation stages, retrieval-augmented generation components, and the supporting knowledge base used for document indexing, retrieval, and reranking. In other words, while LLM4WAF focuses on system operation and interaction, LLMShield provides the learned intelligence and retrieval capabilities that support WAF rule generation and refinement.

A key architectural aspect is that these two repositories are not only separated logically, but also deployed differently. The LLMShield repository is designed to be installed and executed on rented GPU infrastructure, where the computationally expensive model loading, inference, and retrieval services can run efficiently. The LLM4WAF repository, by contrast, remains the lightweight control layer that sends prompts, model selections, and runtime parameters to the remote inference environment. To connect these layers in a practical and flexible way, the system uses ngrok as a

tunneling gateway. This tunnel exposes the services running on the rented GPU to the application layer, allowing the local or externally hosted LLM4WAF tool to communicate with the remotely deployed LLMShield backend without requiring a fixed public deployment of the GPU machine.

This repository split provides several benefits. First, it separates application logic from model engineering, which makes the system easier to maintain and evolve. Second, it allows the tool layer to remain portable while the heavy model layer is deployed only where GPU resources are available. Third, it supports hybrid operation: the application layer can call external services such as general-purpose LLM APIs when needed, while also routing requests to the project’s own specialized model stack through the GPU-backed tunnel. Therefore, the repository structure reflects not only code organization, but also the actual deployment and interaction pattern of the proposed system.

5.2.4 Main Functionality

Module 1: WAF Detection. This module identifies whether the target domain is protected by a WAF and extracts basic fingerprint information. Figure 5.11 summarizes the interaction between the user, frontend, backend, the WAF detector, and the target website.

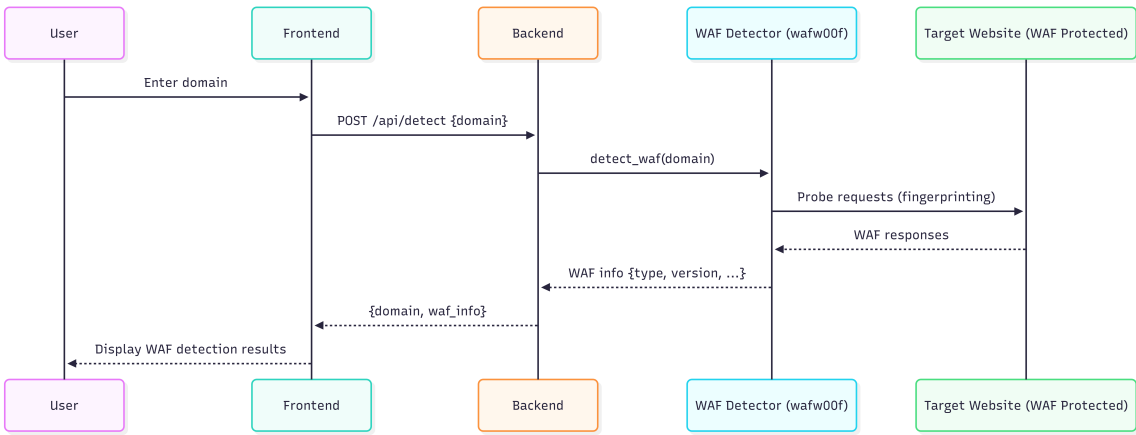


Figure 5.11: Sequence diagram for WAF detection (/api/detect).

Request flow.

1. The user enters a domain on the frontend.

2. The frontend sends `POST /api/detect` with `{domain}`.
3. The backend calls `detect_waf(domain)` via a detector component (e.g., `wafw00f`).
4. The detector issues probe requests (fingerprinting) to the target website and analyzes the responses.
5. The backend returns `{domain, waf_info}` to the frontend.

Key implementation details.

- **Validation:** the backend validates the domain format and enforces basic constraints (scheme normalization, allowed ports if applicable) before running detection.
- **Timeouts and stability:** fingerprint probes use conservative timeouts and bounded retries to avoid hanging requests and to ensure predictable UI responsiveness.
- **Output schema:** `waf_info` is normalized into a consistent object (e.g., `name`, `version` if available, `confidence/evidence`) so subsequent modules can consume it reliably.

Module 2: Attack Payload Generation. This module is responsible for generating candidate attack payloads that are later used in the testing stage. The generation workflow is implemented through the `/api/generate_payload` endpoint, which supports both initial random payload generation and adaptive payload generation based on previous attack feedback. Figure 5.12 presents the sequence of interactions for this module.

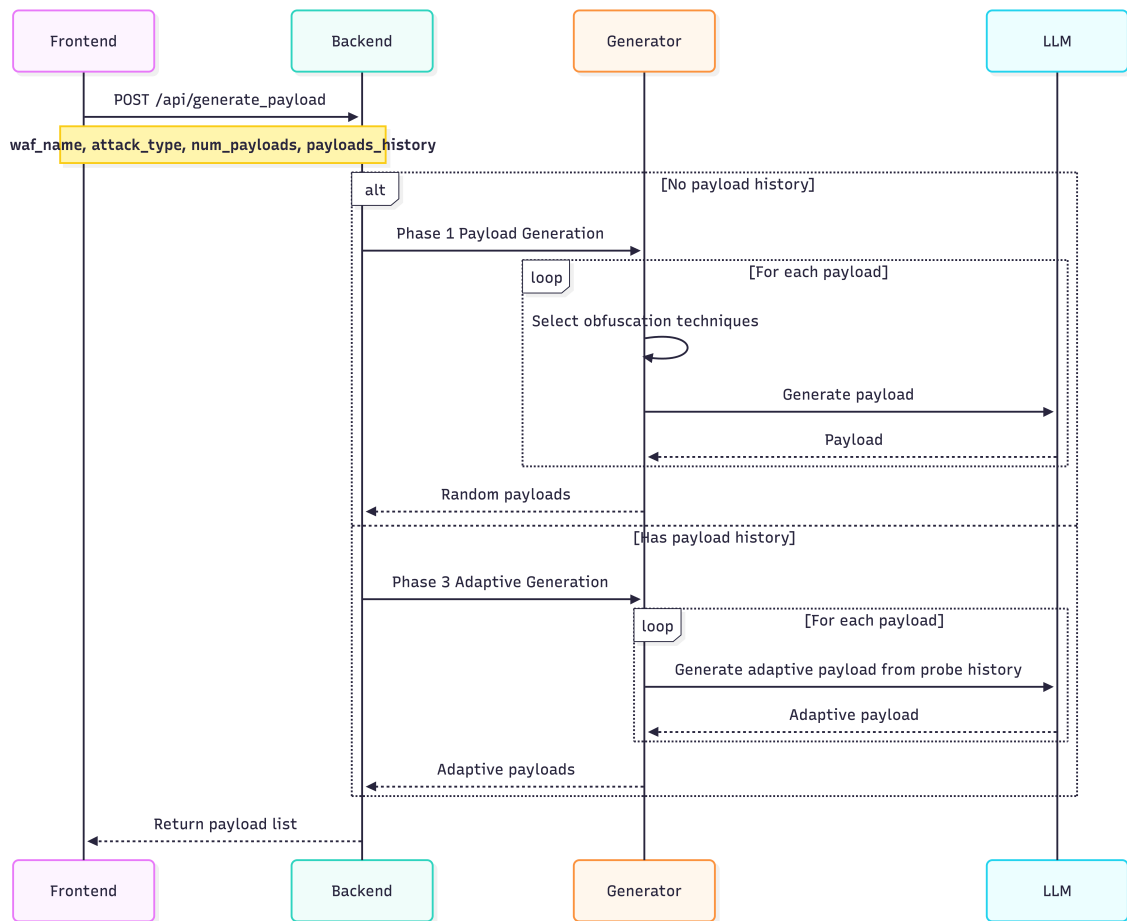


Figure 5.12: Sequence diagram for payload generation (`/api/generate_payload`).

Request flow.

- The user provides the target WAF context, selects an attack type, and specifies the number of payloads to generate through the frontend interface.
- The frontend sends a POST request to `/api/generate_payload`. The implemented request includes the detected `waf_name`, the selected `attack_type`, the requested `num_payloads`, and an optional `payloads_history` field:

`{waf_name, attack_type, num_payloads, payloads_history}`.

- After receiving the request, the backend determines which generation strategy should be used. If no previous payload history is provided, the system starts the initial generation stage. Otherwise, it switches to the adaptive generation stage.
- In the initial stage, the generator produces Phase 1 payloads. For each payload, the system selects one or more obfuscation or evasion techniques associated with

the chosen attack type and then invokes the generation model to create a candidate payload.

- In the adaptive stage, the backend uses the previous payload history as probe history. The model then generates new payloads that are conditioned on prior blocked and bypassed outcomes, allowing the generation process to adapt to the observed WAF behavior.
- Once generation is complete, the backend returns a structured list of payload objects to the frontend. Each payload contains the generated content together with metadata such as attack type and generation technique.
- The frontend displays the resulting payloads for later execution, comparison, export, or reuse in the attack-testing phase.

Module 3: Automated Attack Testing with Benchmark Environment. This module evaluates the generated payloads by executing them against the benchmark environment and recording their outcomes. In the current implementation, the benchmark target is DVWA, which is used to simulate vulnerable application behavior under WAF protection. The evaluation workflow is implemented through the `/api/test_attack` endpoint. Figure 5.13 shows the sequence of this process.

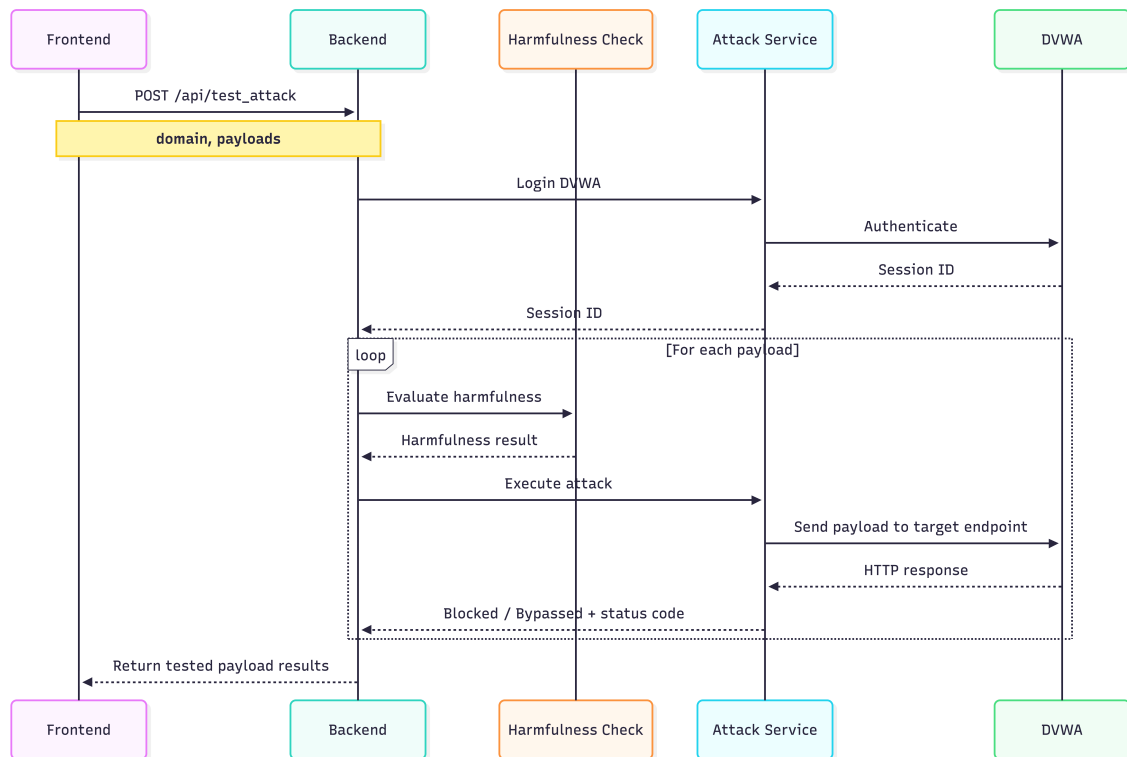


Figure 5.13: Sequence diagram for attack testing (/api/test_attack).

Request flow.

- After payload generation, the user initiates the attack-testing step from the frontend.
- The frontend sends a POST request to /api/test_attack with the target domain and the list of generated payloads:

{domain, payloads}.

- The backend first authenticates to the DVWA environment in order to obtain a valid session for subsequent attack requests.
- The backend then processes the payload list iteratively. Before execution, each payload is evaluated by the harmfulness-checking stage, where XSS and SQL injection inputs are inspected to determine whether they contain meaningful harmful content for testing.
- After the harmfulness check, the corresponding attack routine is selected according to the payload's attack type. The backend sends the payload to the appropriate DVWA endpoint and collects the HTTP response.

- Based on the returned response, the system determines whether the payload was blocked or bypassed by the WAF. The result is recorded together with the HTTP status code and the harmfulness indicator.
- After all payloads have been tested, the backend returns the updated payload list to the frontend. Each payload now contains its testing outcome, allowing the user to identify which payloads bypassed the protection layer and should be forwarded to the defense-generation stage.

Module 3: Defense Rule Generation from Bypassed Cases. This module transforms bypassed attack cases into deployable defensive rules through a structured blue-team pipeline. Rather than directly returning a single raw LLM response, the backend performs multi-stage processing, including payload clustering, retrieval-augmented context enrichment, candidate rule generation, syntax validation, rule refinement, deployment support, and post-deployment retesting. Figure 5.14 illustrates the sequence of interactions for the `/api/defend` workflow.

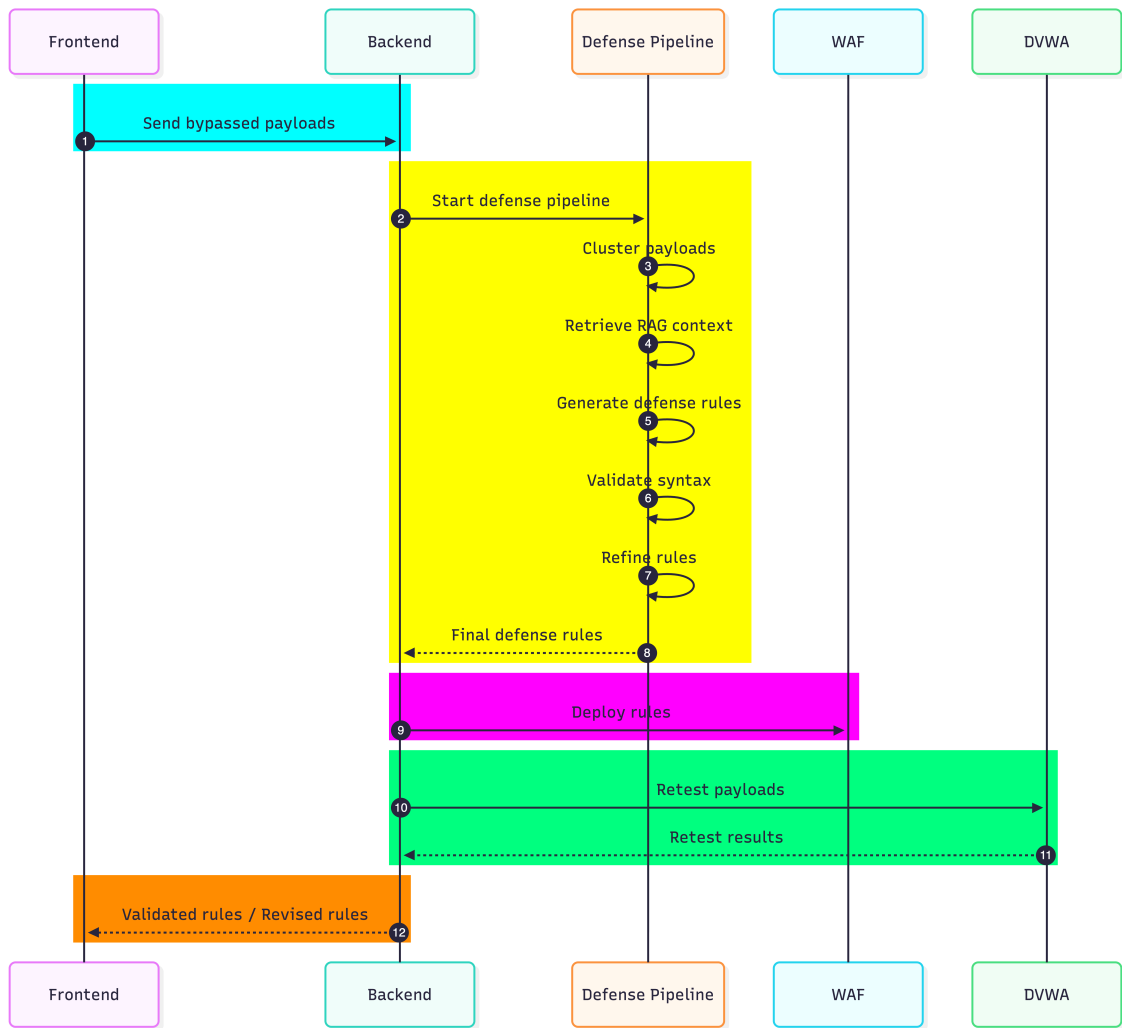


Figure 5.14: Sequence diagram for defense rule generation (/api/defend).

Request flow.

- The user selects the payloads that successfully bypassed the protection layer and triggers the defense generation process from the interface.
- The frontend sends a POST request to /api/defend. In the implemented backend, the request contains the WAF identifier, the payload list, the attack type, and optional defense settings such as pre-existing rules and the selected LLM provider. A representative request structure is:

`{waf_name, payloads, attack_type, existing_rules, llm_provider}`.

- After receiving the request, the backend filters the input to retain the payloads that are both bypassed and harmful. These payloads are then passed to the defense

pipeline as the main evidence for defensive analysis.

- The defense pipeline first groups similar payloads through a clustering stage in order to reduce redundancy and identify shared attack patterns.
- Next, the pipeline enriches the generation process with Retrieval-Augmented Generation (RAG), supplying contextual defensive knowledge and relevant references before rule synthesis.
- Using the clustered payloads and retrieved context, the rule generation stage produces an initial set of candidate WAF rules together with implementation-oriented guidance.
- The generated rules are then checked through syntax validation to ensure compatibility with the target WAF format. After validation, an additional refinement stage is applied to improve rule quality, reduce overlap, and strengthen deployability.
- The backend returns a structured response rather than a single raw model output. The response includes intermediate and final artifacts such as clustered payloads, retrieved RAG sources, generated candidate rules, finalized defense rules, and pipeline statistics:

```
{clustered_payloads, rag_sources, generated_rules, final_rules, stats}.
```

- The frontend presents the resulting rules in a structured format for inspection and deployment. After deployment to the WAF, the system can retest the previously bypassed payloads against DVWA to determine whether the new rules have been correctly applied and are effective. If the retest fails, the rules enter a revision loop for further improvement.

5.3 Fine Tuning Implementation

5.3.1 Model Choice and Rationale

In this work, the base model is **Gemma 2B**¹, a lightweight decoder-only causal language model that is suitable for instruction-style generation while remaining feasible to fine-tune under limited GPU resources. We chose Gemma 2B for three practical reasons:

- **Compute efficiency:** 2B parameters allows faster iteration and lower memory usage compared to larger LLMs.
- **Compatibility with PEFT:** Gemma 2B works well with **LoRA** adapters, enabling effective adaptation without full fine-tuning of all weights.
- **Stable chat formatting:** the training pipeline formats samples into a chat-like messages structure, which is well aligned with instruction-following behaviors.

5.3.2 Technologies, Libraries, and Training Setup

Table below summarizes the main technologies and libraries used for the fine-tuning pipeline.

¹Google. (n.d.). Gemma 2B: Lightweight open-weight large language model. Retrieved from <https://huggingface.co/google/gemma-2b>

Table 5.3: Fine-tuning technology stack and its role in the implementation.

Component	Technology / Library	Purpose in Implementation
Programming language	Python	Implement dataset processing, training scripts, and RL loop.
Frontend integration	React (UI layer)	Used for GUI interface frontend of the application workflow.
Model + tokenizer	transformers	Load Gemma 2B with <code>AutoModelForCausalLM/AutoTokenizer</code> ; run generation/training.
Dataset IO	datasets	Load and preprocess JSON/JSONL datasets; map to chat-format samples.
Parameter-efficient FT	peft (LoRA)	Train low-rank adapters instead of full model weights to reduce compute/memory.
Quantization	bitsandbytes (4-bit)	Reduce GPU memory footprint (QLoRA-style setup) while keeping training practical.
SFT training loop	trl (SFTTrainer)	Supervised fine-tuning trainer for instruction/chat data with standard LM loss.
Optimization	PyTorch / <code>torch.optim</code>	Core training backend; optimizer, gradient accumulation, checkpointing.
Experiment logging	TensorBoard (via HF / <code>torch</code>)	Track loss curves, reward trends (RL), and training metadata.
Config management	YAML configuration	Centralize hyperparameters (model name, data paths, batch size, lr, epochs, etc.).

5.3.3 Three-Phase Training Pipeline

The fine-tuning workflow is organized into three phases, matching the implemented scripts and datasets.

Phase 1: RED dataset enrichment. We generate an expanded RED dataset by applying controlled transformations (e.g., encoding variants, casing/spacing changes, lightweight obfuscation patterns) to increase diversity across techniques and reduce

overfitting to a small set of payload styles. The output remains label-consistent and is saved as a clean training corpus for later phases.

Phase 2: Context-aware dataset construction. We build a second-stage dataset that explicitly includes *history context*: blocked attempts (history) are paired with successful/bypassed samples and task metadata (attack type, injection point, technique). Samples are serialized into chat-style messages to align with instruction-following training and downstream usage.

Phase 3: Fine-tuning + RL refinement.

- **Supervised Fine-Tuning (SFT):** Gemma 2B is fine-tuned using LoRA (and optional 4-bit quantization) with standard LM loss on the constructed chat dataset.
- **Reinforcement Learning (REINFORCE):** we further refine the policy using an environment-driven reward signal (via a custom WAF evaluation environment). Rewards are stabilized with a baseline and training is checkpointed periodically.

This implementation keeps the pipeline modular: Phase 1/2 produce datasets, while Phase 3 consumes them for SFT and optional RL refinement.

5.4 Defense Pipeline Implementation

The defense pipeline in *LLMShield* is implemented as a structured blue-team workflow that transforms successful WAF bypass evidence into defensive rules and deployment guidance. As shown in Figure 5.15, the pipeline starts from a preparation stage where bypassed payloads, attack types, firewall types, and optional existing rules are collected and normalized. Instead of directly asking an LLM to generate rules from raw payloads, the system processes the input through several stages: payload clustering, retrieval-augmented prompt construction, LLM-based rule generation, syntax validation, and final rule refinement.

Workflow architecture Defense Workflow

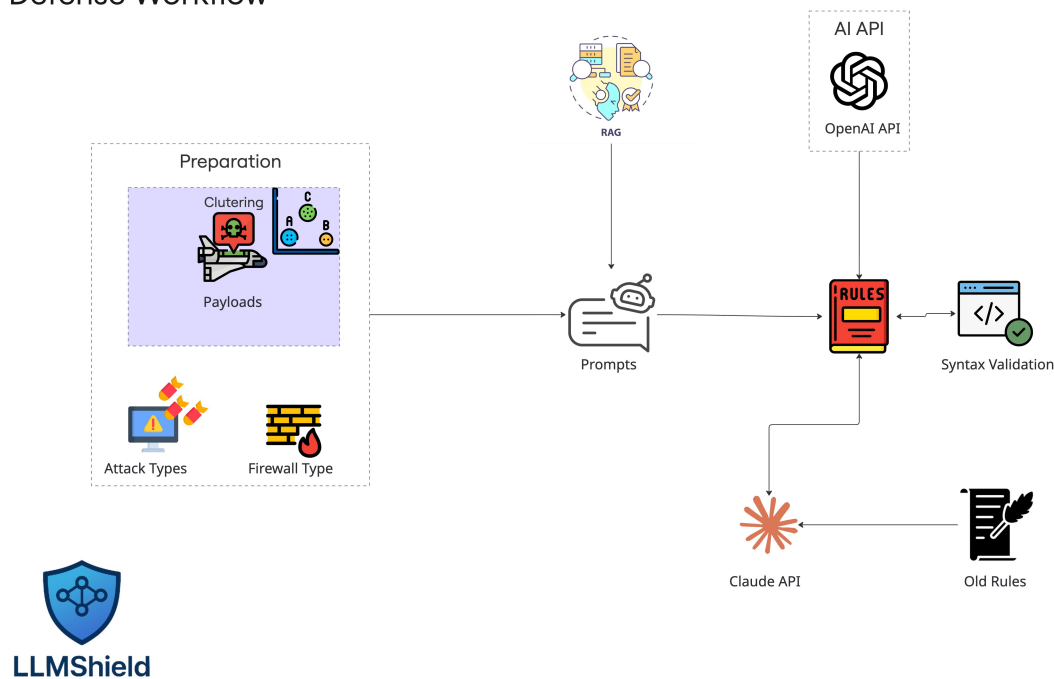


Figure 5.15: LLMShield defense workflow

The first stage of the implementation focuses on preparing the bypass evidence. The system extracts only payloads that successfully bypassed the target WAF, because these payloads represent concrete failure cases of the current defensive configuration. Each payload is stored together with its attack type, technique, status code, and bypass result. The target WAF is also mapped to a specific firewall family such as ModSecurity, NAXSI, Cloudflare, or AWS WAF. This mapping is necessary because the same defensive idea must be expressed differently depending on the syntax and policy model of each WAF. If the user provides existing rules, they are parsed and preserved for later comparison so that the generated output does not unnecessarily duplicate already deployed protections.

After input preparation, LLMShield applies clustering to group similar bypassed payloads into common evasion patterns. This step is important because generating one rule for every single payload would easily produce narrow and redundant rules. By grouping payloads with similar structure, encoding style, or attack behavior, the system allows the defense stage to reason at the pattern level rather than at the individual-string level. In the implementation, this is suitable for web attack payloads because small

textual transformations such as URL encoding, case randomization, SQL comments, whitespace manipulation, or event-handler variations often determine whether a WAF is bypassed.

The next stage enriches the rule-generation prompt using retrieval-augmented generation. LLMShield queries an internal knowledge base containing WAF-related rule-writing documents and security references. The retrieval module uses vector search and reranking to select relevant references for the current attack type and target firewall. These references provide concrete information about rule syntax, operators, transformations, request fields, and configuration examples. This design reduces the risk of relying only on the model's general knowledge and helps the LLM generate rules that are more consistent with the target WAF's actual rule format.

Based on the clustered payloads and retrieved context, the system then uses a high-capability LLM to generate candidate defensive rules. The generation step is implemented as a structured output task rather than a free-form response. Each generated item contains both a `rule` field and an `instructions` field. The `rule` field stores the WAF rule or configuration, while the `instructions` field provides short deployment guidance for the operator. This is useful because practical WAF defense does not stop at generating a pattern; the operator also needs to understand how the rule should be configured, inserted, or adjusted in the target firewall environment.

After rule generation, the candidate rules are passed through a syntax validation stage. This stage checks whether the generated rule follows the expected syntax of the selected WAF. It acts as an implementation boundary between LLM-generated text and deployable configuration. This is especially necessary because ModSecurity, NAXSI, Cloudflare, and AWS WAF use different rule formats and configuration models. A rule may be logically meaningful but still unusable if it contains incorrect escaping, unsupported fields, invalid operators, or malformed actions. Therefore, syntax validation helps filter invalid candidates before the final refinement stage.

The final stage is rule refinement. In the current implementation, `claude-sonnet-4-6` is used as a second-pass refinement model rather than as the primary rule generator. This stage receives the validated rules, bypassed payloads, target WAF type, and optional existing rules. Its role is to improve the generated defense set by removing duplicate

rules, comparing new rules with existing ones, preserving valid syntax, and rewriting rules or instructions when necessary. This separation is useful because refinement requires broader judgment over multiple artifacts, not only initial rule synthesis. Claude Sonnet is therefore used as a stronger post-processing model for rule consolidation, deduplication, and final polishing.

Overall, the implemented defense pipeline shows that LLMShield is not a single-prompt rule generator. It is a multi-stage workflow that combines bypass evidence, clustering, RAG-based grounding, structured LLM generation, syntax validation, and model-based refinement. The LLM contributes reasoning and rule synthesis, the RAG module provides WAF-specific implementation knowledge, the validator improves syntactic correctness, and the refinement stage improves deployability and rule quality. Together, these components form a more practical pipeline for iterative WAF hardening than directly generating rules from payloads without intermediate processing.

Chapter 6

Experiments

6.1 Testing Approaches

To ensure the proper functionality and effectiveness of the fine-tuned Large Language Model (LLM) integrated into the Application, various testing methodologies are employed. These approaches aim to evaluate the performance of both the attack generation and defensive rule creation mechanisms. The testing process is structured into multiple phases, each designed to address different aspects of the system's operation.

Unit Testing

Unit testing focuses on verifying each isolated component and interface to ensure correct behavior before running end-to-end scenarios. First, domain input validation is tested to confirm that the backend correctly parses and validates domains, including format checking, scheme normalization, and invalid-input handling, while also returning consistent error messages. Next, attack type selection and option handling are examined to ensure that supported attack categories, such as SQLi, XSS, and OS_INJECTION, are correctly recognized and mapped to the intended generation and testing pipeline. The WAF detector module is then validated to confirm that the detection component returns a normalized `waf_info` object containing the expected name and fingerprint fields, while also handling timeout cases or unreachable targets gracefully.

The frontend and interface-level behavior are also evaluated. In particular, UI request and response integration testing is conducted to verify the stability of form submission, loading states, result table rendering, and bypass highlighting, thereby ensuring reliable interaction with the backend APIs. In parallel, the performance of the LLM after fine-tuning is assessed for the Red Team workflow, while the API-based LLM is evaluated for the Blue Team workflow, in order to verify that the system can generate both appropriate attack payloads and valid defensive rules. The CLI option verification stage further checks that command-line arguments and flags, including required parameters, default values, and invalid combinations, behave consistently across different configurations.

Additional unit tests are devoted to the defensive components. Standalone defense rule generation is verified by testing whether `DefensePipeline.generate_defense_rules()` can be invoked independently, without requiring a preceding red-team attack run, provided that the user supplies a list of bypassed payloads and a target WAF type. The corresponding unit test confirms that the returned `PipelineResult` contains at least one valid rule, that the `stage` field reaches `COMPLETE`, and that the pipeline handles edge cases appropriately, such as an empty payload list, which should return the `FAILED` stage, or an unsupported WAF type, which should be handled gracefully.

The payload clustering module is tested in isolation by supplying a controlled set of payloads with known similarity groups and verifying that the output labels correctly separate distinct attack patterns. More specifically, the test verifies that character-level TF-IDF vectorization produces a matrix with the expected shape, that TruncatedSVD dimensionality reduction does not raise errors when the input dimension is already below the target threshold, that both HDBSCAN and HAC return exactly one label for each input payload, and that noise points labeled as `-1` by HDBSCAN do not break the downstream `ClusterInfo` construction, which instead falls back to a single-cluster grouping strategy.

Finally, RAG retrieval and prompt enhancement are validated through controlled tests of the `enhance_defense_generation()` function by mocking the external `rag_retrieve()` call. These tests confirm that when the mock returns a non-empty `sources` list, the enhanced prompt contains the expected `[Reference #N : source]` markers while preserving the original base prompt. They also confirm

that when `sources` is empty, the function returns the base prompt unchanged; that if `rag_retrieve()` raises a network exception or returns `None`, the pipeline falls back to the base prompt without propagating the error; and that the metadata fields `num_queries`, `num_docs_all`, and `num_docs_filtered` are correctly forwarded to the `PipelineResult.rag_sources` attribute.

Use-case Testing

Use-case testing evaluates two realistic end-to-end workflows that together cover the full Red Team attack cycle and the Blue Team defense cycle, while emphasizing measurable outcomes at each stage.

Use-case 1: Red Team Attack Pipeline (Phase 1 + Phase 3). This use-case exercises the complete offensive path from WAF detection through adaptive payload generation and live testing. The process begins with WAF detection. Given a target domain, the system invokes `WAFW00F` to determine whether a WAF is present and extracts a normalized `waf_info` object containing the firewall name and fingerprint. The detected WAF name is then forwarded to the downstream payload generation stage as the primary conditioning signal.

After detection, the workflow proceeds to Phase 1, in which the Red Team LLM, namely the fine-tuned Gemma-2B model served by `LLMShield` via `adapter_name="phase1"`, receives the WAF name and attack type and generates an initial batch of obfuscated payloads. The techniques used at this stage are sampled randomly from the registered obfuscation catalogue. For `SQLi`, these may include techniques such as double URL-encoding, comment injection, and case randomization, whereas for `XSS` they may include event-handler injection and `SVG` or `IMG`-based variants. These techniques are explicitly included in the prompt as conditioning context.

The generated payloads are then subjected to live testing. Each payload is dispatched to the target application in the DVWA test environment, and the verifier determines whether the payload is bypassed or blocked based on the observed HTTP response code and WAF block-page detection. Through this process, the system builds a *probe history* that records the observed outcomes for all tested payloads.

The workflow then enters Phase 3, which performs adaptive payload refinement. At this stage, the probe history, including both bypassed and blocked payloads, is fed back into the LLM through `adapter_name="phase3_r1"`. Using this feedback, the model produces a second and more informed batch of payloads that attempts to avoid patterns already blocked by the WAF while reinforcing patterns that were previously successful. These refined payloads are tested again, and the final bypass count is recorded. The main outcome metric for this use-case is the **bypass rate**, defined as the proportion of payloads across both phases that successfully evade the WAF.

Use-case 2: Blue Team Defense Pipeline and Re-verification. This use-case takes the bypassed payloads obtained from Use-case 1 as input and evaluates the full defensive rule-generation and deployment loop. The process begins with payload clustering, in which bypassed payloads are grouped into semantically similar clusters through character-level TF-IDF vectorization, TruncatedSVD dimensionality reduction, and Hierarchical Agglomerative Clustering (HAC). Each resulting `ClusterInfo` record identifies the representative payload, attack type, and number of members in the cluster. This design allows the LLM to generate one targeted rule per attack pattern rather than one rule per individual payload.

The clustered payload summary is then passed to the Blue Team LLM, namely GPT-4o, together with knowledge-base context retrieved from the LLMShield RAG service. During this stage, the retriever performs multi-query lookup across the document corpus, including the Rules, XSS, and SQLi sub-folders, using FAISS. The retrieved results are then re-ranked with a CrossEncoder, and up to `final_k` highly relevant rule references are injected into the prompt. Based on this enriched input, the model generates structured JSON output containing both rule and instructions fields for each cluster.

After generation, syntax validation is performed for every candidate rule. Each generated rule is checked by the `SyntaxValidator` against the syntax requirements of the target WAF, whether that be ModSecurity SecRule format, Cloudflare expression syntax, AWS WAF JSON, or Naxsi MainRule. Rules that fail validation are retried up to `max_retries` times using a focused fix prompt, while rules that continue to fail are

discarded.

The set of valid rules is then passed to the `GeminiRuleAgent` for further refinement. At this stage, overlapping rules are deduplicated, the newly generated rules are compared against any pre-existing WAF ruleset, and deployment notes are added where necessary. The final rule list is returned together with `refinement_notes` and a `duplicates_removed` count.

The final step is rule deployment and re-testing. The recommended rules are applied to the test WAF environment, and all previously bypassed payloads are dispatched again to verify whether they are now blocked. The main outcome metric for this use-case is the **patch rate**, defined as the proportion of previously bypassed payloads that are no longer able to evade the WAF after rule deployment.

End-to-End Testing

End-to-end testing covers the full workflow across four WAF targets: **ModSecurity**, **Cloudflare WAF**, **AWS WAF**, and **Naxsi**. Each WAF is tested with both SQLi and XSS attack types, yielding a total of eight scenarios. The rule format and deployment method for each target are summarized in Table 6.1.

Table 6.1: End-to-end test targets

WAF	Rule format	Deployment
ModSecurity	SecRule directive	Apache/Nginx config file
Cloudflare	Expression syntax	Cloudflare dashboard / API
AWS WAF	JSON ByteMatchStatement	AWS Console / CloudFormation
Naxsi	MainRule directive	Nginx config file

For each of the eight scenarios, the same general sequence is carried out. First, the WAF detector identifies the target firewall and extracts `waf_info`, which is then used as the conditioning signal for the Red Team LLM. Next, Phase 1 generates an initial batch of obfuscated payloads, each of which is tested live against the corresponding WAF instance, and its outcome is recorded as either **bypassed** or **blocked**. The workflow then advances to Phase 3, which uses the accumulated probe history to generate a

second, refined batch of payloads that reinforces successful patterns while avoiding patterns already detected by the WAF. These refined payloads are likewise tested, and the combined bypass count is reported.

After the offensive stages are completed, the defense pipeline processes all bypassed payloads. Clustering first groups them according to attack pattern, after which the RAG-enhanced LLM generates candidate rules in the correct target syntax. The SyntaxValidator filters out malformed rules and retries them when possible, and the Gemini agent deduplicates and refines the remaining valid rules into a final deployable set. These generated rules are then deployed to the corresponding WAF instance, and all previously bypassed payloads are re-dispatched to verify whether they are now blocked.

A scenario is considered passing when the **bypass rate** after Phase 3 exceeds a baseline threshold, thereby confirming that the Red Team LLM is capable of generating effective adversarial payloads, and when the **patch rate** after rule deployment satisfies an acceptance criterion, thereby confirming that the Blue Team pipeline produces correct and deployable defensive rules for that WAF.

6.2 Benchmark Building

6.2.1 Benchmark Architecture

The benchmark architecture was designed to evaluate LLMSHield under heterogeneous WAF deployment settings, including both managed commercial services and self-hosted open-source firewalls. As shown in Figure 6.1, the testing workflow begins from the LLMSHield application, where attack payloads are generated and sent as HTTP requests through different WAF paths before reaching the vulnerable target application. The target resource is DVWA, deployed inside a Docker container, which provides reproducible vulnerabilities such as SQL injection and cross-site scripting. This controlled environment allows each payload to be evaluated consistently based on whether it is blocked by the WAF, forwarded to the backend, or successfully triggers the vulnerable behavior.

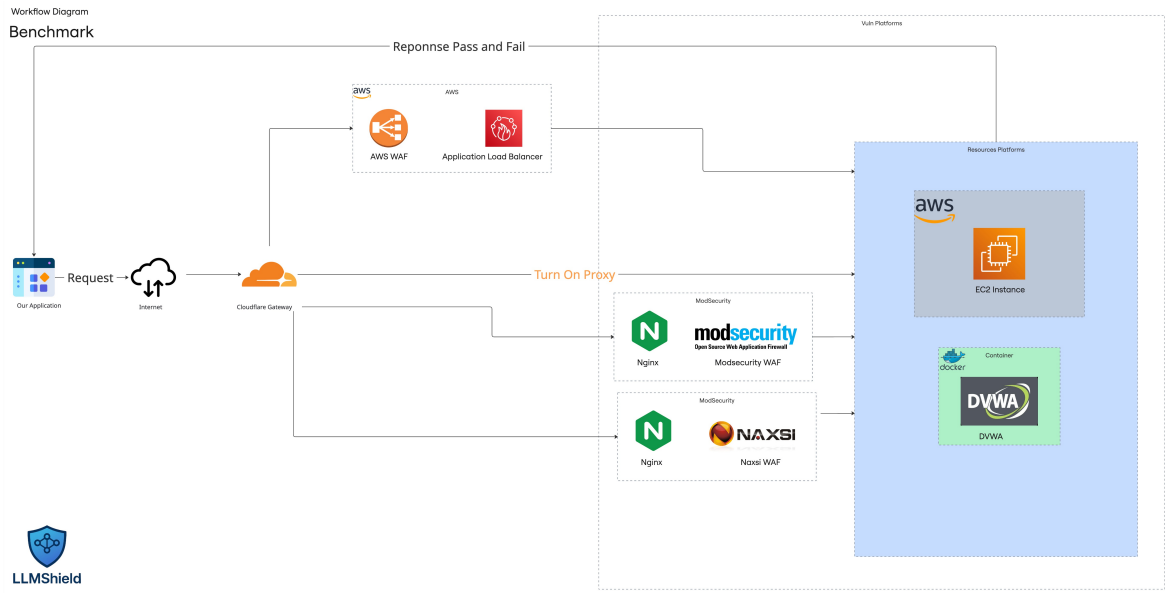


Figure 6.1: Benchmark architecture for evaluating payload bypass and WAF defense rule generation.

The benchmark includes two commercial WAF platforms, Cloudflare WAF and AWS WAF, using their free-tier or freely accessible configurations. The purpose of selecting these platforms is to expose LLMShield to realistic managed WAF environments while keeping the benchmark reproducible and cost-efficient during development. Since commercial WAFs are commonly deployed in production systems, evaluating against them helps assess whether AI-generated payloads can adapt to practical filtering mechanisms beyond local rule-based setups. At the same time, using free configurations enables rapid experimentation, repeated testing, and UI-driven interaction without requiring expensive enterprise subscriptions. This is important for LLMShield because the system is intended to support an iterative workflow in which users can generate payloads, observe WAF responses, and refine defensive rules through a clear web interface.

In addition to commercial WAFs, the benchmark also includes two open-source WAFs: ModSecurity and NAXSI. ModSecurity is selected as the baseline open-source WAF because it is widely adopted and commonly used together with the OWASP Core Rule Set (CRS), a standardized set of rules for detecting common web attacks. This makes ModSecurity a suitable reference point for evaluating whether generated payloads can bypass a rule-based WAF that follows well-known community security

practices. Its rule transparency also makes it useful for analyzing why a request is blocked and for validating whether LLMShield can generate meaningful defensive rules.

NAXSI is included as the most challenging open-source WAF in this benchmark because it is specifically designed to detect and block common web injection attacks, particularly SQL injection and cross-site scripting. Unlike generic filtering systems, NAXSI applies a strict rule-based security model that focuses on suspicious characters, patterns, and request structures frequently used in SQLi and XSS payloads. Since these two attack categories are also the main focus of the benchmark, NAXSI provides a stricter evaluation target for measuring both the bypass capability and the defense-generation capability of LLMShield. By combining Cloudflare, AWS WAF, ModSecurity, and NAXSI, the benchmark covers a diverse range of WAF designs, from managed cloud services to configurable open-source engines.

6.2.2 Benchmark Implementation

Hosting Infrastructure. The benchmark infrastructure was deployed on Amazon EC2 in order to provide an isolated and reproducible environment for evaluating different WAF configurations. In this implementation, five virtual machines were used in total. Four EC2 instances were provisioned using the `t3.micro` instance type, which is suitable for lightweight benchmark services and can be operated under the free-tier model with up to 720 hours per month during the first year. These instances were used to host the main benchmark components, including the vulnerable application environment and the self-hosted WAF deployments. The use of low-cost EC2 instances makes the benchmark practical for iterative testing, since LLMShield requires repeated payload generation, request execution, and defense-rule validation during development.

A dedicated domain, `llmshield.click`, was also registered to provide a realistic DNS-based routing setup. Cloudflare was configured as the DNS provider and edge proxy for the benchmark domain. This allowed the benchmark to test both proxied and non-proxied traffic paths. The root domain was routed through Cloudflare proxy mode, enabling the evaluation of Cloudflare WAF behavior. Other subdomains were configured as DNS-only records so that requests could be routed directly to the corresponding EC2

instances or AWS WAF endpoint. This setup provides clear separation between different WAF targets while maintaining a unified domain namespace for the benchmark.

Table 6.2 summarizes the DNS records used in the benchmark. Each record maps a benchmark hostname to a specific WAF deployment or backend resource. The root domain `llmshield.click` is configured as a proxied Cloudflare entry point. The `modsec` and `naxsi` subdomains point directly to EC2 instances running Nginx with ModSecurity and NAXSI, respectively. The `aws` subdomain is configured as a CNAME record pointing to the AWS WAF deployment through an Application Load Balancer. This DNS layout enables LLMSHield to switch between WAF targets by changing only the benchmark URL, while keeping the payload generation and evaluation workflow unchanged.

Table 6.2: DNS and infrastructure mapping for the benchmark environment.

DNS Name	Record Type	Target / Content	Proxy Status	WAF Type
<code>llmshield.click</code>	A	<code>160.187.246.104</code>	Proxied	Cloudflare WAF
<code>modsec.llmshield.click</code>	A	<code>3.27.76.103</code>	DNS only	ModSecurity WAF
<code>naxsi.llmshield.click</code>	A	<code>13.238.200.26</code>	DNS only	NAXSI WAF
<code>aws.llmshield.click</code>	CNAME	<code>aws-waf-1135774-492.ap-southeast-1.elb.amazonaws.com</code>	DNS only	AWS WAF

In practice, this implementation allows the benchmark to evaluate WAF behavior under multiple deployment models. Cloudflare represents an edge-proxy WAF where traffic is filtered before reaching the origin server. AWS WAF represents a managed cloud WAF integrated with load-balancing infrastructure. ModSecurity and NAXSI represent self-hosted reverse-proxy WAFs deployed close to the application layer. By hosting these components on separate EC2 instances and exposing them through dedicated DNS records, the benchmark ensures that each WAF can be tested independently while sharing the same vulnerable application target and evaluation methodology.

WAF Configurations. Cloudflare WAF was configured through the Cloudflare dashboard using the built-in security configuration interface. Since Cloudflare operates as an edge proxy, the benchmark domain was first routed through Cloudflare by enabling proxy mode for the root DNS record. After that, WAF behavior was controlled mainly through the *Security Rules* and *Custom Rules* sections. These rules allow the benchmark to define filtering conditions based on request attributes such as URI path, query string, HTTP method, headers, and payload patterns. This configuration model is suitable for the benchmark because it allows WAF policies to be updated quickly through a web interface without modifying the backend server.

AWS WAF was configured in front of the benchmark application through an Application Load Balancer (ALB). In this setup, incoming requests are first routed to the ALB, where AWS WAF inspects the traffic before forwarding allowed requests to the protected EC2 resource. The WAF configuration was managed through AWS WAF Manager, where a web ACL was created and associated with the ALB. The web ACL can contain managed rule groups as well as custom rules for detecting malicious request patterns. This deployment represents a common cloud-native WAF architecture, where traffic filtering is integrated into the AWS networking and load-balancing layer.

For ModSecurity, the WAF was deployed as a self-hosted reverse proxy using Nginx and ModSecurity inside a Docker-based environment. The benchmark used the OWASP Core Rule Set (CRS) as the baseline rule set, since CRS provides a standardized collection of detection rules for common web attacks such as SQL injection and cross-site scripting. Additional custom rules generated during the defense phase were mounted directly into the Docker Compose configuration as external rule files. This approach makes the rule configuration transparent and reproducible, because the active ModSecurity policy can be controlled by modifying the mounted rule files and restarting the corresponding container.

NAXSI was also deployed with Nginx as a reverse-proxy WAF in the Docker environment. Similar to the ModSecurity setup, NAXSI rule files were mounted directly through Docker Compose so that custom rules could be added or replaced during the benchmark. NAXSI relies on a strict rule-based detection model, particularly for SQL injection and cross-site scripting patterns, which makes its configuration

important for evaluating difficult bypass cases. By mounting custom NAXSI rules into the container, the benchmark can repeatedly test generated defense rules under a consistent and isolated configuration while keeping the deployment simple and reproducible.

6.3 Testing Conduction

In this section, we describe the execution of testing activities based on the benchmark and evaluation methodology defined in Sections 6.1 and 6.2. The objective of this phase is to validate the functional correctness of system components and to evaluate the effectiveness of the proposed workflow in detecting Web Application Firewalls (WAFs), generating bypass payloads using Large Language Models (LLMs), and improving WAF protection through automated rule generation.

6.3.1 Unit Testing of System Components

Each component listed in Section 6.1 was tested in isolation before integration.

The CLI was tested by running each subcommand (`detect`, `generate`, `attack`) with valid inputs, missing required flags, and invalid flag combinations to confirm that the tool enforces argument constraints and produces consistent output. The GUI was tested through manual interaction, covering form submission, loading state display, result table rendering, and bypass payload highlighting, verifying that each UI action correctly triggers the corresponding backend call and reflects the response.

Beyond the two interfaces, the remaining tests cover backend input validation, the WAF detector module, LLM adapters for both Red Team (Phase 1 and Phase 3) and Blue Team (GPT-4o), and the three defense sub-modules: standalone pipeline, payload clustering, and RAG retrieval. Results for all ten components are reported in Table 6.3.

Table 6.3: Unit Testing Checklist

Component	Test Objective	Result
Domain input validation	Parse domain format, normalise scheme, return error for invalid input.	Passed
Attack type selection	SQLi/XSS/OS_INJECTION mapped to correct generation pipeline.	Passed
WAF detector module	Return waf_info with name and fingerprint; handle timeouts gracefully.	Passed
CLI option verification	Enforce required flags, apply defaults, reject invalid combinations.	Passed
GUI request/response	Form submission, loading states, result tables, and bypass highlighting work correctly.	Passed
Fine-tuned LLM (Red Team)	Phase 1 and Phase 3 adapters return valid payloads for the given attack type.	Passed
API LLM (Blue Team)	GPT-4o returns JSON with rule and instructions for bypassed payloads.	Passed
Standalone defense pipeline	generate_defense_rules() returns valid rules with payloads; returns FAILED on empty input.	Passed
Payload clustering module	TF-IDF, SVD, HDBSCAN and HAC each assign one label per payload; noise points fall back to single cluster.	Passed
RAG retrieval and enhancement	Non-empty sources inject reference markers; empty sources return base prompt; network errors fall back silently.	Passed

6.3.2 Use Case Testing: Payload Generation and WAF Bypass

In this use case, the system follows the defined workflow in which the LLM first identifies the presence and characteristics of the WAF. Based on the detected WAF behavior, the payload generation module uses the locally trained LLM to generate candidate attack payloads targeting SQL Injection and Cross-Site Scripting

vulnerabilities. Each generated payload is automatically executed against the protected application, and the system records whether the payload is blocked or successfully bypasses the WAF based on the observed HTTP responses.

The objective of this test is to evaluate the effectiveness of the LLM-generated payloads across the four WAF solutions that were fully executed in the experiment, namely ModSecurity, Naxsi, Cloudflare, and AWS WAF. SafeLine was prepared in the benchmark design but its execution results were still pending at the time of writing, and is therefore excluded from the visual analysis in this subsection. For each WAF and each attack category, 50 payloads were executed in Phase 1 and Phase 3. The following figures visualize the outcome distribution between blocked and bypassed payloads, while also separating safe and harmful samples in order to show both bypass capability and filtering precision.

Table 6.4: Payload Generation and WAF Bypass Results Across WAF Solutions

WAF	Attack Type	Phase	Bypassed	B-Safe	B-Harm	Blocked	BK-Safe	BK-Harm
ModSecurity	XSS (DOM)	Phase 1	9 (18%)	4 (8%)	5 (10%)	41 (82%)	6 (12%)	35 (70%)
ModSecurity	XSS (DOM)	Phase 3	35 (70%)	17 (34%)	18 (36%)	15 (30%)	10 (20%)	5 (10%)
ModSecurity	XSS (Refl.)	Phase 1	7 (14%)	4 (8%)	3 (6%)	43 (86%)	9 (18%)	34 (68%)
ModSecurity	XSS (Refl.)	Phase 3	42 (84%)	16 (32%)	26 (52%)	8 (16%)	3 (6%)	5 (10%)
ModSecurity	XSS (Stored)	Phase 1	38 (76%)	7 (14%)	31 (62%)	12 (24%)	0 (0%)	12 (24%)
ModSecurity	XSS (Stored)	Phase 3	44 (88%)	17 (34%)	27 (54%)	6 (12%)	6 (12%)	0 (0%)
ModSecurity	SQLi	Phase 1	18 (36%)	7 (14%)	11 (22%)	32 (64%)	5 (10%)	27 (54%)
ModSecurity	SQLi	Phase 3	45 (90%)	19 (38%)	26 (52%)	5 (10%)	0 (0%)	5 (10%)
ModSecurity	SQLi (Blind)	Phase 1	24 (48%)	17 (34%)	7 (14%)	26 (52%)	6 (12%)	20 (40%)
ModSecurity	SQLi (Blind)	Phase 3	42 (84%)	26 (52%)	16 (32%)	8 (16%)	0 (0%)	8 (16%)
Naxsi	XSS (DOM)	Phase 1	1 (2%)	1 (2%)	0 (0%)	49 (98%)	7 (14%)	42 (84%)
Naxsi	XSS (DOM)	Phase 3	40 (80%)	5 (10%)	35 (70%)	10 (20%)	7 (14%)	3 (6%)
Naxsi	XSS (Refl.)	Phase 1	1 (2%)	1 (2%)	0 (0%)	49 (98%)	6 (12%)	43 (86%)
Naxsi	XSS (Refl.)	Phase 3	42 (84%)	11 (22%)	31 (62%)	8 (16%)	8 (16%)	0 (0%)
Naxsi	XSS (Stored)	Phase 1	0 (0%)	0 (0%)	0 (0%)	50 (100%)	10 (20%)	40 (80%)
Naxsi	XSS (Stored)	Phase 3	34 (68%)	7 (14%)	27 (54%)	16 (32%)	5 (10%)	11 (22%)
Naxsi	SQLi	Phase 1	1 (2%)	0 (0%)	1 (2%)	49 (98%)	20 (40%)	29 (58%)
Naxsi	SQLi	Phase 3	26 (52%)	6 (12%)	20 (40%)	24 (48%)	13 (26%)	11 (22%)
Naxsi	SQLi (Blind)	Phase 1	1 (2%)	1 (2%)	0 (0%)	49 (98%)	12 (24%)	37 (74%)
Naxsi	SQLi (Blind)	Phase 3	19 (38%)	5 (10%)	14 (28%)	31 (62%)	11 (22%)	20 (40%)
Cloudflare	XSS (DOM)	Phase 1	9 (18%)	4 (8%)	5 (10%)	41 (82%)	4 (8%)	37 (74%)

Continued on next page

Table 6.4 – continued from previous page

WAF	Attack Type	Phase	Bypassed	B-Safe	B-Harm	Blocked	BK-Safe	BK-Harm
Cloudflare	XSS (DOM)	Phase 3	45 (90%)	20 (40%)	25 (50%)	5 (10%)	1 (2%)	4 (8%)
Cloudflare	XSS (Refl.)	Phase 1	12 (24%)	2 (4%)	10 (20%)	38 (76%)	2 (4%)	36 (72%)
Cloudflare	XSS (Refl.)	Phase 3	40 (80%)	17 (34%)	23 (46%)	10 (20%)	2 (4%)	8 (16%)
Cloudflare	XSS (Stored)	Phase 1	41 (82%)	7 (14%)	34 (68%)	9 (18%)	2 (4%)	7 (14%)
Cloudflare	XSS (Stored)	Phase 3	45 (90%)	11 (22%)	34 (68%)	5 (10%)	3 (6%)	2 (4%)
Cloudflare	SQLi	Phase 1	14 (28%)	8 (16%)	6 (12%)	36 (72%)	7 (14%)	29 (58%)
Cloudflare	SQLi	Phase 3	41 (82%)	13 (26%)	28 (56%)	9 (18%)	4 (8%)	5 (10%)
Cloudflare	SQLi (Blind)	Phase 1	15 (30%)	11 (22%)	4 (8%)	35 (70%)	8 (16%)	27 (54%)
Cloudflare	SQLi (Blind)	Phase 3	45 (90%)	13 (26%)	32 (64%)	5 (10%)	0 (0%)	5 (10%)
AWS WAF	XSS (DOM)	Phase 1	12 (24%)	3 (6%)	9 (18%)	38 (76%)	5 (10%)	33 (66%)
AWS WAF	XSS (DOM)	Phase 3	42 (84%)	21 (42%)	21 (42%)	8 (16%)	4 (8%)	4 (8%)
AWS WAF	XSS (Refl.)	Phase 1	11 (22%)	2 (4%)	9 (18%)	39 (78%)	3 (6%)	36 (72%)
AWS WAF	XSS (Refl.)	Phase 3	45 (90%)	16 (32%)	29 (58%)	5 (10%)	3 (6%)	2 (4%)
AWS WAF	XSS (Stored)	Phase 1	40 (80%)	9 (18%)	31 (62%)	10 (20%)	1 (2%)	9 (18%)
AWS WAF	XSS (Stored)	Phase 3	49 (98%)	17 (34%)	32 (64%)	1 (2%)	1 (2%)	0 (0%)
AWS WAF	SQLi	Phase 1	50 (100%)	14 (28%)	36 (72%)	0 (0%)	0 (0%)	0 (0%)
AWS WAF	SQLi	Phase 3	50 (100%)	10 (20%)	40 (80%)	0 (0%)	0 (0%)	0 (0%)
AWS WAF	SQLi (Blind)	Phase 1	50 (100%)	15 (30%)	35 (70%)	0 (0%)	0 (0%)	0 (0%)
AWS WAF	SQLi (Blind)	Phase 3	50 (100%)	21 (42%)	29 (58%)	0 (0%)	0 (0%)	0 (0%)

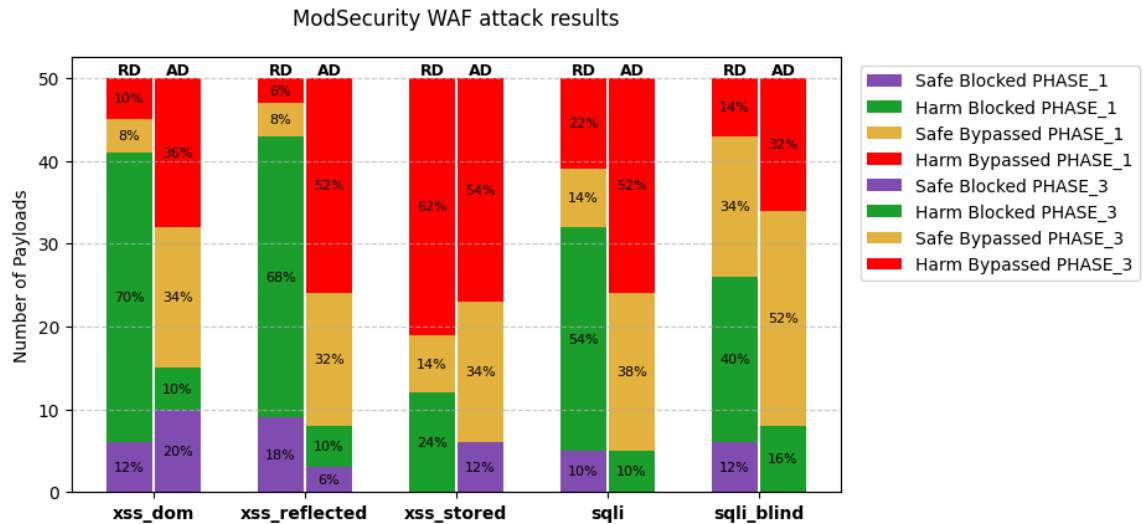


Figure 6.2: Visualization of payload execution results against ModSecurity across attack categories and generation phases

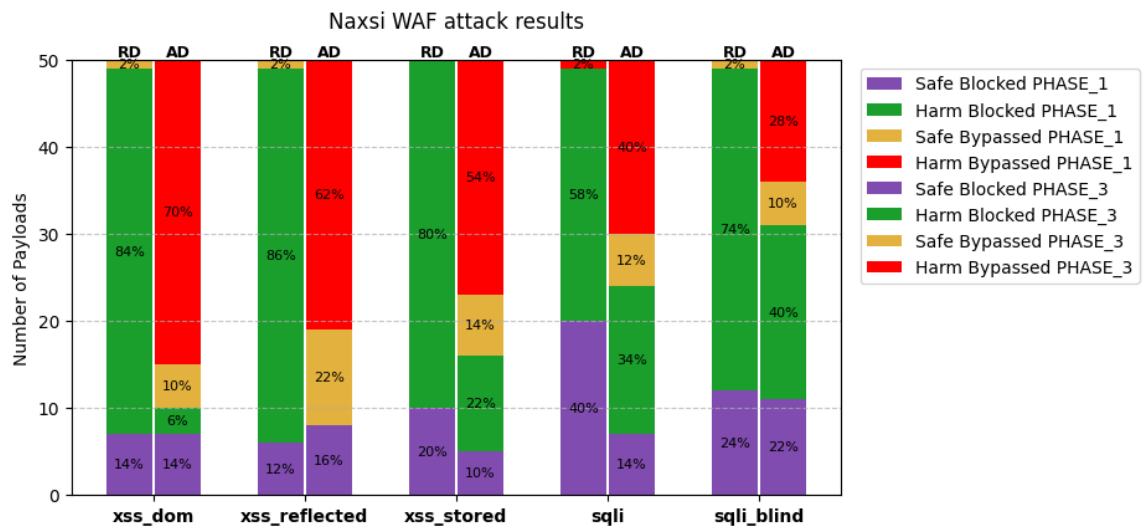


Figure 6.3: Visualization of payload execution results against Naxsi across attack categories and generation phases

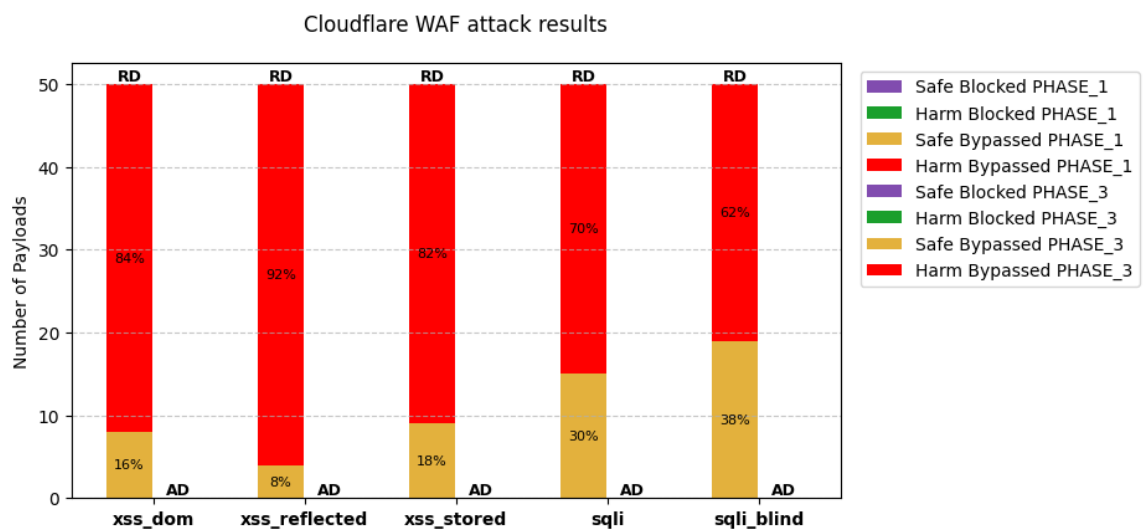


Figure 6.4: Visualization of payload execution results against Cloudflare across attack categories and generation phases

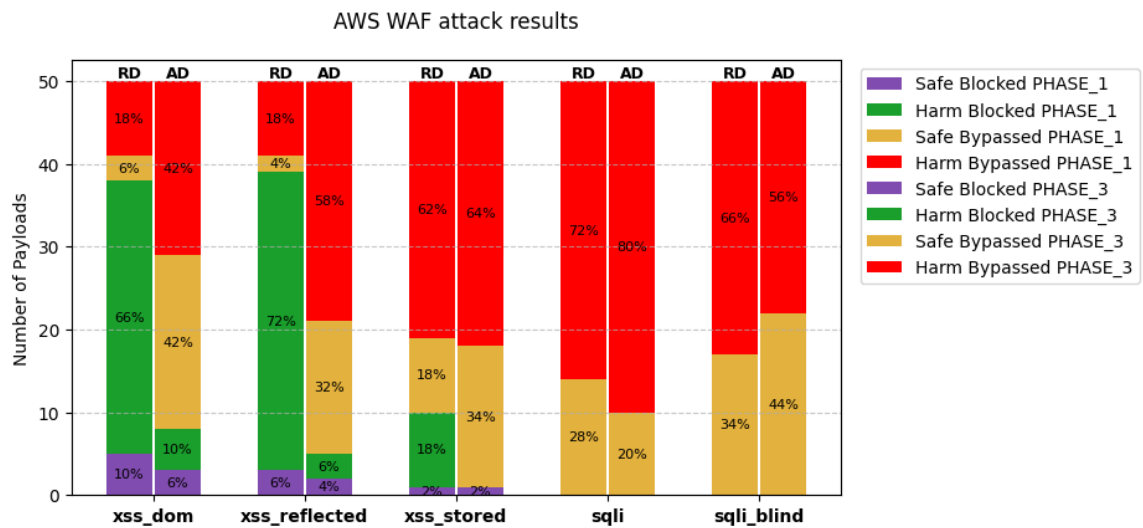


Figure 6.5: Visualization of payload execution results against AWS WAF across attack categories and generation phases

Figures 6.2 to 6.5 show that Phase 3 consistently improves attack effectiveness over Phase 1. The increase is clear on ModSecurity, where bypass rises from 18% to 70% for DOM XSS, from 14% to 84% for reflected XSS, and from 36% to 90% for SQLi. Naxsi shows the largest gap between the two phases, with DOM XSS increasing from 2% to 80% and reflected XSS from 2% to 84%.

Cloudflare and AWS WAF are weaker targets overall. On Cloudflare, stored XSS already reaches 82% bypass in Phase 1 and rises to 90% in Phase 3, while SQLi and blind SQLi increase from 28% to 82% and from 30% to 90%, respectively. AWS WAF is the most exposed to SQL injection, with both SQLi and blind SQLi remaining at 100% bypass in both phases. Its XSS results also worsen in Phase 3, rising from 24% to 84% for DOM XSS, from 22% to 90% for reflected XSS, and from 80% to 98% for stored XSS.

Overall, the figures show that Phase 3 does not only increase bypass count, but also improves the ability of generated payloads to remain practically exploitable after validation. ModSecurity and Naxsi retain stronger blocking behavior in Phase 1, whereas Cloudflare and especially AWS WAF expose broader weaknesses across multiple attack types.

6.3.3 Use Case Testing: Rule Generation and WAF Improvement

This subsection presents the results of the defense pipeline, which takes bypassed payloads as input and produces WAF-specific defense rules through a four-stage process: payload clustering, RAG-enhanced prompt construction, LLM-based rule generation with syntax validation, and rule refinement.

Payload Clustering. Before generating rules, the pipeline groups bypassed payloads into clusters based on structural similarity using TF-IDF vectorization with character-level n-grams and Hierarchical Agglomerative Clustering. This step ensures that the LLM receives semantically grouped payloads rather than a flat, unstructured list, which improves rule coverage and reduces redundancy. Table 6.5 summarizes the clustering results for each WAF and attack type combination.

Table 6.5: Number of bypass payload clusters per WAF and attack type.

WAF	Attack Type	Bypassed Payloads	Clusters
Naxsi	XSS (DOM)	40	6
Cloudflare	SQLi (Blind)	50	8
AWS WAF	SQLi	50	5
ModSecurity	XSS (Stored)	35	4

For example, in the Naxsi XSS case, the 40 bypassed payloads were divided into 6 clusters: Cluster #4 contained 4 SVG-based payloads using Cyrillic homoglyphs with onload event handlers, Cluster #2 contained 10 iframe-based payloads with fullwidth percent encoding, Cluster #5 contained 9 script-based payloads, and the remaining clusters grouped payloads by their use of img tags, CJK obfuscation characters, and JavaScript URI schemes. This clustering allows the LLM to analyze each group independently and generate targeted rules for each evasion strategy.

RAG-Enhanced Prompt Construction. After clustering, the pipeline constructs a defense prompt and enriches it using Retrieval-Augmented Generation (RAG). The base prompt provides the LLM with the WAF name, the clustered bypass payloads, and

WAF-specific syntax constraints. RAG then retrieves relevant documentation from the knowledge base to supply concrete implementation references.

For each defense request, the RAG module generates four retrieval queries derived from the attack type and target WAF. For instance, when defending Naxsi against XSS, the queries are:

1. *XSS defense strategies and prevention for Naxsi*
2. *Naxsi XSS WAF rules syntax and configuration*
3. *XSS bypass techniques against Naxsi*
4. *Naxsi security rules best practices XSS*

These queries retrieve an initial pool of documents (typically 47–53 documents), which are then filtered down to the top 5 most relevant sources. The retrieved sources are injected into the prompt under a dedicated section titled **Knowledge Base References for Rule Generation**, instructing the LLM to prioritize WAF-specific syntax, operators, and concrete examples from the references. An example of the enrichment structure is shown below:

Before RAG: The base prompt contains the WAF name, clustered bypass payloads, rule generation instructions, and WAF-specific syntax constraints (e.g., for Naxsi: use only `MainRule`, `BasicRule`, `CheckRule` directives; do not output `ModSecurity` syntax).

After RAG: The prompt is appended with a knowledge base section containing retrieved references such as *Naxsi Rules* documentation (rule syntax, score format, match zone definitions) and *XSS Survey* literature (mitigation techniques and detection methods), followed by the instruction: “*Use the references above as implementation evidence for writing WAF rules.*”

Rule Generation, Validation, and Refinement. The enriched prompt is sent to the LLM, which generates candidate defense rules in the target WAF syntax. Table 6.6 summarizes the rule generation pipeline results using GPT-5.4 as the generation model.

The pipeline produces between 8 and 23 candidate rules per WAF depending on the number of clusters and attack complexity. The syntax validation stage verifies that each

Table 6.6: Defense rule generation pipeline results (GPT-5.4).

WAF	Attack Type	Generated	Valid	After Refine	Duplicates
Naxsi	XSS (DOM)	23	22	21	21
Cloudflare	SQLi (Blind)	14	12	10	4
AWS WAF	SQLi	8	4	4	0
ModSecurity	XSS (Stored)	10	10	4	7

generated rule conforms to the target WAF format. Invalid rules are retried once with corrective feedback. The refinement stage then removes duplicate or redundant rules by comparing against the existing rule base. For Naxsi, 21 out of 22 valid rules were retained after removing duplicates with the existing 46 core rules. For Cloudflare, 4 duplicates were removed, leaving 10 production-ready rules. For ModSecurity, the refinement stage aggressively consolidated 10 valid rules down to 4 by merging overlapping detection patterns with the existing OWASP CRS rules.

6.3.4 End to End Testing: Full Pipeline

In this use case, the system enters the Blue Team stage after the offensive workflow has finished. More specifically, only payloads that were simultaneously validated as harmful and observed to bypass the target WAF are forwarded to the defend pipeline. Within this stage, the system does not simply ask an external LLM to write ad hoc signatures. Instead, the bypassed payloads are first grouped by structural similarity, then passed to the rule-generation module together with WAF-specific constraints so that the generated rules follow the syntax and enforcement semantics of the corresponding platform. After that, the candidate rules are checked by the syntax validator and, when existing rules are available, refined further in the advanced-defense setting to reduce duplication and better align the generated rules with the active rule base.

The objective of this test is therefore not only to verify whether the LLM can produce syntactically valid rules, but also to measure whether those generated rules can materially improve the blocking performance of the WAF against the exact payload families that had previously succeeded. For each WAF and each attack category, the same evaluation set of 50 payloads was executed again after rule deployment. As in the previous subsection,

the following figures separate safe and harmful outcomes and distinguish blocked from bypassed requests, so that the analysis reflects both mitigation strength and the residual attack surface after improvement.

Table 6.7: Rule Generation and WAF Improvement Results Across WAF Solutions

WAF	Attack Type	Setting	Bypassed	B-Safe	B-Harm	Blocked	BK-Safe	BK-Harm
Cloudflare	XSS (DOM)	RD	45 (90%)	7 (14%)	38 (76%)	5 (10%)	1 (2%)	4 (8%)
Cloudflare	XSS (DOM)	AD	42 (84%)	8 (16%)	34 (68%)	8 (16%)	2 (4%)	6 (12%)
Cloudflare	XSS (Refl.)	RD	38 (76%)	3 (6%)	35 (70%)	12 (24%)	1 (2%)	11 (22%)
Cloudflare	XSS (Refl.)	AD	36 (72%)	4 (8%)	32 (64%)	14 (28%)	2 (4%)	12 (24%)
Cloudflare	XSS (Stored)	RD	50 (100%)	9 (18%)	41 (82%)	0 (0%)	0 (0%)	0 (0%)
Cloudflare	XSS (Stored)	AD	48 (96%)	8 (16%)	40 (80%)	2 (4%)	0 (0%)	2 (4%)
Cloudflare	SQLi	RD	9 (18%)	4 (8%)	5 (10%)	41 (82%)	11 (22%)	30 (60%)
Cloudflare	SQLi	AD	7 (14%)	3 (6%)	4 (8%)	43 (86%)	12 (24%)	31 (62%)
Cloudflare	SQLi (Blind)	RD	8 (16%)	5 (10%)	3 (6%)	42 (84%)	14 (28%)	28 (56%)
Cloudflare	SQLi (Blind)	AD	6 (12%)	4 (8%)	2 (4%)	44 (88%)	15 (30%)	29 (58%)
ModSecurity	XSS (DOM)	RD	9 (18%)	4 (8%)	5 (10%)	41 (82%)	6 (12%)	35 (70%)
ModSecurity	XSS (DOM)	AD	35 (70%)	17 (34%)	18 (36%)	15 (30%)	10 (20%)	5 (10%)
ModSecurity	XSS (Refl.)	RD	7 (14%)	4 (8%)	3 (6%)	43 (86%)	9 (18%)	34 (68%)
ModSecurity	XSS (Refl.)	AD	42 (84%)	16 (32%)	26 (52%)	8 (16%)	3 (6%)	5 (10%)
ModSecurity	XSS (Stored)	RD	38 (76%)	7 (14%)	31 (62%)	12 (24%)	0 (0%)	12 (24%)
ModSecurity	XSS (Stored)	AD	44 (88%)	17 (34%)	27 (54%)	6 (12%)	6 (12%)	0 (0%)
ModSecurity	SQLi	RD	18 (36%)	7 (14%)	11 (22%)	32 (64%)	5 (10%)	27 (54%)
ModSecurity	SQLi	AD	45 (90%)	19 (38%)	26 (52%)	5 (10%)	0 (0%)	5 (10%)
ModSecurity	SQLi (Blind)	RD	24 (48%)	17 (34%)	7 (14%)	26 (52%)	6 (12%)	20 (40%)
ModSecurity	SQLi (Blind)	AD	42 (84%)	26 (52%)	16 (32%)	8 (16%)	0 (0%)	8 (16%)
Naxsi	XSS (DOM)	RD	1 (2%)	1 (2%)	0 (0%)	49 (98%)	7 (14%)	42 (84%)
Naxsi	XSS (DOM)	AD	2 (4%)	1 (2%)	1 (2%)	48 (96%)	11 (22%)	37 (74%)
Naxsi	XSS (Refl.)	RD	1 (2%)	0 (0%)	1 (2%)	49 (98%)	6 (12%)	43 (86%)
Naxsi	XSS (Refl.)	AD	3 (6%)	3 (6%)	0 (0%)	47 (94%)	16 (32%)	31 (62%)
Naxsi	XSS (Stored)	RD	0 (0%)	0 (0%)	0 (0%)	50 (100%)	10 (20%)	40 (80%)
Naxsi	XSS (Stored)	AD	0 (0%)	0 (0%)	0 (0%)	50 (100%)	12 (24%)	38 (76%)
Naxsi	SQLi	RD	1 (2%)	0 (0%)	1 (2%)	49 (98%)	20 (40%)	29 (58%)
Naxsi	SQLi	AD	0 (0%)	0 (0%)	0 (0%)	50 (100%)	13 (26%)	37 (74%)
Naxsi	SQLi (Blind)	RD	0 (0%)	0 (0%)	0 (0%)	50 (100%)	13 (26%)	37 (74%)
Naxsi	SQLi (Blind)	AD	3 (6%)	3 (6%)	0 (0%)	47 (94%)	13 (26%)	34 (68%)
AWS WAF	XSS (DOM)	RD	12 (24%)	3 (6%)	9 (18%)	38 (76%)	5 (10%)	33 (66%)
AWS WAF	XSS (DOM)	AD	41 (82%)	21 (42%)	20 (40%)	9 (18%)	3 (6%)	6 (12%)

Continued on next page

Table 6.7 – continued from previous page

WAF	Attack Type	Setting	Bypassed	B-Safe	B-Harm	Blocked	BK-Safe	BK-Harm
AWS WAF	XSS (Refl.)	RD	11 (22%)	2 (4%)	9 (18%)	39 (78%)	3 (6%)	36 (72%)
AWS WAF	XSS (Refl.)	AD	45 (90%)	16 (32%)	29 (58%)	5 (10%)	2 (4%)	3 (6%)
AWS WAF	XSS (Stored)	RD	40 (80%)	9 (18%)	31 (62%)	10 (20%)	1 (2%)	9 (18%)
AWS WAF	XSS (Stored)	AD	49 (98%)	17 (34%)	32 (64%)	1 (2%)	1 (2%)	0 (0%)
AWS WAF	SQLi	RD	50 (100%)	14 (28%)	36 (72%)	0 (0%)	0 (0%)	0 (0%)
AWS WAF	SQLi	AD	50 (100%)	10 (20%)	40 (80%)	0 (0%)	0 (0%)	0 (0%)
AWS WAF	SQLi (Blind)	RD	50 (100%)	17 (34%)	33 (66%)	0 (0%)	0 (0%)	0 (0%)
AWS WAF	SQLi (Blind)	AD	50 (100%)	22 (44%)	28 (56%)	0 (0%)	0 (0%)	0 (0%)

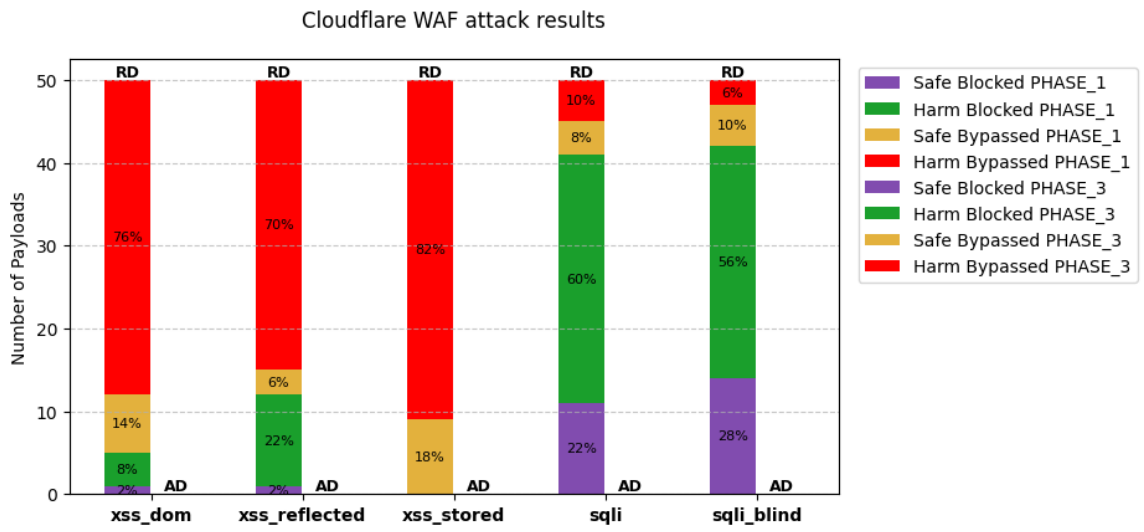


Figure 6.6: Visualization of payload execution results against Cloudflare after rule generation and WAF improvement

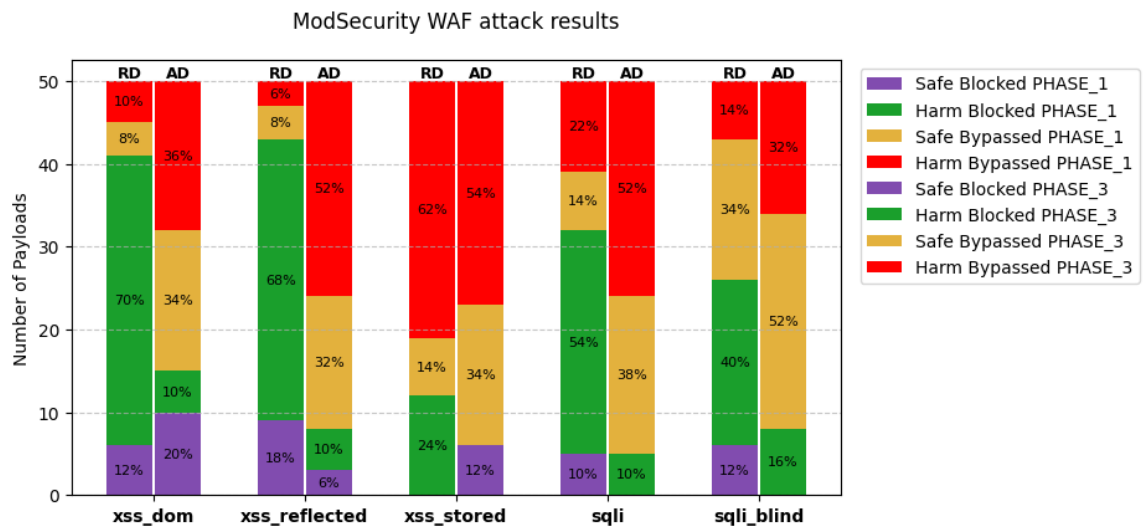


Figure 6.7: Visualization of payload execution results against ModSecurity after rule generation and WAF improvement

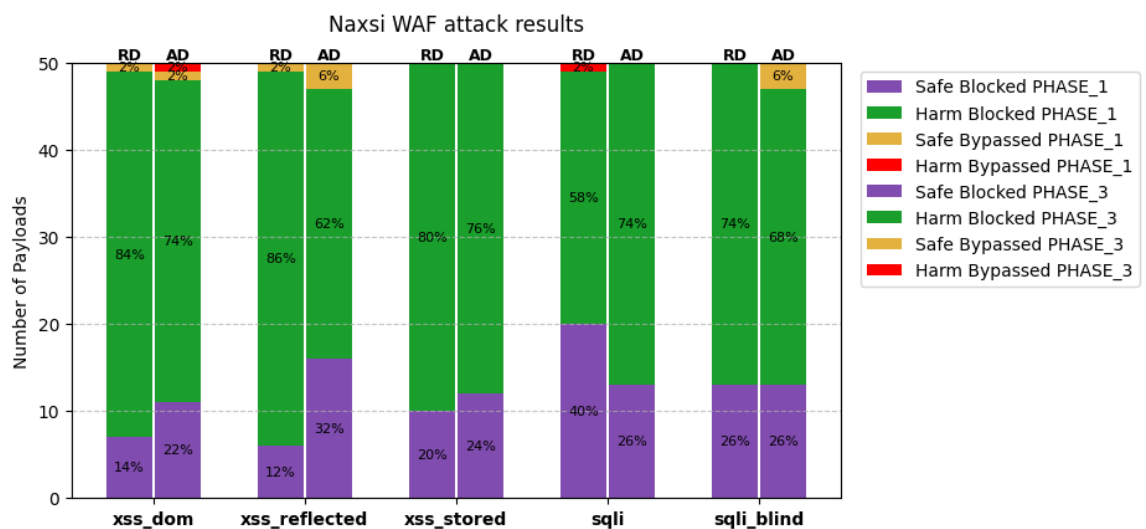


Figure 6.8: Visualization of payload execution results against Naxsi after rule generation and WAF improvement

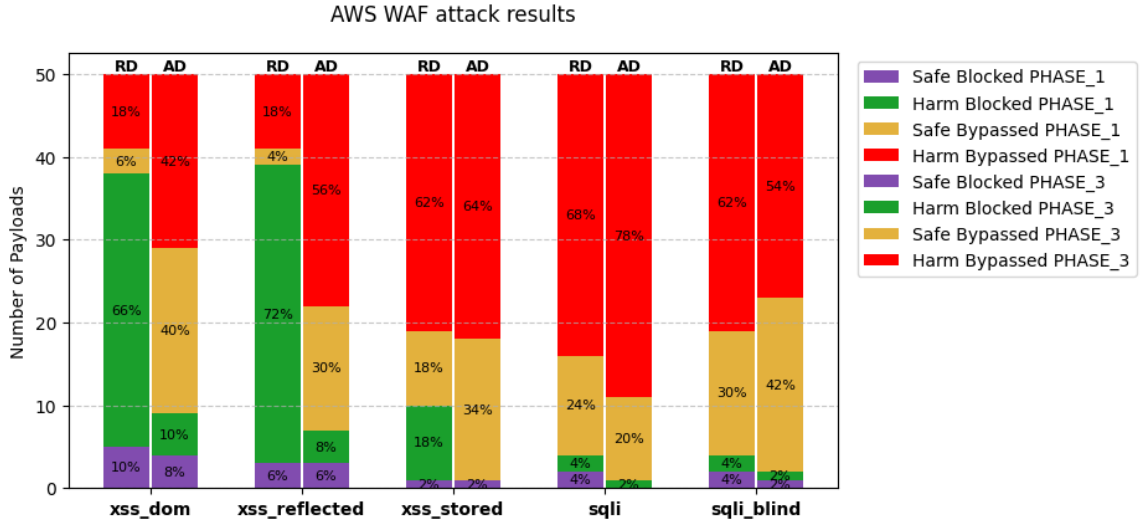


Figure 6.9: Visualization of payload execution results against AWS WAF after rule generation and WAF improvement

Figures 6.6 to 6.9 show that the proposed Blue Team workflow improves WAF protection unevenly across platforms, which is consistent with the implementation itself: the system generates rules under engine-specific constraints, and the final effectiveness depends not only on the LLM output quality but also on how expressive and compatible the target rule language is with the bypass patterns that were observed.

For Cloudflare, the post-improvement behavior remains highly asymmetric between XSS and SQL injection. Under the RD setting, harmful bypass still dominates all three XSS categories, reaching 76% for DOM XSS, 70% for reflected XSS, and 82% for stored XSS. The corresponding harmful blocking rates are only 8%, 22%, and 0%, which indicates that the generated Cloudflare expressions are still unable to generalize sufficiently over the XSS obfuscation patterns previously discovered by the Red Team stage. In contrast, the same workflow is markedly more effective for SQL-related attacks. Harmful blocking rises to 60% for SQLi and 56% for blind SQLi, while harmful bypass is reduced to 10% and 6%, respectively. The safe-bypassed portions remain limited at 8% for SQLi and 10% for blind SQLi, which suggests that the generated Cloudflare rules are able to recover a meaningful amount of protection for injection-style payloads without collapsing entirely into over-blocking. Overall, Cloudflare benefits from the rule-generation stage, but the benefit is concentrated mainly on SQL patterns rather than XSS payload families.

ModSecurity exhibits a more informative contrast between RD and AD. In the RD setting, the generated rules substantially improve defense on several attack classes. Harmful bypass falls to 10% for DOM XSS, 6% for reflected XSS, 62% for stored XSS, 22% for SQLi, and 14% for blind SQLi. At the same time, harmful blocking reaches 70%, 68%, 24%, 54%, and 40%, respectively. These values indicate that the pipeline is particularly effective on DOM XSS, reflected XSS, and the two SQL injection categories, while stored XSS remains much harder to recover after bypass. However, once the experiment moves to the AD setting, the improvement becomes noticeably less stable. Harmful bypass rises again to 36% for DOM XSS, 52% for reflected XSS, 54% for stored XSS, 52% for SQLi, and 32% for blind SQLi, whereas harmful blocking drops to only 10%, 10%, 0%, 10%, and 16%. This means that, for ModSecurity, the rule-generation pipeline works well when it synthesizes direct defensive rules from the harmful bypass set, but the performance degrades when the generated rules must coexist with a larger inherited rule base. In other words, the advanced-defense configuration does not strengthen ModSecurity further in this experiment; instead, it exposes the difficulty of integrating newly generated rules into an already dense CRS-style policy environment.

Naxsi is the clearest success case of the entire mitigation experiment. Under the RD setting, harmful bypass is reduced to 2% for DOM XSS, 0% for reflected XSS, 0% for stored XSS, 2% for SQLi, and 0% for blind SQLi. The corresponding harmful blocking rates remain extremely high at 84%, 86%, 80%, 58%, and 74%. Under the AD setting, the performance remains consistently strong. Harmful bypass is 2% for DOM XSS and 0% for all remaining attack categories, while harmful blocking reaches 74% for DOM XSS, 62% for reflected XSS, 76% for stored XSS, 74% for SQLi, and 68% for blind SQLi. What is important here is not only that Naxsi blocks most harmful payloads after improvement, but also that this behavior is preserved across both RD and AD. Unlike ModSecurity, Naxsi does not show a sharp degradation when the advanced-defense setting is enabled. This strongly suggests that the generated rules are more naturally aligned with Naxsi’s rule model and that the pipeline is able to generalize bypass patterns into deployable mitigation logic with comparatively little interference from the pre-existing rule base.

AWS WAF remains the weakest platform after the defense stage. In the RD setting, harmful bypass is still 18% for DOM XSS, 18% for reflected XSS, 62% for stored XSS, 68% for SQLi, and 62% for blind SQLi. The associated harmful blocking rates are 66%, 72%, 18%, 4%, and 4%, respectively. This already shows that the rule-generation workflow can still help for the simpler XSS categories, but it contributes almost no meaningful recovery for SQL injection. The AD setting confirms this weakness even more clearly. Harmful bypass rises to 42% for DOM XSS, 56% for reflected XSS, 64% for stored XSS, 78% for SQLi, and 54% for blind SQLi, while harmful blocking falls to 10%, 8%, 0%, 2%, and 2%. The safe-bypassed portions also remain high, especially 40% for DOM XSS, 30% for reflected XSS, 34% for stored XSS, 38% for SQLi, and 52% for blind SQLi in the AD setting. Taken together, these results show that the automatically generated AWS WAF rules are not able to recover the defensive posture to the same extent as in Naxsi or even RD-based ModSecurity. The problem is especially severe for SQL injection, where the mitigation stage leaves harmful bypass as the dominant outcome.

From a system perspective, these results are consistent with the architecture of the Blue Team pipeline. Because the workflow clusters harmful bypasses, applies WAF-specific prompting, and attempts rule synthesis under syntax and compatibility constraints, its success depends on how well the target WAF can express generalized detection logic for the observed payload mutations. Naxsi is the platform that benefits most from this process, as both RD and AD maintain very low harmful bypass rates after deployment. Cloudflare shows partial recovery, mainly for SQL injection rather than XSS. ModSecurity achieves useful gains in RD, but the gains are not preserved in AD, indicating that compatibility with existing rules remains a significant challenge. AWS WAF derives the least benefit from the proposed improvement stage, especially against SQL-related bypass families. Therefore, the rule-generation and WAF-improvement use case demonstrates that the proposed Blue Team workflow is technically viable, but its operational effectiveness is strongly dependent on the expressive power of the target rule language and on the interaction between newly generated rules and the WAF's pre-existing policy set.

Chapter 7

Conclusion

Summary

This project has evolved into LLMShield, a unified platform that handles the complete workflow for LLM-assisted WAF assessment and rule generation. In its current form, LLMShield supports an end-to-end security workflow that identifies the target WAF, generates offensive payloads in both initial and adaptive modes, tests them against the target application, and then produces defensive WAF rules through a structured pipeline. Rather than stopping at basic rule suggestion, the defense stage clusters successful bypass payloads, enriches rule generation with retrieved security knowledge, validates the syntax of candidate rules for the target WAF, and applies an additional refinement stage to improve rule quality, reduce duplication, and compare newly generated rules with existing ones. The project scope is also more specific than the earlier high-level description of only SQL Injection and Cross-Site Scripting, as the current implementation explicitly works across multiple concrete attack variants within those families and introduces exploit verification. This allows the evaluation to distinguish between requests that are blocked, requests that merely bypass the WAF, and requests that actually succeed in exploiting the underlying vulnerability. Overall, LLMShield is better characterized as a modular red-team and blue-team experimentation platform for iterative WAF assessment and rule generation than as a single fine-tuned model pipeline with fixed headline performance numbers. More importantly, it is not merely a lab-oriented project designed for trainee engineers to practice firewall tuning and

study different attack techniques. It is also a research-oriented platform developed horizontally across the entire pipeline, providing a foundation on which future studies can build to improve individual processes and adapt them more effectively to specific security contexts, ultimately producing higher-quality and more practically useful results.

Limitations

Despite these improvements, the current version of LLMShield still has several important limitations. First, the offensive workflow should not be understood as a complete penetration testing process. At this stage, the project mainly focuses on generating and obfuscating attack payloads using known bypass techniques in order to test whether a WAF can be evaded. Therefore, a successful bypass does not always carry strong practical security meaning. In many cases, the payload may pass through the firewall, but this does not necessarily prove that the target application can be fully exploited in a realistic attack scenario. This limitation makes the current evaluation more suitable for studying WAF behavior and payload transformation strategies than for representing end-to-end penetration testing. Second, the defensive rule generation process still faces practical deployment challenges. Although the system integrates RAG-based knowledge retrieval and syntax validation, the generated rules can still contain minor encoding, escaping, or formatting issues. These errors may appear small at the textual level, but they can easily lead to incorrect deployment or unexpected behavior in real WAF configurations, especially for ModSecurity and Naxsi. As a result, rule validity cannot be guaranteed only through automatic syntax checking; manual review and real deployment testing are still necessary. Another limitation comes from the evaluation of commercial WAFs. Because the project relies on limited or free-tier access for some commercial firewall services, it cannot fully test all available protection features and rule configuration options. In addition, the high cost of commercial WAF usage restricts the number of rules and experiments that can be tested. In some cases, rules have to be merged into a single rule to reduce cost or fit service constraints, which may reduce detection granularity. Some commercial WAF settings also do not allow

full inspection of the raw request body under the available plan, making it difficult to evaluate rule effectiveness comprehensively. Finally, the project expanded across many features within a limited development period. Because of this, some components have not yet been tested, tuned, and optimized deeply enough for the specific goals of the project. For example, clustering, RAG retrieval, rule refinement, and rule comparison are promising parts of the pipeline, but they still require more systematic testing to confirm whether they actually improve the final output quality. In the current version, these components provide useful structure for the workflow, but their effectiveness should be further validated through controlled experiments, ablation studies, and more detailed evaluation across different attack variants and WAF configurations.

Future Work

Future development can build on the current architecture in several meaningful directions without changing the core idea of the project. One natural extension is to broaden the vulnerability coverage beyond the current SQLi and XSS variants, so that the same workflow can reason about additional web attack classes while preserving the project's current emphasis on exploit-aware evaluation rather than simple request blocking. Another valuable direction is to make the validation loop more realistic by moving beyond DVWA-centered scenarios toward richer targets and browser-aware execution for client-side attacks. This would improve confidence in whether a payload truly succeeds and whether a generated rule meaningfully mitigates the exploit path. The defensive pipeline can also be strengthened by expanding and curating the RAG knowledge base, improving retrieval and reranking quality, and refining the clustering, deduplication, and rule-conflict analysis stages. As a result, the generated rules would not only be valid but also easier to integrate into real WAF configurations. In addition to generating defensive rules, the system could also be extended to generate detailed configuration instructions, making it easier for operators to understand, adjust, and deploy the rules in practical firewall environments. At the systems level, the project has strong potential to grow into a more automated closed-loop platform in which generated rules can be staged, tested, compared against existing policies, and iteratively improved

with less manual intervention. In addition, the training and fine-tuning process of large language models could be improved through more advanced optimization algorithms, especially reinforcement learning methods. This direction could help the LLM generate more effective attack payloads than the current approach, thereby improving both exploit validation quality and the generation of corresponding defensive rules. Another important future direction is to expand access to and experimentation with a wider range of commercial web application firewalls. Evaluating the approach across diverse commercial WAFs would help verify the generalizability of the method, clarify the compatibility of generated rules with real-world protection systems, and increase the practical value of the project in enterprise deployment environments. Finally, the research value of the project would increase substantially with a more reproducible evaluation package that includes standardized datasets, versioned logs and result files, clearer benchmark protocols, and comparative measurements across WAF types, attack variants, and model configurations.

References

- [1] Vahid Babaey and Arun Ravindran. “GenSQLi: A Generative Artificial Intelligence Framework for Automatically Securing Web Application Firewalls Against Structured Query Language Injection Attacks”. In: (2024). URL: <https://doi.org/10.3390/fi17010008>.
- [2] Vahid Babaey and Arun Ravindran. *GenXSS: an AI-Driven Framework for Automated Detection of XSS Attacks in WAFs*. 2025. arXiv: [2504 . 08176](https://arxiv.org/abs/2504.08176) [cs.CR]. URL: <https://arxiv.org/abs/2504.08176>.
- [3] Tran Gia Bao, Dinh Cong Duc, and Phan The Duy. “Generating Evasive Payloads for Assessing Web Application Firewalls with Reinforcement Learning and Pre-trained Language Models”. In: *Journal of Science and Technology on Information Security* 2.CS (25) (2025), pp. 62–79.
- [4] Stefan Cirković et al. “Utilizing Fine-Tuning of Large Language Models for Generating Synthetic Payloads: Enhancing Web Application Cybersecurity through Innovative Penetration Testing Techniques”. In: *Computers, Materials & Continua* 82.3 (2025), pp. 4409–4430. DOI: [10.32604/cmc.2025.059696](https://doi.org/10.32604/cmc.2025.059696).
- [5] Google DeepMind. *Gemini: A Family of Highly Capable Multimodal Models*. arXiv:2312.11805, published December 19, 2023. 2024. URL: <https://arxiv.org/abs/2312.11805>.
- [6] Adel ElZemity, Budi Arief, and Shujun Li. “CyberLLMInstruct: A Pseudo-malicious Dataset Revealing Safety-Performance Trade-offs in Cyber Security LLM Fine-tuning”. In: *AI Sec '25*. arXiv:2503.09334v3. 2025. arXiv: [2503 . 09334v3](https://arxiv.org/abs/2503.09334v3) [cs.CR].

- [7] Mohamed Amine Ferrag et al. *Generative AI in Cybersecurity: A Comprehensive Review of LLM Applications and Vulnerabilities*. arXiv:2405.12750 (v2), 17 Jan 2025. 2025. URL: <https://arxiv.org/abs/2405.12750>.
- [8] Vladimir Karpukhin et al. “Dense Passage Retrieval for Open-Domain Question Answering”. In: *Proceedings of the 2020 Conference on Empirical Methods in Natural Language Processing (EMNLP)*. 2020, pp. 6769–6781. DOI: [10.18653/v1/2020.emnlp-main.550](https://doi.org/10.18653/v1/2020.emnlp-main.550). URL: <https://aclanthology.org/2020.emnlp-main.550/>.
- [9] Sang-Hoon Kim, Sangjin Lee, and Huy Kang Kim. “Attacker group detection method based on HTTP payload analysis”. In: *Journal of Information Processing Systems* 17.3 (2021), pp. 639–651. URL: <https://doaj.org/article/5925939bce414124a7a1aa7e1d3edc1a>.
- [10] Patrick Lewis et al. “Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks”. In: *Advances in Neural Information Processing Systems*. Vol. 33. 2020. URL: <https://proceedings.neurips.cc/paper/2020/hash/6b493230205f780e1bc26945df7481e5-Abstract.html>.
- [11] Manez Moreno et al. “Leveraging LLMs for Automated IDS Rule Generation: A Novel Methodology for Securing Industrial Environments”. In: *Conference/Proceedings title if required by your template*. 2025.
- [12] Mihai Nadăș, Laura Dioșan, and Andreea Tomescu. “Synthetic Data Generation Using Large Language Models: Advances in Text and Code”. In: *IEEE Access* 13 (2025), pp. 134615–134633. ISSN: 2169-3536. DOI: [10.1109/access.2025.3589503](https://doi.org/10.1109/access.2025.3589503). URL: <http://dx.doi.org/10.1109/ACCESS.2025.3589503>.
- [13] Rodrigo Nogueira and Kyunghyun Cho. *Passage Re-ranking with BERT*. arXiv:1901.04085. 2019. URL: <https://arxiv.org/abs/1901.04085>.
- [14] OpenAI. *GPT-4 Technical Report*. arXiv:2303.08774, published March 15, 2023. 2024. URL: <https://arxiv.org/abs/2303.08774>.
- [15] OWASP. *OWASP Top 10: 2025*. Accessed: December 8, 2025. URL: <https://owasp.org/Top10/2025/>.

- [16] Nils Reimers and Iryna Gurevych. “Sentence-BERT: Sentence Embeddings using Siamese BERT-Networks”. In: *Proceedings of the 2019 Conference on Empirical Methods in Natural Language Processing and the 9th International Joint Conference on Natural Language Processing (EMNLP-IJCNLP)*. 2019. DOI: [10.18653/v1/D19-1410](https://doi.org/10.18653/v1/D19-1410). URL: <https://aclanthology.org/D19-1410/>.
- [17] Kaitao Song et al. “MPNet: Masked and Permuted Pre-training for Language Understanding”. In: *Advances in Neural Information Processing Systems*. Vol. 33. 2020. URL: <https://arxiv.org/abs/2004.09297>.
- [18] Trần Ngọc Tín. “Cải thiện tường lửa ứng dụng web bằng các kỹ thuật học máy”. Hội đồng LVTN: Khoa học máy tính. Tập thể hướng dẫn: Nguyễn An Khương, Nguyễn Lê Thành, Nguyễn Văn Hòa. Luận văn tốt nghiệp. Khoa Khoa học và Kỹ thuật Máy tính, Trường Đại học Bách Khoa, Đại học Quốc gia TP. Hồ Chí Minh, Aug. 2020.
- [19] Sentence Transformers. *all-mpnet-base-v2*. Model card. Accessed: April 26, 2026. URL: <https://huggingface.co/sentence-transformers/all-mpnet-base-v2>.
- [20] Sentence Transformers. *Cross-Encoder: ms-marco-MiniLM-L-6-v2*. Model card. Accessed: April 26, 2026. URL: <https://huggingface.co/cross-encoder/ms-marco-MiniLM-L6-v2>.
- [21] Andrea Valenza et al. “WAF-A-MoLE: An adversarial tool for assessing ML-based WAFs”. In: *SoftwareX* 11 (2020), p. 100367. ISSN: 2352-7110. DOI: <https://doi.org/10.1016/j.softx.2019.100367>. URL: <https://www.sciencedirect.com/science/article/pii/S2352711019302997>.
- [22] Wenhui Wang et al. “MiniLM: Deep Self-Attention Distillation for Task-Agnostic Compression of Pre-Trained Transformers”. In: *Advances in Neural Information Processing Systems*. Vol. 33. 2020. URL: https://proceedings.neurips.cc/paper_files/paper/2020/hash/3f5ee243547dee91fbd053c1c4a845aa-Abstract.html.

- [23] Cong Wu et al. *WAFBOOSTER: Automatic Boosting of WAF Security Against Mutated Malicious Payloads*. 2025. arXiv: 2501.14008 [cs.CR]. URL: <https://arxiv.org/abs/2501.14008>.
- [24] Wei Zou et al. *PoisonedRAG: Knowledge Corruption Attacks to Retrieval-Augmented Generation of Large Language Models*. 2024. arXiv: 2402.07867 [cs.CR]. URL: <https://arxiv.org/abs/2402.07867>.